操作系统实验指导书 - 2026秋 | 哈工大（深圳）

> **声明**
>
> 本课程资料仅限于哈尔滨工业大学（深圳）《操作系统》2026 秋季课程实验使用，如有其他用途，请联系夏文老师：xiawen@hit.edu.cn。

> **学术诚信与 AI 使用**
>
> 本课程允许在完成实验的过程中，合理使用大模型、AI Agent、智能 IDE 等工具辅助学习与开发。但请记住：AI 可以是你的助手，但不能是你的替身——它无法替代你对实验方案、代码和结果的理解，也无法替你承担应有的责任。
>
> 请遵守以下要求：
>
> - **独立完成**：实验的关键设计、实现和调试应由本人参与完成；不提交任何自己无法解释、无法复现、无法修改的内容。
> - **留存记录**：建议保留主要的 AI 使用记录；必要时能说明使用了哪些工具、辅助完成了哪些工作，以及你在其基础上做了哪些修改与验证。
> - **经得起验收**：提交与验收时，应能清晰阐述最终代码的设计逻辑与技术细节，独立回答问题，并按要求完成修改或调试；报告中的数据、截图与结论，必须来自你实际完成的实验。
>
> 请始终秉持学术诚信（What is Academic Integrity），以诚实、独立、负责的态度完成每一次实验。把最关键的思考、设计、实现与调试留给自己——唯有真正做到“自己想过、自己做过、自己改过、自己讲得清”，这门课才算没有白修。
>
> 星光不问赶路人，时光不负有心人。愿你在与系统的对话中，遇见那个更强大的自己。

0. 导读

本实验文档为哈尔滨工业大学（深圳）《操作系统》课程实验指导材料。页面顶端为实验指导书，左侧为目录，右侧为当前页面索引，右上角支持全文搜索。

本学期共设置 6 次实验课，每次 4 个学时，总计 24 个学时，包含 5 个实验项目和 2 次现场验收，具体安排见 3. 实验安排。

前 4 个实验项目以开源操作系统 XV6 为基础，并结合课程要求进行了针对性的修改与扩展。第 5 个实验项目为基于 FUSE 的青春版 EXT2 文件系统实验。该实验案例由我校学生自主设计，曾

获全国计算机类课程实验设计一等奖，并有多名学生基于该项目继续开展拓展研究，在 OSDI、FAST、ASPLOS 等 CCF-A 类系统会议发表论文。

由于我们已经对 XV6 的实验内容及要求进行了修改，请大家从下面的“4. 获取 XV6 实验框架代码”所给仓库下载代码，并务必在这套代码上完成实验。使用其他 XV6 仓库代码的同学，极有可能无法通过实验考核。

建议同学们按顺序阅读指导书，先掌握实验平台、Linux 基本命令和调试方法，再进入具体实验内容。对于有兴趣、有余力的同学，也欢迎挑战 Lab2、Lab3 和 Lab5 的附加题，以及 2020 版本的 XV6 实验 中的两个实验：Copy-on-Write 和 mmap；若能完成会有意想不到的惊喜！

为帮助同学们理解 XV6 的整体结构、代码组织以及调试方法，我校 OS 课题组与历届学长助教们录制了 XV6 讲解与调试演示视频 ，推荐按需观看。

如果同学们对本课程实验内容、实验安排、实验指导书或代码框架有意见与建议，也欢迎及时与老师或助教交流。

1. 课程信息

- 课程名称：操作系统（实验）
- 学期：2026 秋季
- 理论课教师：夏文、卢瑶、邹翔宇、陈芳林、陈俊杰、刘川意、陈斌（按任课班级顺序）
- 实验课教师：仇洁婷、苏婷、匡慈维

2. 实验相关链接

- 实验代码下载地址：https://gitee.com/ftutorials/xv6-oslabs-hitsz.git
- 实验课件、报告及工具下载地址（需校内网）：https://file.cs-lab.top/s/xcAKpIRmnrFUOPe
- 实验提交地址（需校内网）：https://grader.hitsz.edu.cn
- 2020 版本的 XV6 实验仓库下载地址：https://gitee.com/ftutorials/xv6-labs-2020.git

3. 实验安排

次数	内容	学时	说明
第 1 次	Lab1：XV6 与 Unix 实用程序	4	含 XV6 启动流程命令行调试

次数	内容	学时	说明
第 2 次	Lab2：进程与系统调用	4	系统调用与进程生命周期
第 3 次	Lab3：锁机制的应用	4	内存分配器与磁盘缓存优化
第 4 次	Lab4：页表；现场验收 Lab1～Lab3	4	页表机制与地址空间管理；代码讲解、问答与必要的现场修改
第 5 次	Lab5：基于 FUSE 的 EXT2 文件系统	4	文件系统设计与实现
第 6 次	现场验收 Lab4～Lab5	4	围绕 Lab4～Lab5 开展代码讲解、问答与必要的现场修改

4. 获取 XV6 实验框架代码

在下载实验代码前，请先搭建好实验平台，搭建方法参考 实验平台 及其后续页面。

如果你已经进入实验环境，请在命令行中执行：

```
git clone https://gitee.com/ftutorials/xv6-oslabs-hitsz.git
```

克隆完成后，将得到 `xv6-oslabs-hitsz` 目录。如遇问题，请联系老师或助教。

5. 视频链接

5.1 XV6 讲解视频

1. 【HITSZ 操作系统课程组讲解 XV6（一）启动过程】
2. 视频链接：https://www.bilibili.com/video/BV1mK411S7N9?share_source=copy_web&vd_source=225a99017e082147ac525beeddd6e3e2
3. 课程 PPT：点这里下载
4. 【HITSZ 操作系统课程组讲解 XV6（二）进程管理】
5. 视频链接：https://www.bilibili.com/video/BV1ge4y1J7Je?share_source=copy_web&vd_source=225a99017e082147ac525beeddd6e3e2
6. 课程 PPT：点这里下载
7. 【HITSZ 操作系统课程组讲解 XV6（三）内存管理】

8. 视频链接：https://www.bilibili.com/video/BV1Te4y1i77z?share_source=copy_web&vd_source=225a99017e082147ac525beeddd6e3e2
9. 课程 PPT：点这里下载

也可以观看 B 站上的 MIT 6.S081/Fall 2020 课程视频 ，帮助你从整体上理解 XV6。

5.2 XV6 调试演示视频

1. 【使用 gdb 调试 xv6 启动流程（1）】 https://www.bilibili.com/video/BV1zAt6e3EfU/?share_source=copy_web&vd_source=a822dcda3537564ccdd0bb45aa0afe33
2. 【使用 gdb 调试 xv6 启动流程（2）】 https://www.bilibili.com/video/BV1MrtUe7ELC/?share_source=copy_web&vd_source=a822dcda3537564ccdd0bb45aa0afe33
3. 【VSCode 调试 xv6 内核代码】 https://www.bilibili.com/video/BV1ZB4y1E7X5?share_source=copy_web&vd_source=a822dcda3537564ccdd0bb45aa0afe33

如果你对本课程实验内容、实验安排、实验指导书或代码框架有宝贵的意见与建议，也欢迎联系老师或助教。

6. 各班级课表

换课说明

- 课表有冲突的同学，可自行对照课表选择合适时段上课，但换课需提前告知老师，避免被误记为缺课。
- 各班人数已接近实验室容量，非必要请勿换课，以防座位不足。
- 若需换课，建议优先选择 T2210 实验室（容量相对较大）的班级。

实验一班

授课教师： 仇洁婷、匡慈维

课次	周次	星期	节次	实验室
1	2	周五	第 5–8 节	T2210
2	4	周三	第 9–12 节	T2210
3	7	周五	第 5–8 节	T2210

课次	周次	星期	节次	实验室
4	9	周五	第 5–8 节	T2210
5	11	周五	第 5–8 节	T2210
6	13	周五	第 5–8 节	T2210

实验二班

授课教师： 仇洁婷

课次	周次	星期	节次	实验室
1	3	周五	第 5–8 节	T2210
2	6	周五	第 5–8 节	T2210
3	8	周五	第 5–8 节	T2210
4	10	周五	第 5–8 节	T2210
5	12	周五	第 5–8 节	T2210
6	14	周五	第 5–8 节	T2210

实验三班

授课教师： 仇洁婷

课次	周次	星期	节次	实验室
1	3	周二	第 5–8 节	T2210
2	4	周二	第 5–8 节	T2210
3	7	周二	第 5–8 节	T2210

课次	周次	星期	节次	实验室
4	9	周五	第 9–12 节	T2210
5	11	周五	第 9–12 节	T2210
6	13	周二	第 9–12 节	T2210

实验四班

授课教师： 苏婷、匡慈维

课次	周次	星期	节次	实验室
1	3	周日	第 1–4 节	T2608
2	6	周六	第 1–4 节	T2608
3	8	周二	第 5–8 节	T2608
4	10	周五	第 9–12 节	T2608
5	12	周五	第 9–12 节	T2608
6	14	周二	第 1–4 节	T2608

实验五班

授课教师： 仇洁婷、匡慈维

课次	周次	星期	节次	实验室
1	3	周日	第 5–8 节	T2210
2	6	周五	第 9–12 节	T2210
3	8	周五	第 9–12 节	T2210

课次	周次	星期	节次	实验室
4	10	周四	第 9–12 节	T2210
5	12	周三	第 5–8 节	T2210
6	14	周一	第 5–8 节	T2210

实验六班

授课教师： 苏婷

课次	周次	星期	节次	实验室
1	3	周二	第 5–8 节	T2608
2	4	周四	第 5–8 节	T2608
3	7	周四	第 5–8 节	T2608
4	9	周四	第 5–8 节	T2608
5	11	周五	第 5–8 节	T2608
6	13	周五	第 5–8 节	T2608

实验七班

授课教师： 苏婷、匡慈维

课次	周次	星期	节次	实验室
1	3	周五	第 5–8 节	T2102
2	6	周五	第 5–8 节	T2102
3	8	周五	第 5–8 节	T2102

课次	周次	星期	节次	实验室
4	10	周五	第 5–8 节	T2102
5	12	周五	第 5–8 节	T2102
6	14	周五	第 5–8 节	T2102

实验八班

授课教师： 苏婷

课次	周次	星期	节次	实验室
1	3	周一	第 5–8 节	T2507
2	5	周一	第 5–8 节	T2507
3	7	周一	第 5–8 节	T2507
4	9	周一	第 5–8 节	T2507
5	11	周一	第 5–8 节	T2507
6	13	周一	第 5–8 节	T2507

致谢

本实验指导书的持续建设，离不开历届本科生与研究生助教的长期投入。无论是实验内容设计、代码框架维护、文档修订，还是环境搭建与问题排查，都凝聚着他们大量细致而扎实的工作。

在此，特别感谢以下参与过课程实验建设的助教同学们（按年级排序）：

- 17级：胡博涵、施杨
- 18级：李程浩、宫浩辰、潘延麒、黎庚祉、刘定邦
- 19级：赵鹏宇、陈一邹、叶自立、张艺枫
- 20级：曾培鑫、陈佳豪、满洋、杨大荣
- 21级：陈睿玮、石全、王华杰、区文灏

- 23级：杨纯懿、曾熙晨

感谢你们将热情与专注留在了这里。

前路漫漫亦灿灿，期待现在的你们接过接力棒，书写更多可能！

实验平台

这页能帮你做到什么

看完这页后，你应该能快速判断： **四种实验环境分别适合什么场景** 、 **自己当前更适合选哪一种** 、以及 **后续应该去看哪篇具体使用指南** 。

如果你现在主要目标是“尽快进入实验”，可以先按下面选择：

- **实验室上课 / 机房使用** ： 优先选 **实验室预装虚拟机**
- **个人电脑长期练习 / 希望离线可用** ： 优先选 **个人电脑虚拟机**
- **临时验证代码 / 不想本地配置环境** ： 优先选 **远程实验平台**
- **想深入理解工具链 / 自定义环境** ： 可以考虑 **自行部署环境**

本课程提供 4 种实验环境方案，同学们可根据自身设备条件、学习需求灵活选择：

对比维度	实验室预装虚拟机	个人电脑虚拟机	远程实验平台	自行部署环境
环境状态	完全现成	镜像导入后可用	完全现成	需手动配置 / 构建
硬件配置	实验室标准配置	依赖个人设备	实验中心服务器	依赖本地 / 自定义
并发性能	无冲突（单机）	无冲突（单机）	差（多用户共享）	无冲突（单机）
数据持久化	临时（关机可能丢失）	持久（本地存储）	临时（服务器清理）	持久（本地 / 自定义）
网络依赖	无	无	必须联网	无（本地部署）
核心优势	省心省力	本地离线可用	快速临时调试	深入理解原理
适用场景	实验室上课	个人离线练习	临时验证代码	进阶学习 / 定制化

1. 实验室预装虚拟机 🖥️

即开即用，详见虚拟机实验环境使用指南

- **环境状态**：Ubuntu 系统 + XV6 工具链 + QEMU 预装完成；
- **使用方式**：开机后打开 VirtualBox，选中 `oslab` 虚拟机点击“启动”，用默认账号（`oslab` / `123456`）登录即可；
- **优势**：零配置成本，适配实验室教学节奏，无需担心设备性能问题；
- **注意事项**：实验代码需及时通过 Git 提交或复制到个人 U 盘，避免关机后数据丢失。

2. 个人电脑虚拟机 💻

本地离线环境，详见虚拟机实验环境使用指南

- **环境状态**：需自行下载 OVA 镜像并导入 VirtualBox；
- **配置要求**：本地磁盘至少 20GB 可用空间，需开启 CPU 虚拟化；
- **使用方式**：
  - 下载 OVA 镜像；
  - 导入 VirtualBox 并启动，登录账号同实验室虚拟机（`oslab` / `123456`）；
- **优势**：本地运行，网络依赖低，可随时调试；
- **注意事项**：个人电脑性能较弱（如 8GB 内存以下）可能出现卡顿，建议优先选择实验室虚拟机或远程实验平台。

3. 远程实验平台 🔗

实验中心服务器端现成环境，详见远程实验平台使用指南

- **环境状态**：服务器端已预装 XV6 完整环境，无需手动配置；
- **使用方式**：通过 VSCode 远程连接实验中心服务器，直接克隆代码运行；
- **优势**：零配置成本，启动速度快，适合临时验证代码；
- **注意事项**：并发性能有限，大量同学同时使用可能出现卡顿、断连；数据仅临时保存，需及时提交 Git 仓库。

4. 自行部署环境 ⚙️

手动配置，详见实验环境搭建指南（自行部署版）

- **实现方式**：在原生 Linux、WSL2 或虚拟机环境中手动安装工具链、编译依赖；

- **优势**：完全自主可控，可自定义工具版本，助力理解交叉编译、调试工具链等底层原理；
- **注意事项**：门槛较高，需具备基础 Linux 命令操作能力，配置过程中可能遇到依赖冲突等问题。

虚拟机实验环境使用指南

> **关于虚拟机实验环境**
>
> 本实验环境基于 **Ubuntu 虚拟机镜像（OVA 格式）** 构建，已预装 XV6 开发所需的全部工具（编译器、调试器、QEMU 等）。

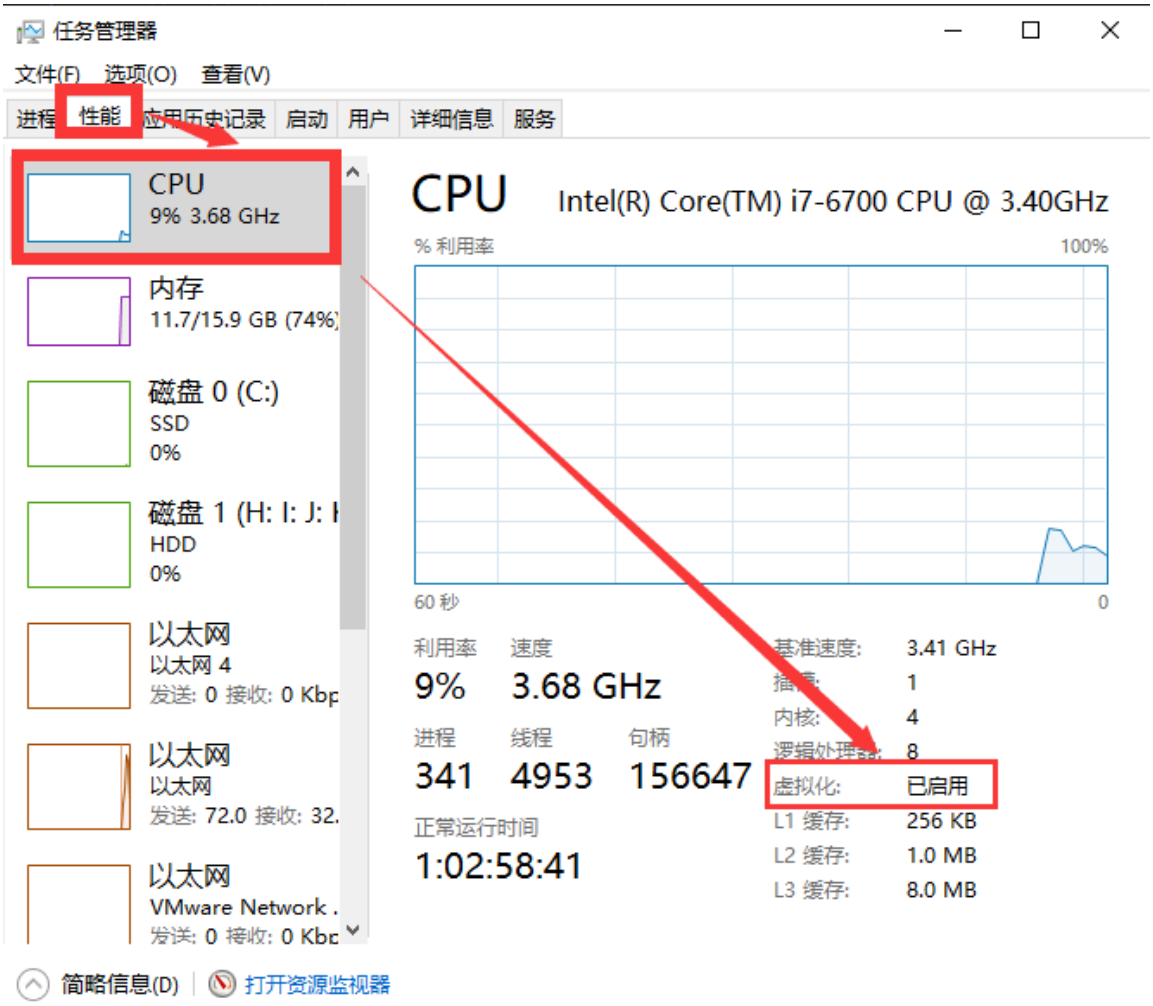
- **实验室同学（T2102/T2210/T2507/T2608）**：实验室电脑已提前导入虚拟机，直接跳过“准备工作”“导入 OVA 文件”，从2. 启动虚拟机与登录开始操作即可；
- **个人电脑使用者**：需按完整步骤下载镜像、导入虚拟机，确保本地磁盘至少有 20GB 可用空间。

1. 准备工作（仅个人电脑使用者需做）

1.1 虚拟化环境检查

⚠️ 必须开启 CPU 虚拟化！否则无法运行虚拟机

- 打开任务管理器 → 性能 → CPU
- 确认 虚拟化 显示 “已启用”

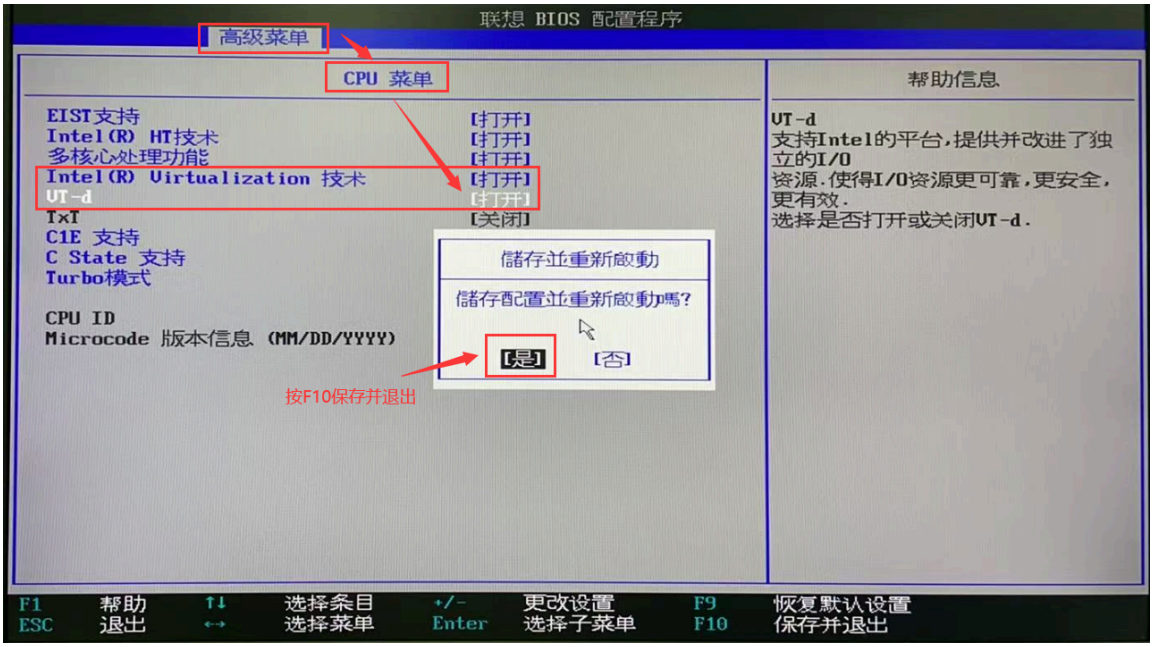


如果虚拟化是禁用，请自行开启CPU虚拟化。

1.2 手动开启 CPU 虚拟化（如未启用）

不同品牌电脑进入 BIOS 的按键不同，以下为通用步骤：

1. 重启电脑，根据电脑品牌按对应按键进入 BIOS（常见按键：联想 / 惠普按 `F1` / `F2`，戴尔按 `F2` / `Del`，华硕按 `Del`，可开机时观察屏幕提示）；
2. 进入 BIOS 后，找到【高级菜单】→【CPU 菜单】（或 “Processor Configuration”）；
3. 启用以下两项功能：
   - “Intel Virtualization Technology”（Intel 芯片）或 “AMD-V”（AMD 芯片）设为 **Enabled**（打开）；
   - “VT-d”（或 “Intel VT-d Technology”）设为 **Enabled** （打开）；
4. 按 `F10` 保存设置并重启电脑。



1.3 安装 VirtualBox 7.1.8

如果你的电脑已安装有VirtualBox，则无需重复安装。

- 下载： 点这里下载实验工具
- 安装： 运行安装程序，按向导完成安装（建议保持默认设置：勾选 “创建桌面快捷方式”“添加到 PATH”）。

1.4 下载 oslab实验环境文件

OVA 镜像约 6GB，已预装所有实验工具，无需额外配置。

- 下载： 点这里下载实验工具

1.5 导入 OVA 文件

- 打开 VirtualBox 双击桌面上的 VirtualBox 图标或从开始菜单中找到并打开它。
- 导入 OVA 文件
- 点击菜单 **管理** → **导入虚拟电脑** 。
- 在“文件”栏，选择下载的 oslab2.ova 文件 → 点击 打开。
- 在“设置”栏，修改“默认虚拟电脑位置”的存储路径（关键！）：
  - 点击 “默认虚拟电脑位置” 旁的文件夹图标 → 选择“其他” → 选择 D/E 盘 下的目录（如 D:\OSLab）。
- 确认配置无误 → 点击 导入，等待进度条完成。

⚠️ 重要提示

- 如果导入失败，检查磁盘空间（需至少 20GB 可用）和文件完整性（重新下载 OVA）。

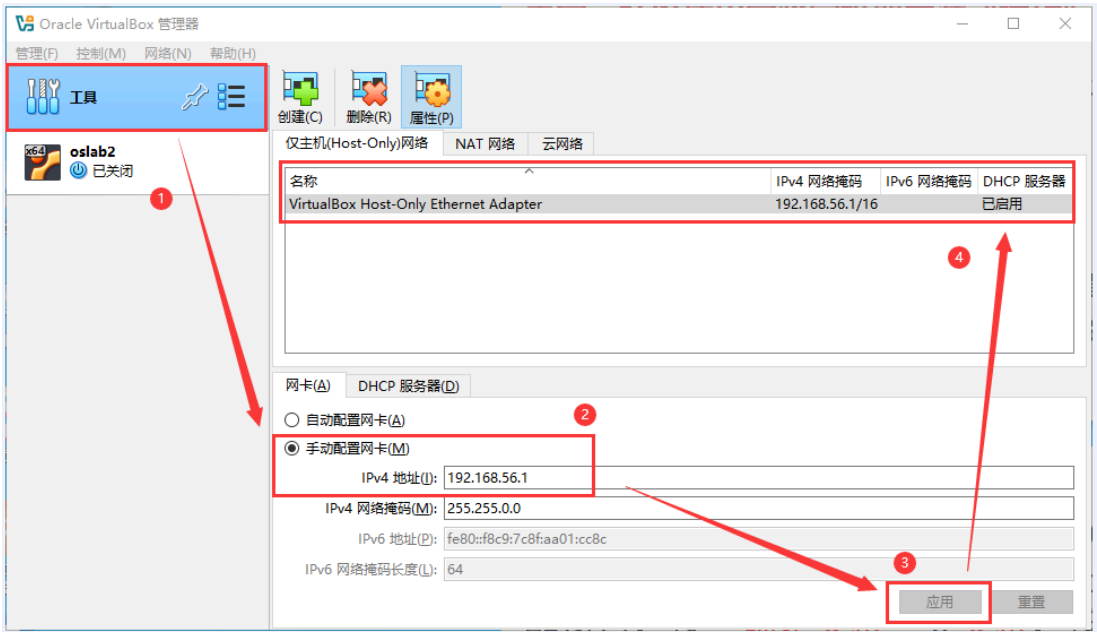
2. 启动虚拟机与登录（所有人必做）

T2102 实验室使用虚拟机前请先配置 Host-only IP

只有在 **T2102 实验室** 使用预装虚拟机时需要执行本提示中的步骤；其他实验室和个人电脑使用者不需要修改。

在启动 `oslab2` 虚拟机之前，请先在 VirtualBox 中把 Host-only 网卡的 IPv4 地址改为 `192.168.56.1`：

1. 打开 VirtualBox 后，先不要启动虚拟机；
2. 点击左侧 **工具** ，进入 **仅主机(Host-Only)网络** 页面；
3. 选中 `VirtualBox Host-Only Ethernet Adapter`；
4. 选择 **手动配置网卡** ，将 **IPv4 地址** 改为 `192.168.56.1`；
5. 点击右下角 **应用** 保存配置。



配置完成后，回到 VirtualBox 主界面，选中 `oslab2` 虚拟机，再点击 **启动** 。

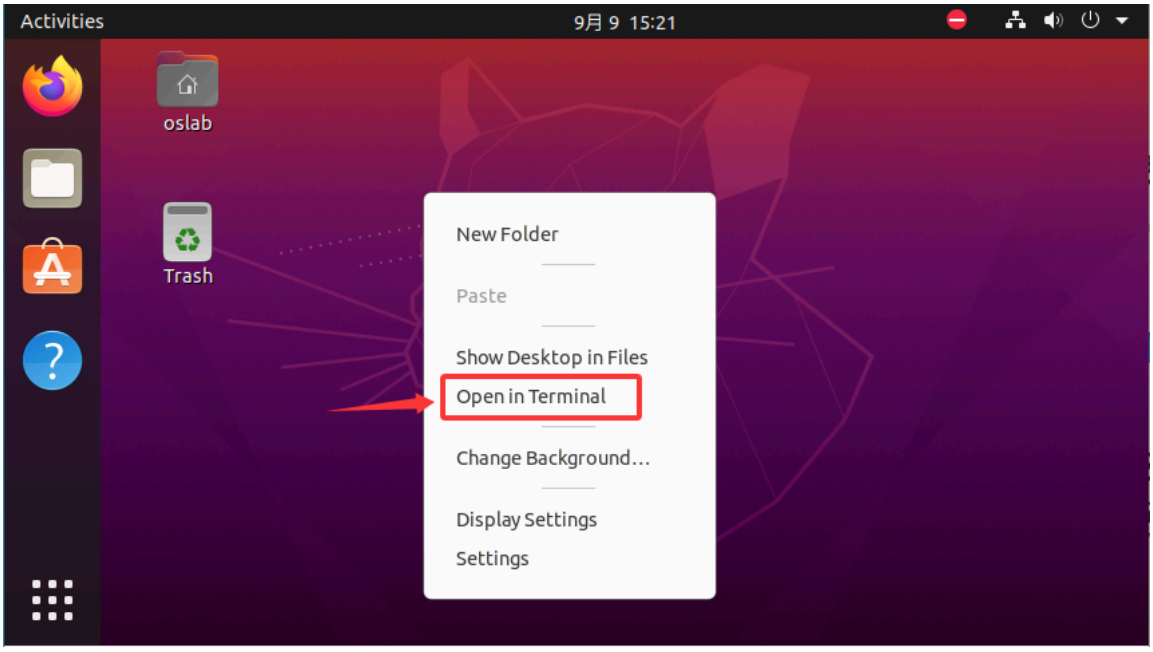


- 启动虚拟机
  - 在 VirtualBox 主界面选中 oslab2 → 点击 启动。
  - 首次启动：等待 1-2 分钟，系统会自动初始化。
- 登录 Ubuntu
  - 用户名： `oslab`
  - 密码： `123456`

2.1 获取虚拟机 IP 地址

为提升开发效率，建议通过 VSCode 远程连接虚拟机（支持图形化编辑、调试），需先获取虚拟机 IP 地址。

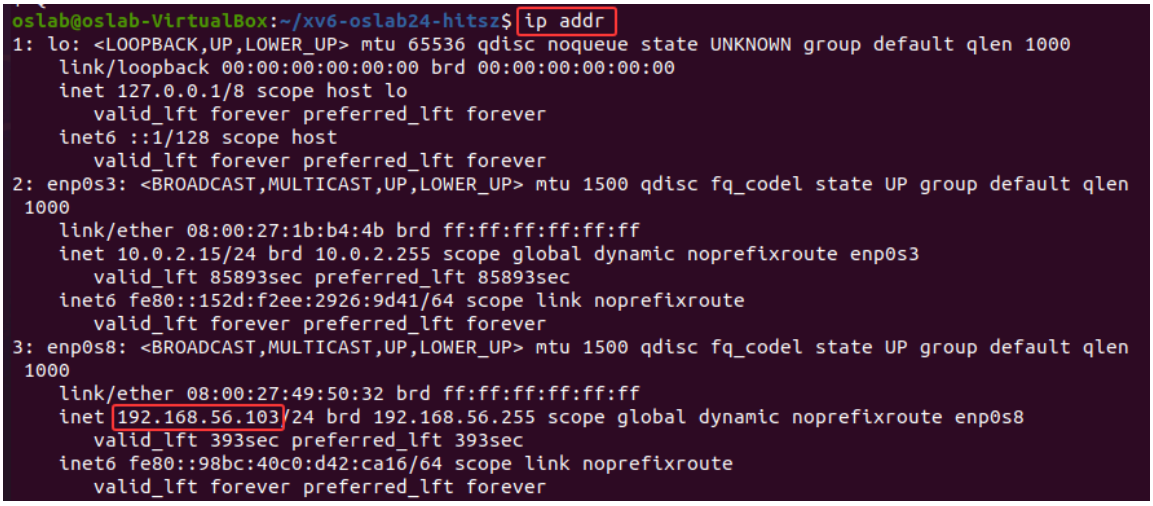
在虚拟机桌面右键，选择【Open in Terminal】（打开终端）。



输入以下命令查看 IP：

```
ip addr
```

找到 `192.168.56.xxx` 格式的 IP 地址（示例：`192.168.56.103`），记录该地址（后续连接需使用）。



2.2 准备远程连接工具

需提前安装以下软件（均为免费工具）：

- **必装**：Visual Studio Code（简称 VSCode，用于代码编辑和调试）；
- **推荐**：MobaXterm（可选，用于 SSH 终端连接，比系统自带终端更易用）。

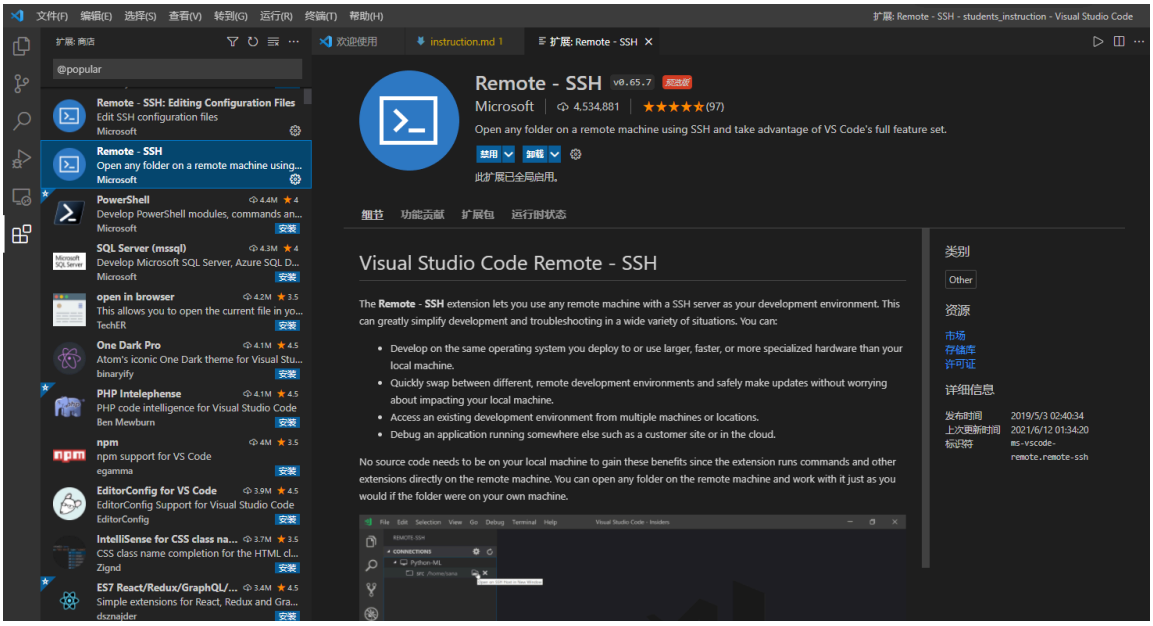
安装 VSCode（未安装时操作）：

1. 访问 VSCode 官网：https://code.visualstudio.com/；
2. 点击【Download for Windows】下载 Windows 版本安装包；
3. 运行安装包，按向导完成安装（建议勾选 “添加到 PATH”“创建桌面快捷方式”）。

2.3 用 VSCode 远程连接虚拟机

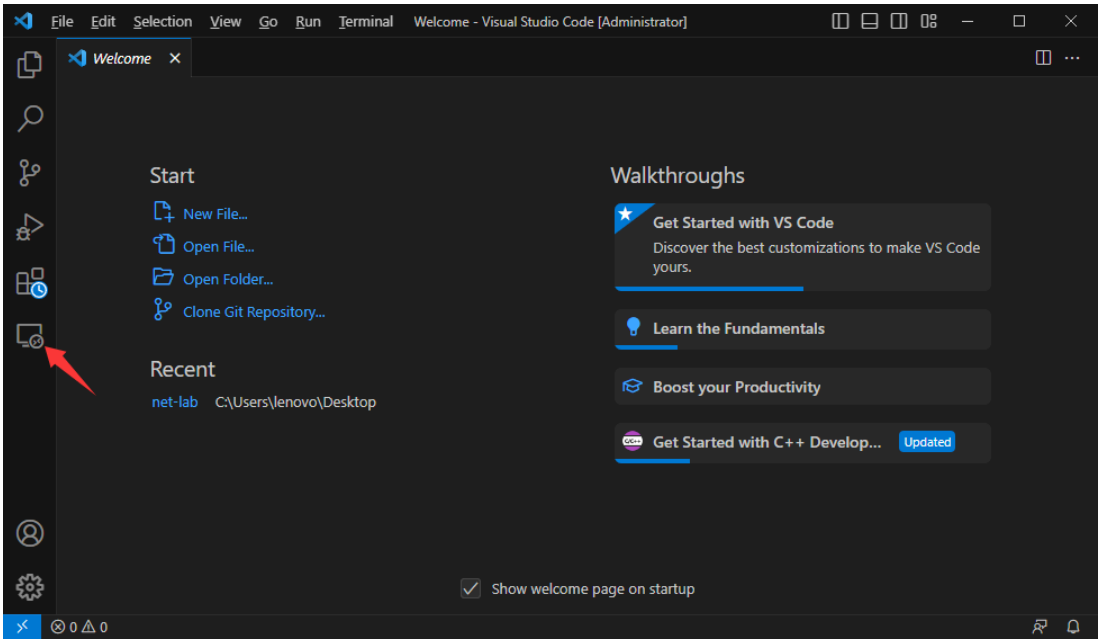
2.3.1 安装 Remote-SSH 插件

1. 启动 VSCode，点击左侧【扩展】图标（四方块图标）；
2. 在搜索框输入 “Remote-SSH”，找到对应插件并点击【安装】；

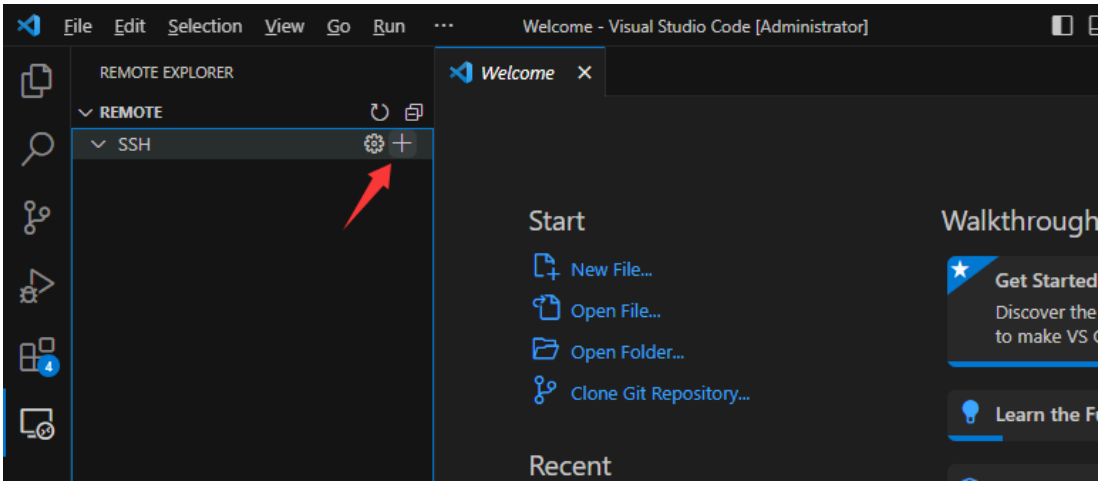


2.3.2 配置 SSH 连接

1. 安装完成后，点击左侧【远程资源管理器】图标（类似 “显示器 + 箭头”）；

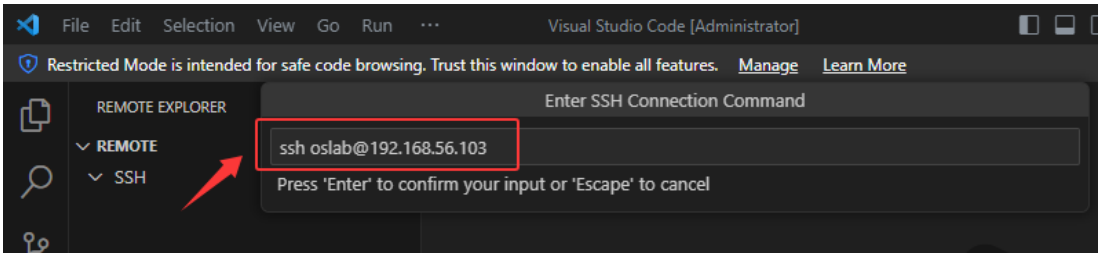


2. 点击 SSH 右侧的 + 图标（添加新连接）。

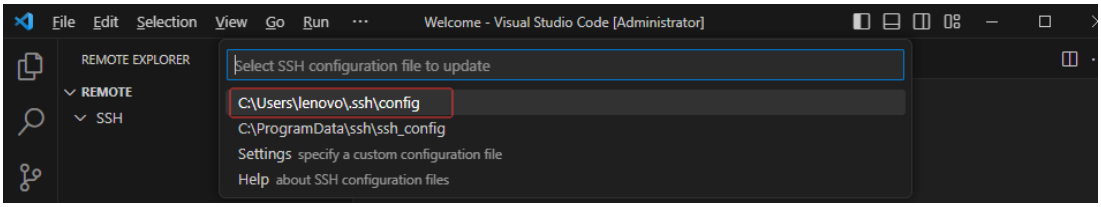


3. 在弹出的命令框中输入 SSH 连接命令（替换为 2.1 中记录的 IP 地址）：

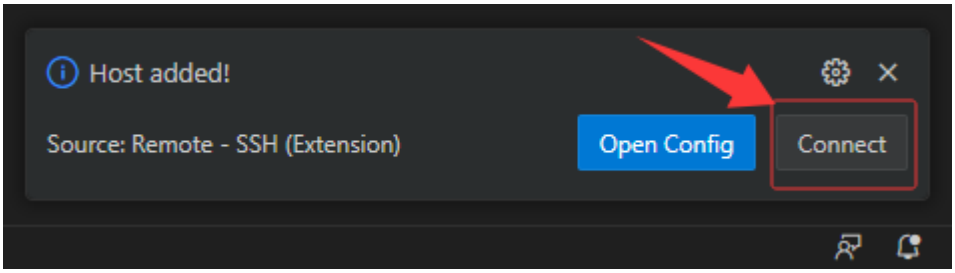
```
ssh oslab@192.168.56.103
```



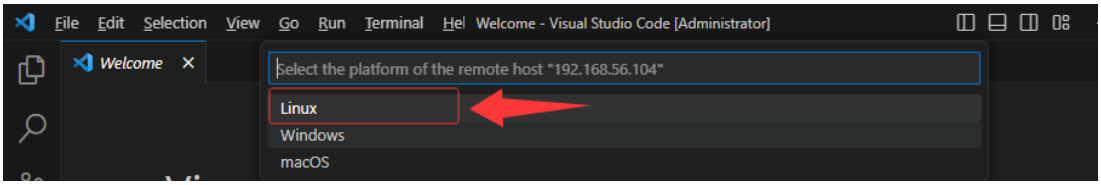
4. 选择 SSH 配置文件保存路径（默认路径即可，通常为 C:\Users\用户名.ssh\config），点击确定。



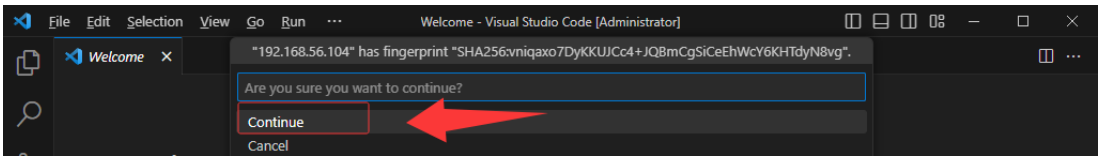
5. 在远程资源管理器中，找到刚添加的连接（显示为`ssh oslab@192.168.56.103`），右键点击【连接到主机】；

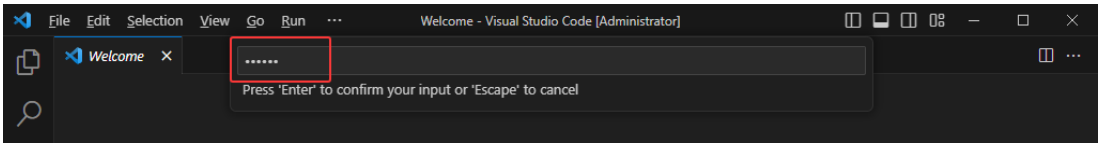


6. 在弹出的系统选择窗口中，选择 Linux（虚拟机操作系统）。

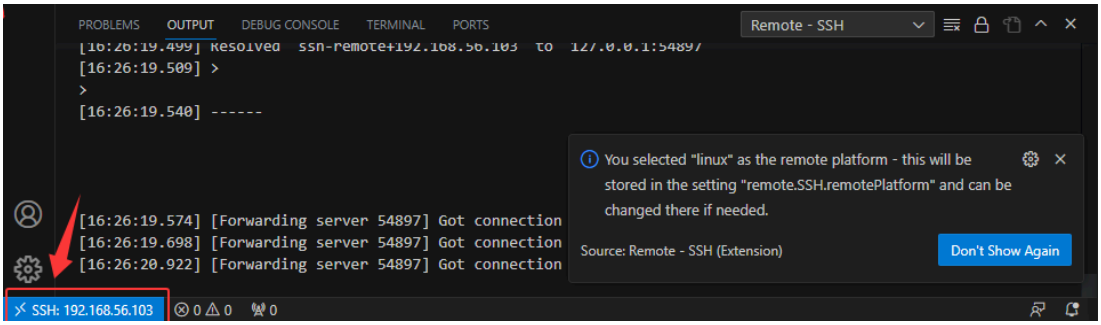


7. 点击 Continue 确认连接，在密码输入框中输入虚拟机密码 `123456` 并回车。

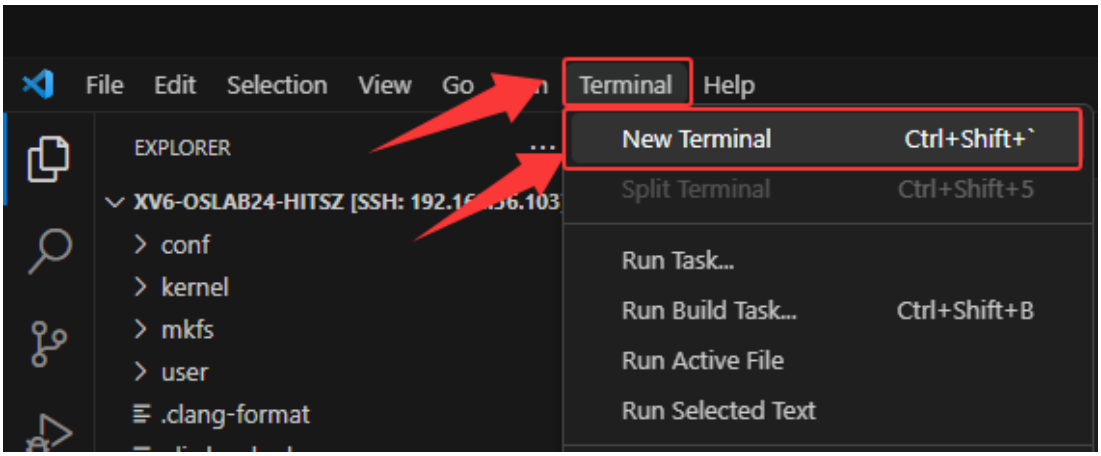




8. 连接成功后，VSCode 左下角会显示 SSH: 192.168.56.103。



9. 选择菜单栏中的Terminal，选择New Terminal。



2.4 运行XV6实验

1. 在New Terminal终端命令行，输入如下命令进入用户主目录

```
cd ~
```

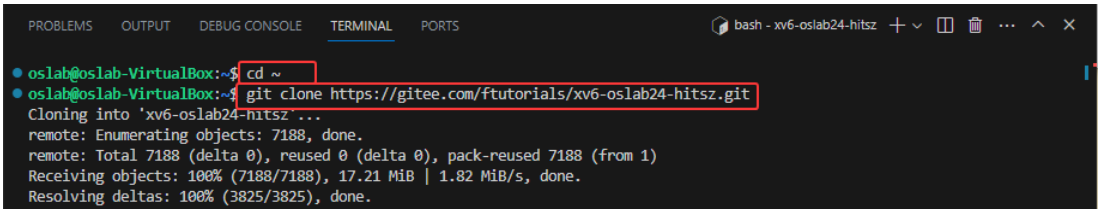
2. 克隆实验仓库

```
git clone https://gitee.com/ftutorials/xv6-oslabs-hitsz.git
```

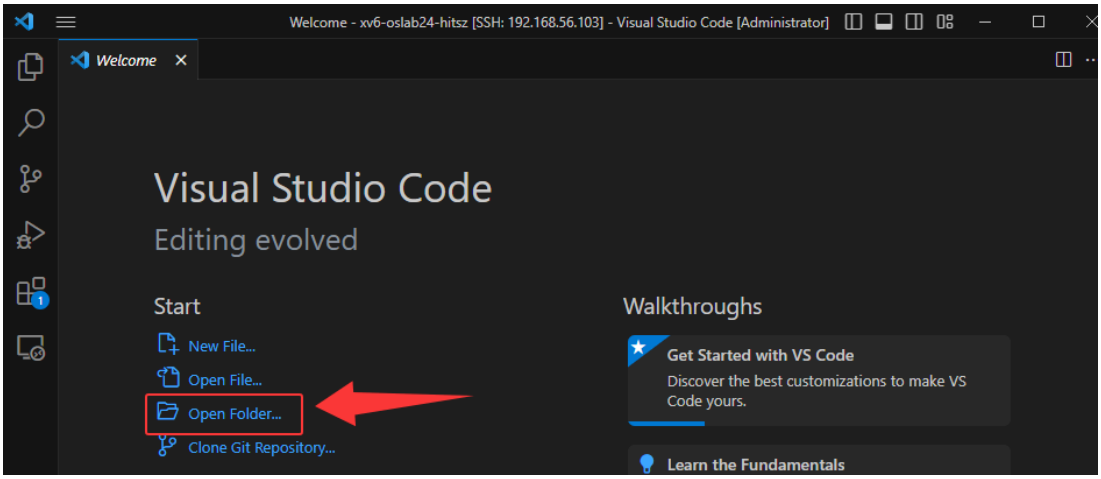
你应该会看到这样的输出：

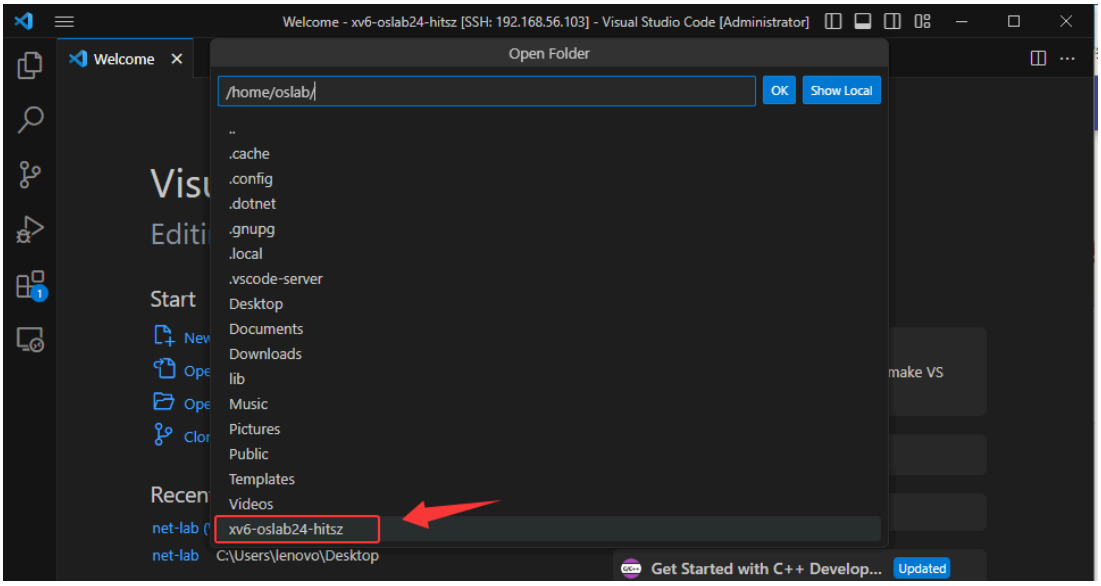
```
Cloning into 'xv6-oslabs-hitsz'...
done.
```

这代表你已经将xv6的副本下载到自己的工作空间了。



3. 点击【Open Folder】，选择 `/home/oslab/xv6-oslabs-hitsz`；

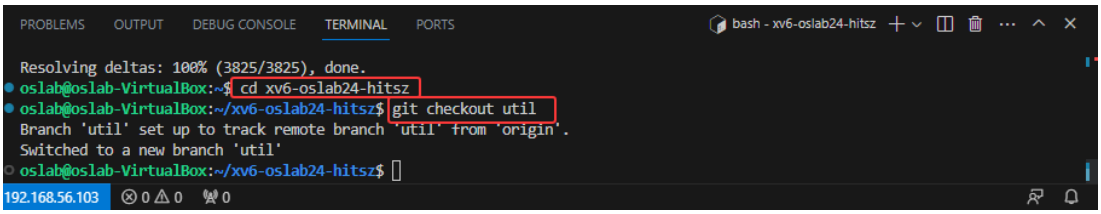




- 再次输入密码 `123456`，点击【确定】；
- 弹出 “信任作者” 提示时，勾选 “Trust the authors of all files in the parent folder 'oslab'”，点击【Yes, I trust the authors】。

4. 在终端命令行，首先进入 XV6 实验目录。
5. 根据实验需求切换至对应分支（不同实验对应不同分支，避免代码冲突），例如：实验一需切换至 util 分支：

```
cd xv6-oslabs-hitsz
git checkout util
```



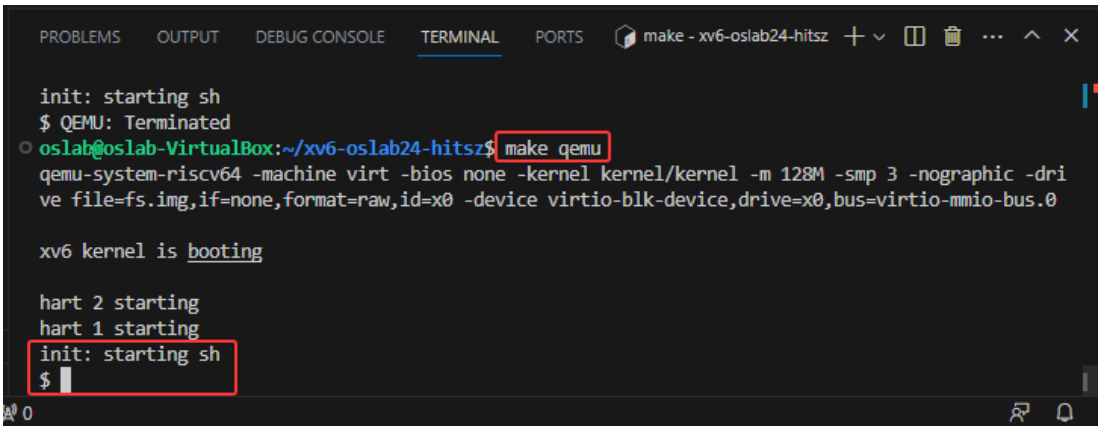
6. 输入`make qemu`启动XV6:

```
make qemu
```

7. 启动成功标志

   终端输出最后显示以下内容，即表示 XV6 启动成功：

```
init: starting sh
$
```



8. 退出 XV6 方法

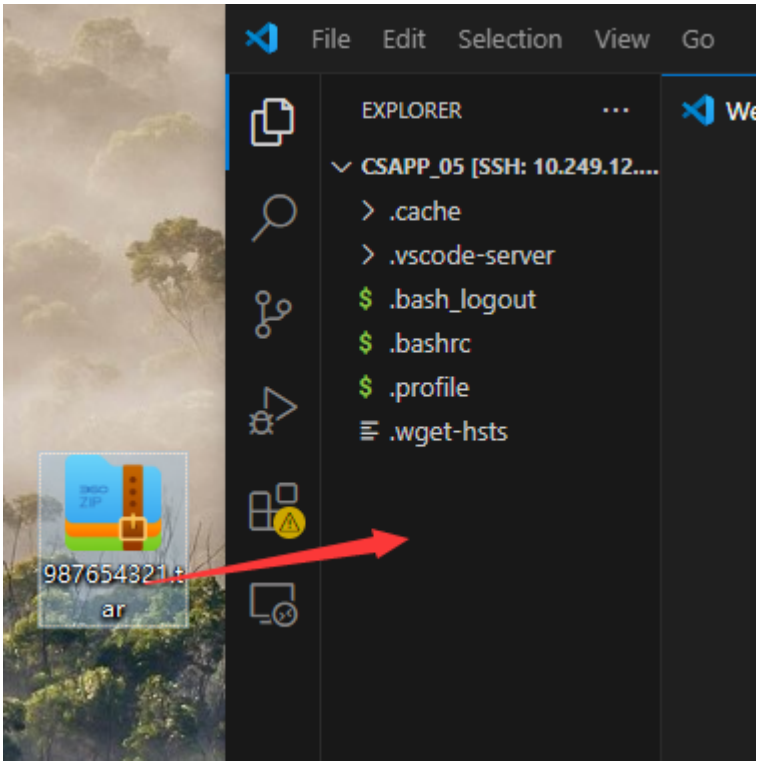
   务必按以下步骤退出，避免进程残留：

   - 先按 `Ctrl + a`
   - 松开后立即按 `x`

3. 文件传输

3.1 使用 VSCode 上传文件并开始实验

连接到 Ubuntu 虚拟机后，VSCode 资源管理器中会显示虚拟机里的目录结构。



你可以直接把本地的实验包或文件拖到 VSCode 左侧的远程目录中，松开鼠标后就会开始上传。

常见用法：

- 上传实验包：把 `学号.tar` 直接拖到 `/home/oslab/` 或你自己的工作目录
- 下载文件到本地：在 VSCode 资源管理器中找到目标文件，右键后选择下载到本地
- 新建文件夹：右键资源管理器中的目录即可创建
- 修改代码：直接在 VSCode 中打开并编辑远程文件

远程实验环境使用指南

安全警告

🔴 违规会掉装备！ 💡 被抓包的同学会将被遣返你自己的实验环境

- **控制资源占用**：请卸载 VSCode 在 remote 远端（10.249.12.98）的代码自动生成插件，包含但不限于 MarsCode、tabnine、Copilot 等。这类工具会大量占用远程实验平台的有限资源，极易造成服务器过载，导致其他同学无法正常开展实验。
- **平台用途限定**：本实验平台仅服务于《计算机系统》、《操作系统》、《计算机设计与实践》等课程实验，请勿用于无关用途。

Tip

尽管我们已经采取了一些措施以确保远程环境的稳定性，但在某些特殊情况下，我们无法保证不会出现故障。为了安全起见，建议同学们及时将代码上传至git仓库或下载到本地进行保存。

0 速查信息

远程实验平台IP地址： `10.249.12.98` ，端口号： `6666`

```
ssh 你的学号@10.249.12.98 -p 6666
```

- 用户名：你的学号
- 初始密码：你的学号
- 权限模式：普通用户（无 `sudo` 权限）

- **登录与认证**：用户名和初始密码都是 **你的学号** 。出于安全考虑，建议你初次登录后修改初始密码。
- **实验资源**：平台已为每位同学分配了专属的4GB存储空间，专用于本课程实验。请注意，你只有非root权限，这意味着你不能执行需要root权限的操作，例如安装未经许可的软件。请务必遵守使用规定，勿将平台用于与本课程无关的其他操作。
- **实验准备**：登录平台后，下载实验所需的代码，即可开始你的实践之旅。请仔细阅读实验指导，确保实验过程顺利进行。

如果遇到问题，请寻求助教或老师的帮助。

远程实验平台共有 8 个计算节点，每一个计算节点都提供相同的服务。你每次登录到的计算节点可能不同，但无须担心，你看到的文件都是一样的。



1 快速入门

1.1 环境准备

需要配合远程实验平台使用的软件：

- Visual Studio Code（必须）
- MobaXterm（推荐）

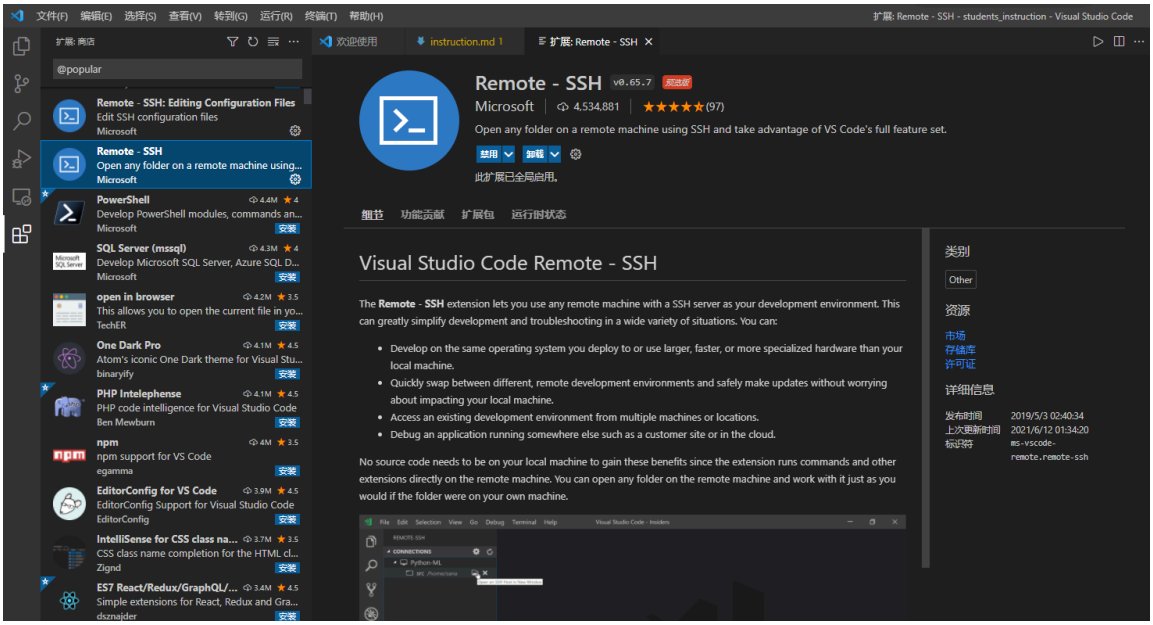
1.1.1 如何安装Visual Studio Code?

打开 Visual Studio Code 的官方网站，点击中间的“Download for Windows”下载它的安装包。双击运行安装包并完成安装即可。你可以从开始菜单中找到新安装的 Visual Studio Code。

1.2 连接到实验平台

1.2.1 将Visual Studio Code连接到实验平台

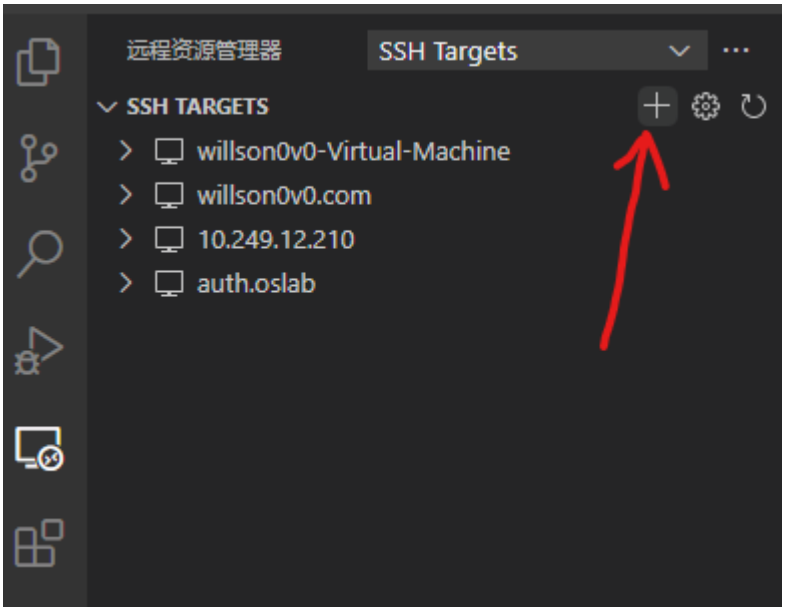
Visual Studio Code 安装完成后，从开始菜单打开它。我们需要安装 Remote - SSH 插件。点击 VSCode 左侧扩展（extension）按钮，在搜索框中键入“remote ssh”，点击 Remote - SSH 插件并安装。



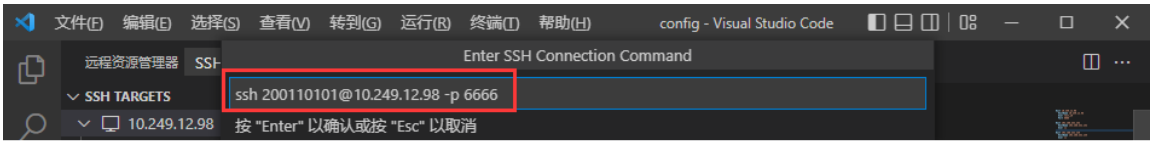
左侧边栏应该会多出一个这样的图标：



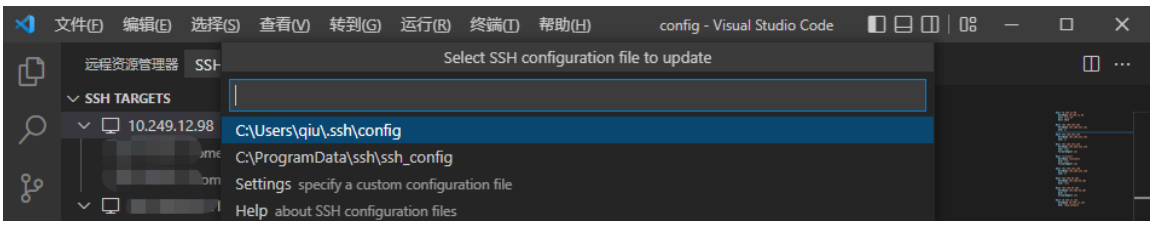
点击这个图标，选择SSH TARGETS右侧的`+`号：



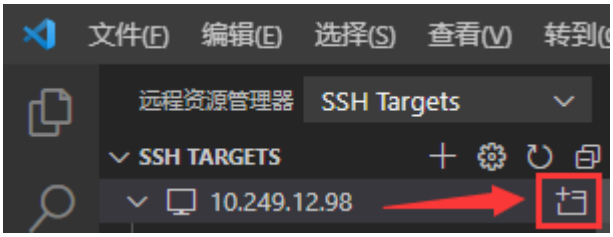
这会在屏幕上方中央弹出输入框，要求输入 ssh 指令。在这里输入 `ssh 你的学号@远程实验平台IP -p 端口号`，例如 `ssh 200110101@10.249.12.98 -p 6666` 就表示学号为 200110101 的同学希望连接到远程实验平台。



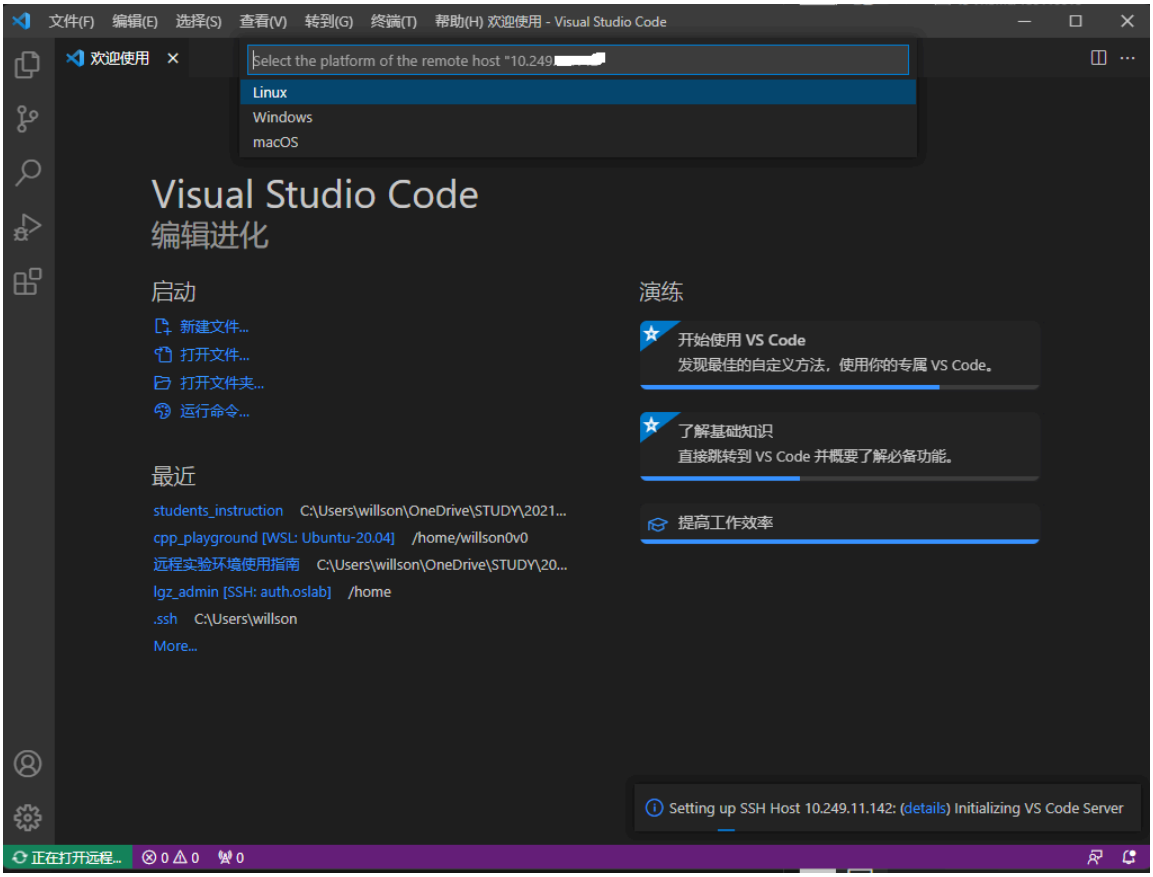
输入完成后，按下回车。如果弹出下图所示的界面，则再次按下回车：



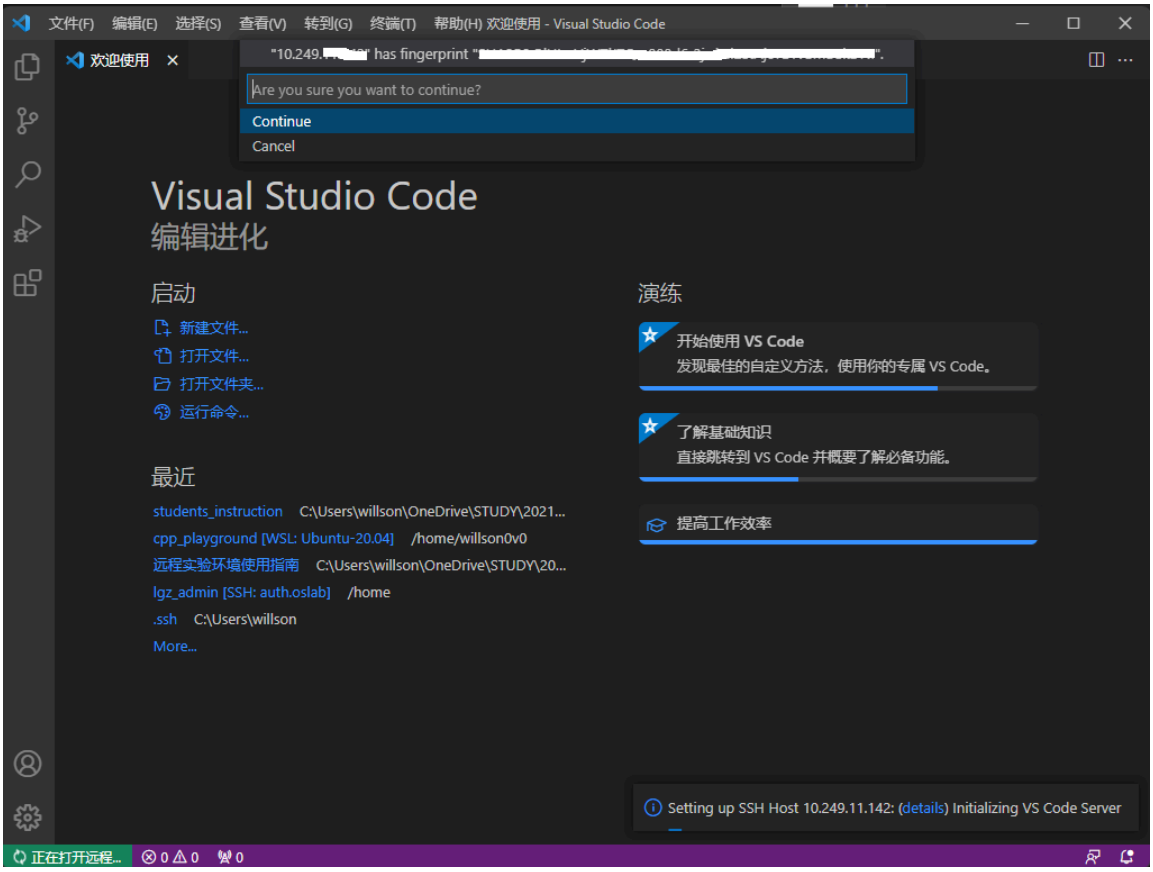
现在，SSH Target 添加成功，左侧 SSH Target 中应该增加了一栏。将鼠标移上去，点击这一栏右侧的小图标：



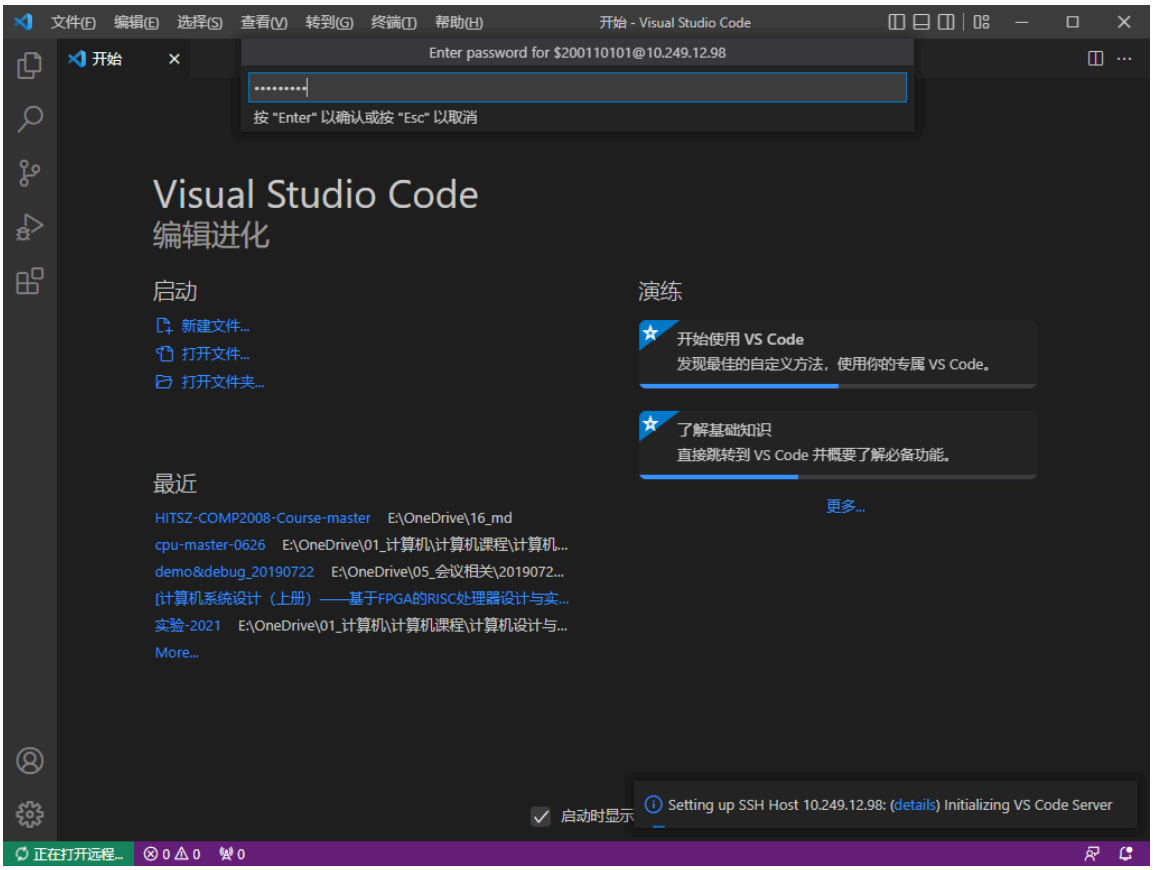
这会打开一个新的 VSCode 窗口。新打开的窗口会要求你选择目标机器的平台，选择 Linux，按下回车：



如果如下图所示要求你确认是否连接，则再次按下回车：



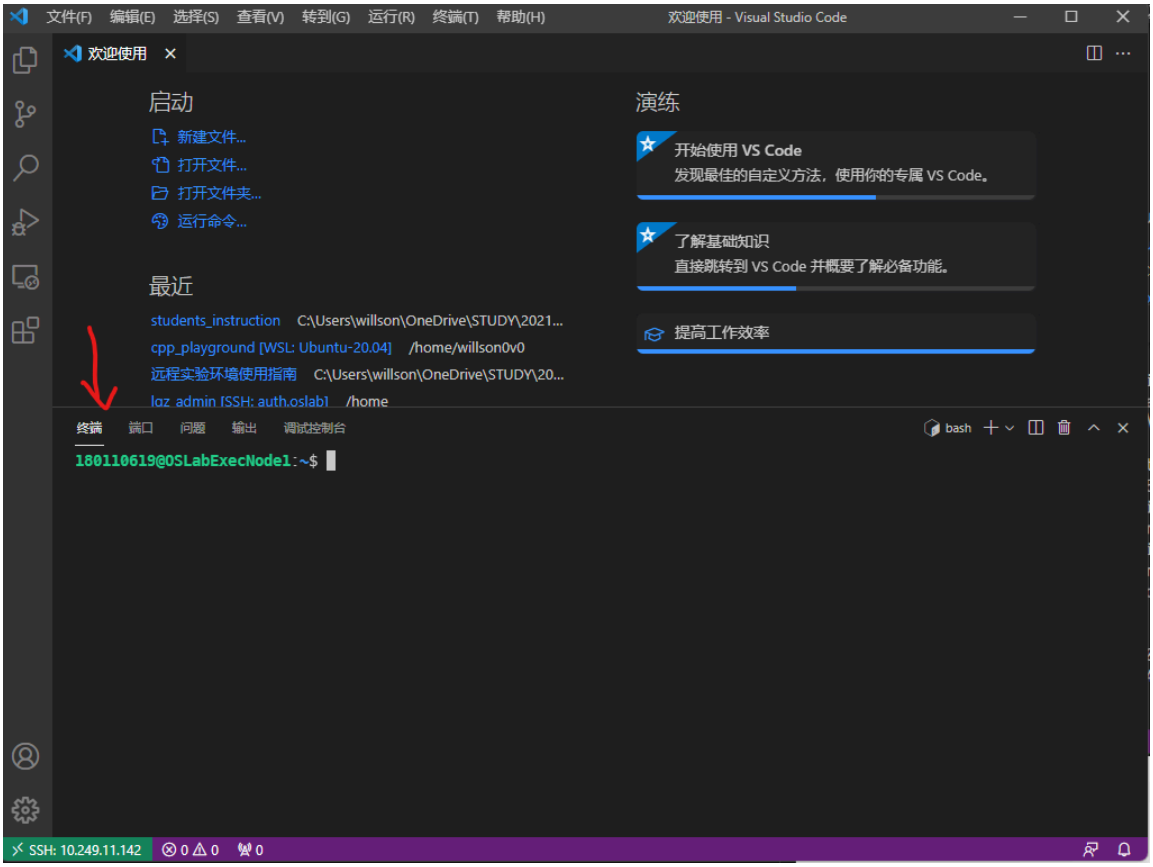
接下来，它会要求你输入密码。 **初始密码即学号** 。



最后一次按下回车。如果没有更多提示，左下角显示 `SSH: 实验平台的IP地址` ，说明连接成功。



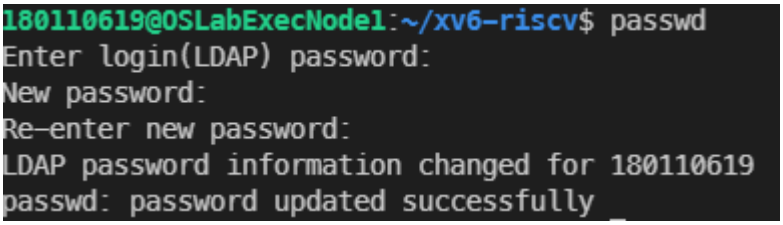
现在，点击下方的“终端”按钮，打开终端：



出于安全考虑，我们首先修改默认密码。点击终端，在其中键入以下内容，并回车：

```
passwd
```

此时，它会要求你输入旧密码。旧密码（即默认密码）为你的学号。完成后回车。 随后，它会要求你输入新密码，并进行确认。



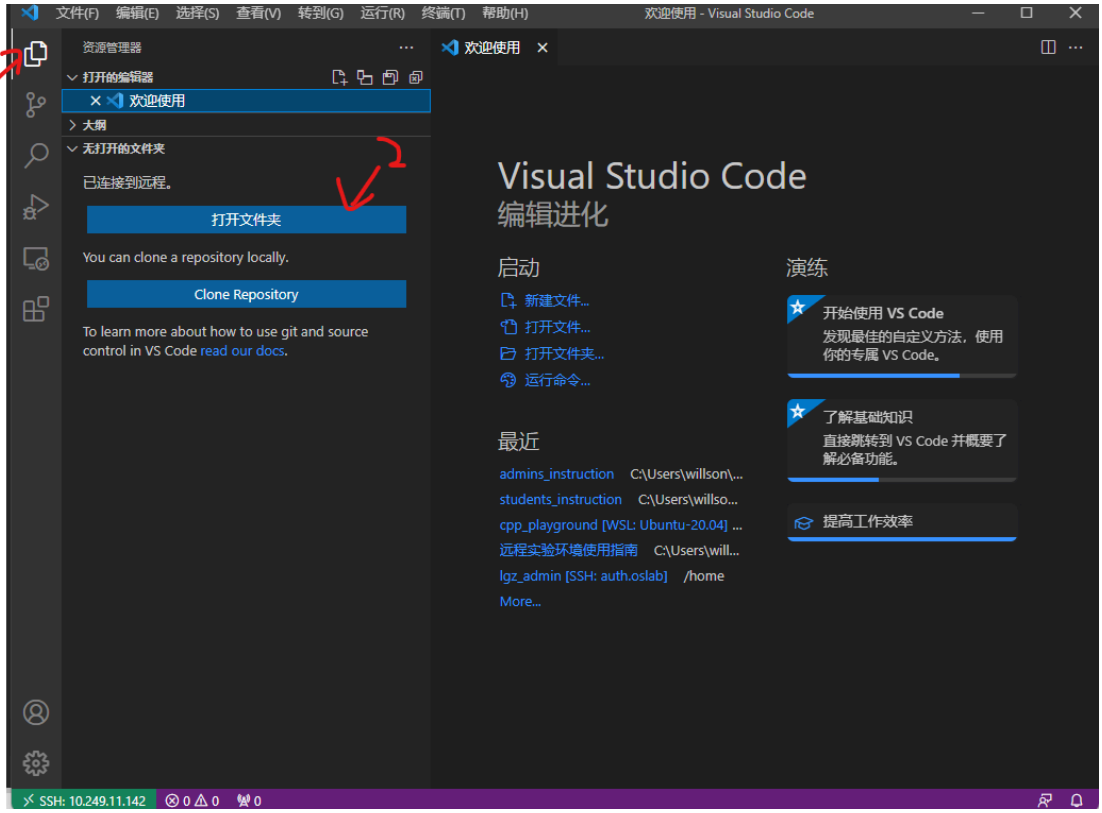
接下来，我们将存放 xv6-oslabs-hitsz 复制到自己的工作空间。 在终端中键入以下内容，并回车：

```
git clone https://gitee.com/ftutorials/xv6-oslabs-hitsz.git
```

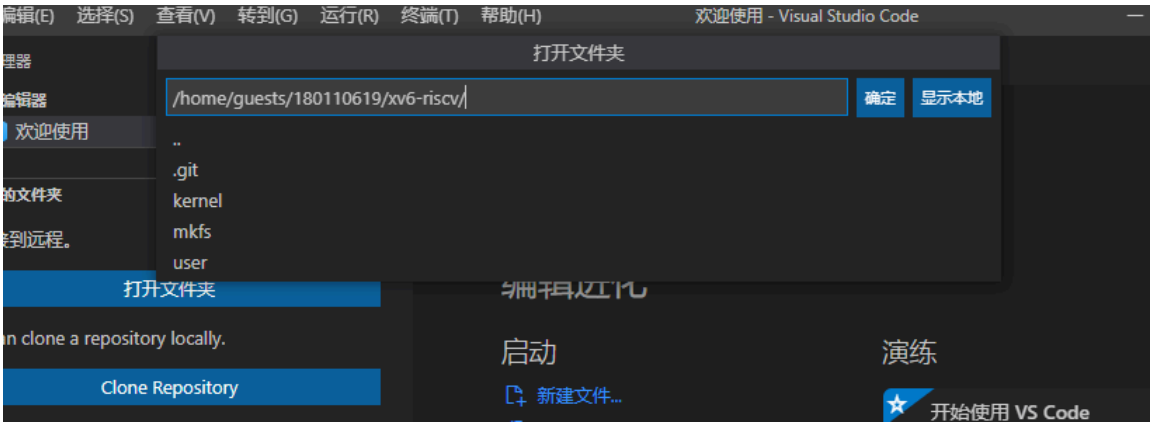
你应该会看到这样的输出：

```
Cloning into 'xv6-oslabs-hitsz'...
done.
```

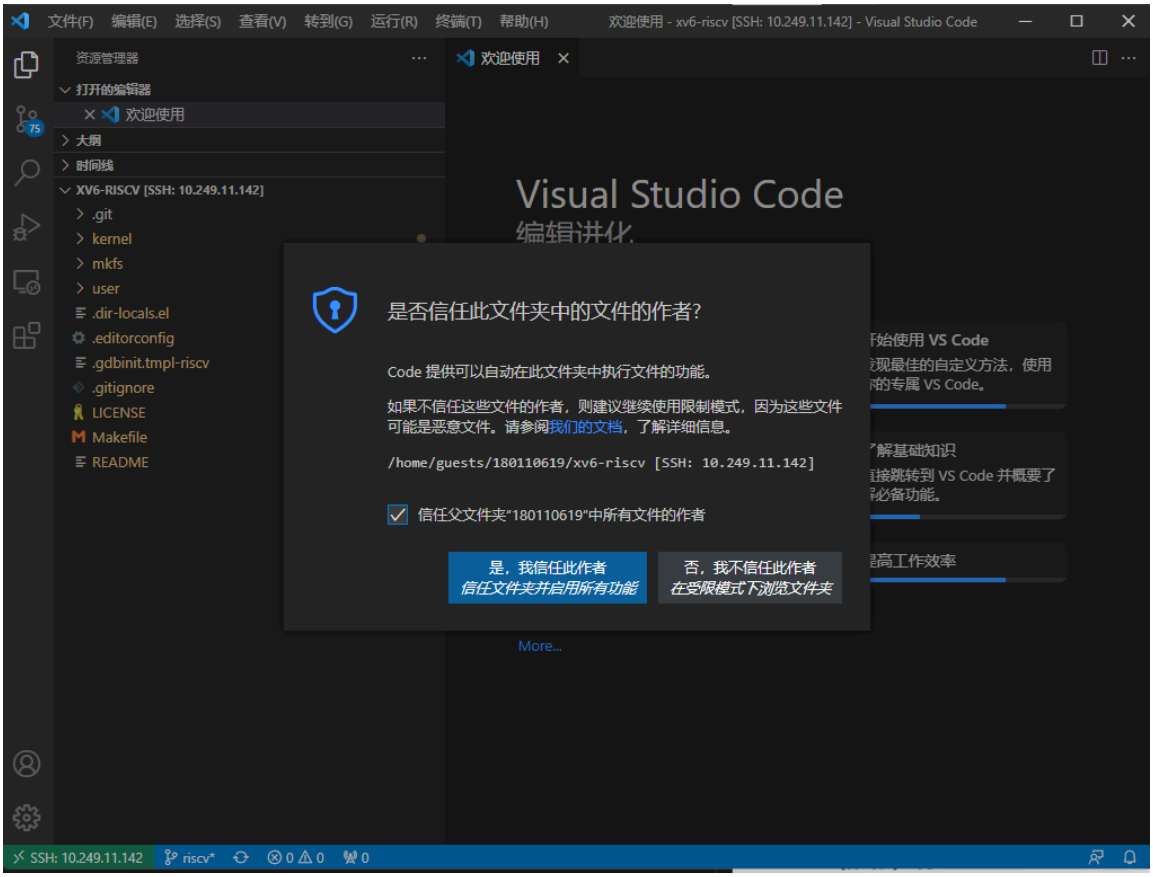
这代表你已经将 xv6 的副本下载到自己的工作空间了。接下来，你可以点击左侧边栏最上面的按钮，并打开文件夹：



在弹出的菜单中，选择 `xv6-oslabs-hitsz`，并按下“确认”：



再次输入密码，按下回车确认，并信任作者。



你初步完成了对工作区的设置。接下来，我们可以测试 xv6 能否正常编译、执行。 如果下方未显示终端，点击上方菜单“查看”-“终端”，或者按下 `Ctrl + \`` （ **注意：这里的 `` 不是单引号，而是键盘左上角的那个按键。** ）以打开终端界面。

由于每个实验项目都在不同的分支上完成，请同学们 **注意切换分支** 。例如，实验一需切换到 util 分支后进行开发。在新弹出的终端中，输入下面的命令：

```
git checkout util
```

然后，输入下面的命令，以编译并在QEMU中运行xv6：

```
make qemu
```

如果看到许多输出，并最终停留在这个输出上：

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

那么说明你的环境已经配置完成。

请记住QEMU的退出方法

先按“`Ctrl+a`”组合键，接着松开所有按键。再按下“`x`”键。

命令解释：“`Ctrl+a`”组合键，用于激活QEMU的全局命令模式。松开所有按键。然后，按下x键，这样将直接终止QEMU进程，返回到你启动QEMU的Shell或命令行界面。

如果在退出QEMU时遇到任何问题（如QEMU进程无法终止），你可能需要使用kill命令来强制结束QEMU进程。你需要重新开启一个Linux Terminal命令行终端界面，输入如下命令，把与自己有关的进程全 kill：

```
killall -u id
```

`killall -u id` 中的 `id` 应该替换为 `你的学号`。

1.2.2 实验常用git命令

之后同学们每次实验的打开方式通常就是：打开 VSCode，登录远程实验平台（或者登录自己搭建的环境），打开终端，进入 `xv6-oslabs-hitsz` 文件夹，然后输入以下命令：

```
git pull
git checkout 本次实验对应的分支名称
```

说明：

- `git pull`：将远程仓库的进度同步到本地
- `git checkout 本次实验对应的分支名称`：切换到本次实验对应的分支，例如：
- Lab1：`git checkout util`
- Lab2：`git checkout syscall`
- Lab3：`git checkout lock`
- Lab4：`git checkout pgtbl`

中间可以检查一下 Git 的状态（确保在 `xv6-oslabs-hitsz` 文件夹下）：

```
git status
```

说明：

- `git status`：会显示哪些文件被修改了，以及这些修改是否已加入暂存区等信息

写完代码之后：

```
cd xv6-oslabs-hitsz
git add .
```

```
git commit -m "做了哪些工作和改动"
make diff
```

说明：

- `git add .`：将当前文件夹及其子文件夹下所有被修改或添加的文件加入暂存区；如果想单独加入某些文件，可以将 `.` 替换为具体文件名或路径
- `git commit -m "做了哪些工作和改动"`：将暂存区中的修改提交
- `make diff`：生成 `commit.patch`，记得和实验报告一起提交

如果遇到问题，请询问老师或助教。

附录

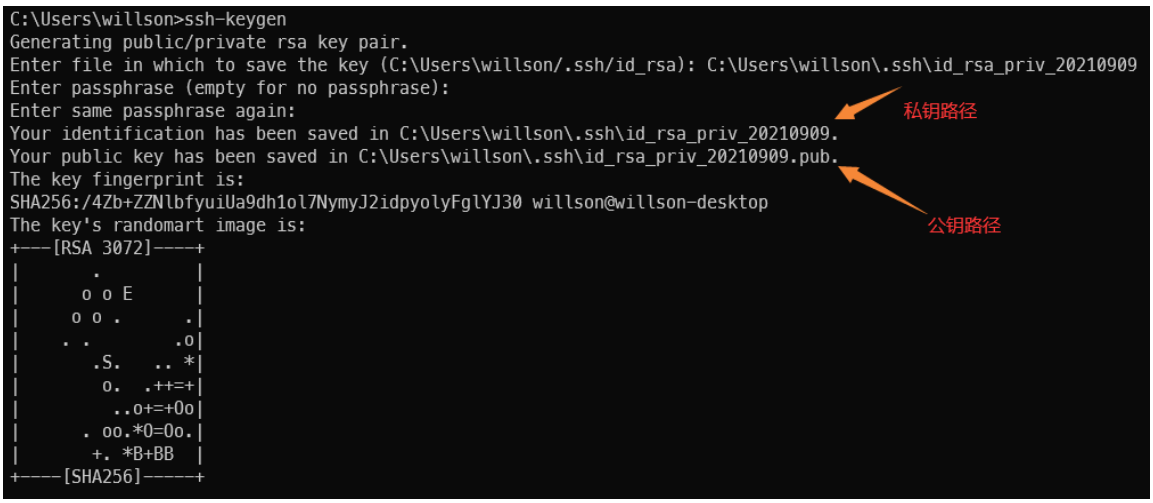
一些进阶指引

使用ssh密钥快捷认证

有没有感觉每次都要输入密码非常麻烦？ssh 密钥可以让认证流程自动化，大大解放双手。在 **本地** Windows 下，以管理员身份打开 CMD，输入：

```
ssh-keygen
```

跟随指引一路回车，便可完成 ssh 密钥的生成。 **请务必保存好自己的私钥，不要泄露！**



接下来，我们将私钥加入到自己的身份认证中。在终端继续输入：

```
ssh-add 你的私钥的完整路径
```

例如，如上图所示，可以输入：

```
ssh-add C:\Users\willson\.ssh\id_rsa_priv_20210909
```

在你的 Windows 上将公钥文件打开（用你喜欢的纯文本编辑器，比如记事本或者 VSCode），复制其中的内容。

最后，登录你想要快捷认证的远程实验平台终端，并执行下面的命令：

```
mkdir -p ~/.ssh
vi ~/.ssh/authorized_keys
```

这会打开一个文件编辑界面。按下 `i` 键进入编辑模式，并把刚才复制的公钥内容粘贴到一个新行里。按下 `Esc`，输入 `:wq` 可保存并退出。如果不放心，可以执行 `cat ~/.ssh/authorized_keys` 检查输出和你刚才粘贴进去的内容是否一致。 配置完成后，下次你从 Windows 登录时，应该就无需密码认证了。

Visual Studio Code：进阶使用

有没有怀念 Visual Studio、CodeBlock 等 IDE 中便捷的一键运行功能？没关系，完备的插件市场配合自动化脚本可以完成你的一切功能需求！ *下略：感兴趣的话可以了解一下Intelli Sense、MakeFile以及VSCode中的Task。*

Q&A

为何要使用远程实验平台?

主要是实验搭起来比较麻烦。有几个选项可以选择：

1. 搭建一台Linux虚拟机。
2. 在自己电脑上启用WSL（Windows Subsystem for Linux）。
3. 整一台运行Linux的电脑——都可以。

但是都挺麻烦。我们帮你把配置的麻烦事先做完了，于是你就可以专心于实验本身。无论你的电脑性能如何，无论你是在宿舍、实验室还是自习室，只要你还能连上校园网，你就能完成你的实验。

Troubleshooting

我不是Windows 10系统，这会带来什么问题吗?

我在使用LINUX/MACOS?

Visual Studio Code 也有 Linux 版和 macOS 版，具体操作大同小异，毕竟环境主要在服务器上。

我正在使用旧的WINDOWS版本/我没有更新我的WINDOWS 10系统?

请更新你的系统。使用过时的系统有各种各样的风险，其中最大的是与时代脱轨。

我正在使用WINDOWS 11?

这篇教程完全兼容 Windows 11。笔者正在用的就是 Windows 11。

我输入的所有命令看起来都是对的，但是为什么运行失败了?

全角标点符号/英文字符?

确保你输入的是正常的半角英文字符与英文标点符号，如图所示：



这可以通过一起按下 `Shift+空格`，或者切换到英文输入法解决。 **永远不要在中文输入法下输入命令!**

如何退出XV6?

同时按下 `Ctrl + A`，接着全部松开，再按下 `x` 以退出 QEMU。

VSCode连接远程实验平台时出现bad owner or permission报错?

解决方法：进入路径 `C:\Users\用户名\.ssh`，右击 `config` 文件进入属性中的高级安全管理，选择禁止继承，并删除所有此对象中继承的权限。

具体可以查看：https://www.cnblogs.com/Akkuman/p/11187776.html

实验环境搭建指南（自行部署版）

为什么推荐自行部署?

自行搭建环境不仅能加深对 Linux、工具链的理解，更能培养问题排查能力—— 这些都是操作系统学习的核心技能。动手试试，收获会远超预期 :)

部署实验平台前，先认识这几类工具

搭建本课程实验环境时，通常会接触下面几类工具：

- **WSL2**

如果你使用的是 Windows 10 / 11，通常最推荐先考虑 WSL2（Windows Subsystem for Linux 2）。它可以在 Windows 中直接运行 Linux 环境，系统调用兼容性和性能都明显好于 WSL1；同时还能较方便地访问 Windows 文件、拖拽文件，整体会比传统虚拟机更轻量。

如果你准备使用 WSL2，可参考官方文档：WSL 文档。

- **VirtualBox 虚拟机**

如果你更习惯使用独立的 Linux 环境，也可以选择在本机安装虚拟机来运行 Linux。VirtualBox 是一个常见的开源方案，优点是免费、资料多；缺点是整体体验通常不如 WSL2 轻便。

- **Linux 发行版**

无论是 WSL2 还是虚拟机，本质上都需要先准备一个 Linux 系统。常见选择包括 Ubuntu、Debian、Fedora、CentOS 等；本页后续步骤将以 **Ubuntu** 为例说明。

- **RISC-V 工具链**

这是交叉编译环境的核心，包含 `gcc`、`binutils` 等工具，用于把 xv6 源码编译成 RISC-V 架构可执行的机器码。

- **QEMU 模拟器**

QEMU 用来在常见的 x86 电脑上模拟 RISC-V 机器。后续运行 xv6 时，你看到的“启动内核、进入 shell、观察系统行为”等过程，都是通过 QEMU 完成的。

> **注意事项**
>
> - 开始前，请先准备好一个可用的 Linux 环境；常见选择包括 Ubuntu、Debian、CentOS 等。
> - 如果你使用的是 Windows，通常推荐优先使用 WSL2；也可以通过虚拟机安装 Linux。
> - 本页后续配置步骤以 **Ubuntu** 为例。如果你准备使用其他 Linux 发行版，可参考 MIT 6.S081 提供的工具链安装说明：https://pdos.csail.mit.edu/6.S081/2020/tools.html

1 手动部署实验环境

1.1 安装基础依赖与工具链

1. 下载工具包

   访问链接下载 binutils-gdb.zip： https://file.cs-lab.top/s/xcAKpIRmnrFUOPe

   并将binutils-gdb.zip上传到Linux环境中。

2. 更新软件源，确保后续安装的工具为最新版本

```
sudo apt-get update
```

3. 安装基础开发工具

   包含git、编译工具、调试工具、QEMU模拟器、RISC-V交叉编译链

```
sudo apt-get install git build-essential gdb-multiarch qemu-system-misc
gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
```

4. 安装编译binutils-gdb依赖包：

```
sudo apt-get install autoconf automake autotools-dev curl python3 python3-
pip libmpc-dev libmpfr-dev libgmp-dev gawk build-essential bison flex
texinfo gperf libtool patchutils bc zlib1g-dev libexpat-dev ninja-build git
cmake libglib2.0-dev libslirp-dev python3-dev
```

5. 解压工具包并进入目录（*通配符匹配带哈希值的长目录名）

```
# 以工具包在~/Downloads为例，若路径不同请替换
unzip ~/Downloads/binutils-gdb.zip -d ~/
cd ~/binutils-gdb*
```

6. 创建并进入编译目录（避免污染源码）：

```
mkdir build && cd build
```

7. 配置编译选项:

- --target：指定目标架构为RISC-V 64位
- --prefix：指定安装路径为/usr/local
- --with-python：关联Python3支持

```
../configure --target=riscv64-unknown-elf --prefix=/usr/local --with-
python=/usr/bin/python3
```

8. 编译与安装（耗时约 5-10 分钟，-j4表示 4 核并行编译）：

```
make -j4
sudo make install
```

9. 安装 RISC-V 架构交叉编译 GCC

```
sudo apt-get update
sudo apt-get install gcc-riscv64-unknown-elf
```

1.2 运行XV6实验

1. 进入用户主目录

```
cd ~
```

2. 克隆实验仓库

```
git clone https://gitee.com/ftutorials/xv6-oslabs-hitsz.git
```

3. 进入实验目录并切换分支：

```
cd xv6-oslabs-hitsz
git checkout util
```

4. 输入`make qemu`启动XV6:

```
make qemu
```

5. 启动成功标志

   终端输出最后显示以下内容，即表示 XV6 启动成功：

```
init: starting sh
$
```

6. 退出 XV6 方法

务必按以下步骤退出，避免进程残留：

- 先按 `Ctrl + a`
- 松开后立即按 `x`

实验实用工具

> **这页怎么用**
>
> 如果你是第一次做实验，建议按这个顺序学习： **VSCode 远程连接 → Git 保存代码 → 常见问题排查** 。

1. SSH 远程连接

SSH 可以理解成： **从你的电脑安全地登录到另一台 Linux 电脑** 。

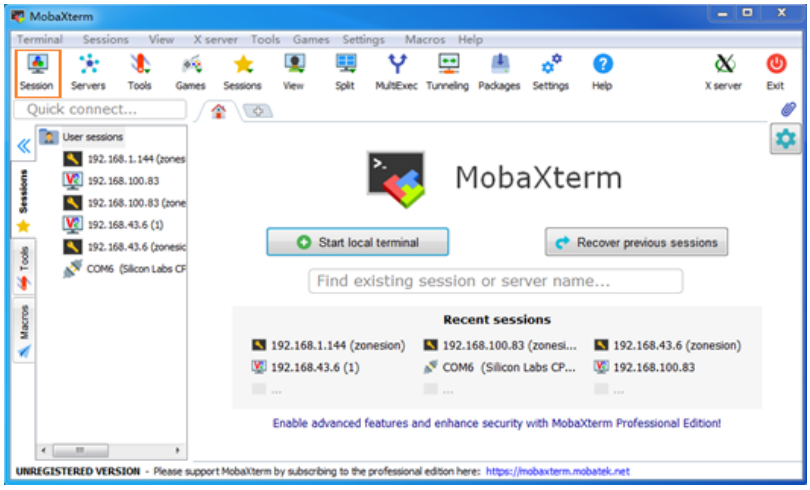
本课程里，SSH 主要用在两种场景：

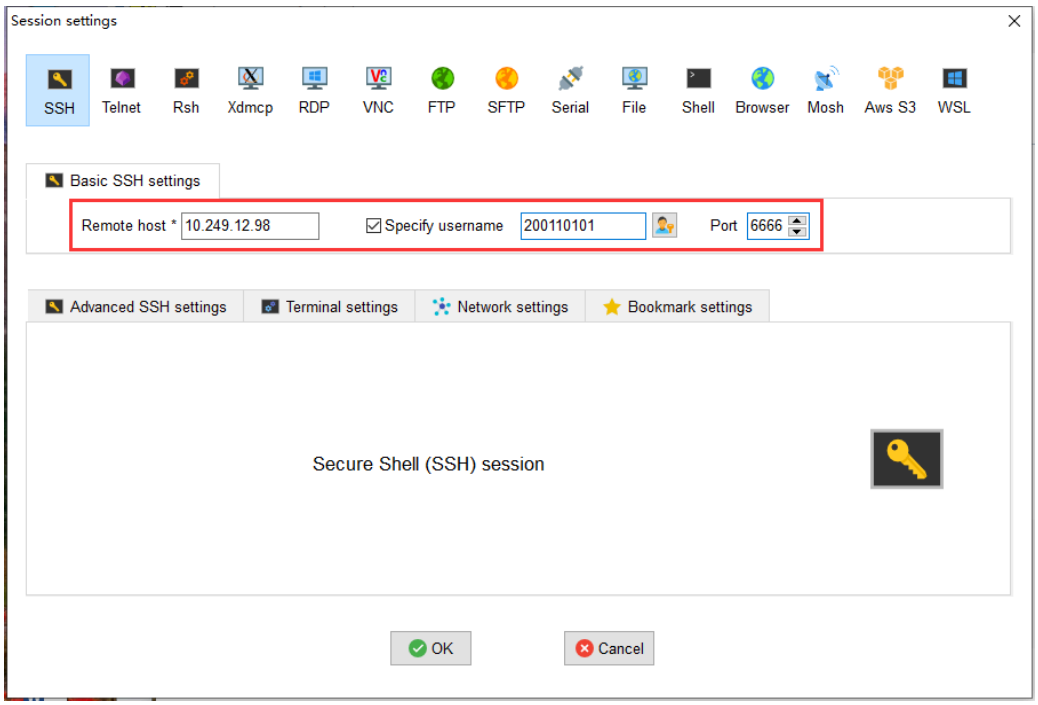
- 连接远程实验平台
- 连接你自己搭建的 Linux 环境

1.1 MobaXterm（Windows 可选）

如果你更习惯图形界面，可以用 MobaXterm；如果你主要使用 VSCode，可以直接看第 2 节。

- 下载链接（校内网）：https://file.cs-lab.top/s/xcAKpIRmnrFUOPe
- 新建连接时选择 `SSH`
- 远程实验平台填写：
  - 主机：`10.249.12.98`
  - 端口：`6666`
  - 用户名：`你的学号`
  - 初始密码：`你的学号`





如果你使用的是自己部署的 Linux 环境：

- 主机：你的 Linux IP 地址（可以使用ip addr命令来查询IP地址）
- 端口：默认通常是 `22`
- 用户名/密码：你自己 Linux 系统的用户名和密码（输入密码是没有显示的，输入完成后敲回车键）



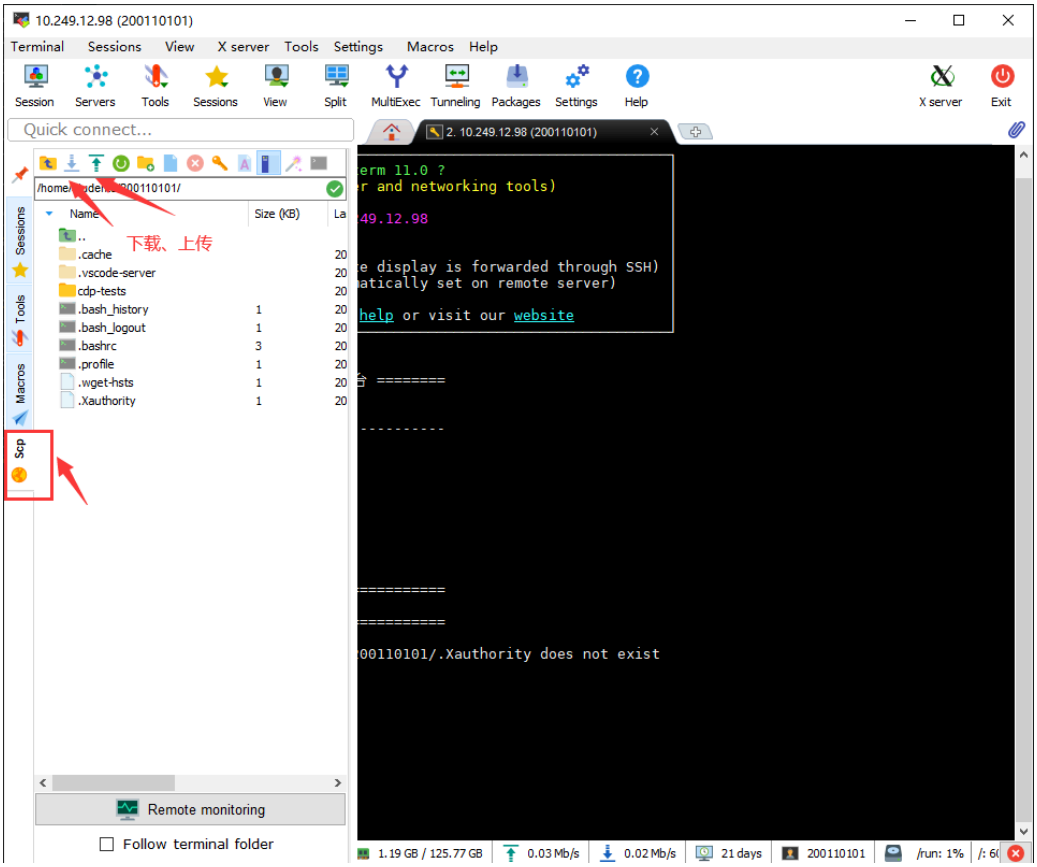
1.2 传文件（SFTP）

MobaXterm 登录成功后，左侧通常会自动出现远程目录。

你只需要知道两件事：

- **Upload**：把本地文件传到 Linux
- **Download**：把 Linux 文件下载到本地

如果你已经在用 VSCode Remote-SSH，那么通常不需要再单独用 SFTP 传文件。



1.3 `tmux`（推荐）

`tmux` 是终端多路复用工具。你可以把它理解成“**终端里的标签页 + 保险箱**”。

它最直接的好处是：**即使网络断了，里面跑着的程序也不会立刻消失**。

常用命令：

```
tmux  # 新建会话
tmux a  # 回到上次会话
```

常用快捷键（先按 `Ctrl+b`，再按后面的键）：

- `%`：左右分屏
- `"`：上下分屏
- `d`：暂时挂起当前会话

- `[`：进入滚动查看模式

如果你在远程环境里跑 `make qemu`、长时间测试或训练脚本，推荐放在 `tmux` 里执行。

2. VSCode（强烈推荐）

VSCode 是本课程最推荐的开发工具。原因很简单： **写代码、查文件、开终端、调试、提交 Git，基本都能在一个界面里完成** 。

2.1 先装这几个扩展

建议安装：

1. `Remote - SSH`
2. `C/C++`
3. `Chinese (Simplified)`（可选）

安装方法很简单，可以按下面步骤操作：

1. 打开 VSCode 左侧的 **Extensions（扩展）** 面板，或直接按 `Ctrl+Shift+X`。
2. 在搜索框中输入扩展名，例如 `Remote - SSH`。
3. 找到对应扩展后，点击 **Install**。
4. 按同样的方法，再安装 `C/C++` 和 `Chinese (Simplified)`。

如果你已经通过 `Remote - SSH` 连上了远程环境，还要特别注意安装位置：

- `Remote - SSH` 一般装在本地 VSCode 即可；
- `C/C++` 这类和代码分析、调试相关的扩展，通常需要安装到 **远程环境**；
- 如果按钮上出现 `Install in SSH: ...`，优先选择安装到远程环境。

这几个扩展分别解决的是：

- `Remote - SSH`：让 VSCode 直接连到远程 Linux；
- `C/C++`：提供 C/C++ 代码高亮、跳转、补全和调试支持；
- `Chinese (Simplified)`：把 VSCode 界面切换成中文，便于刚开始使用时识别各个按钮和菜单。

2.2 连接远程环境

最常见的连接方式，是先点击 VSCode 左侧边栏中的 **远程连接按钮**（Remote Explorer / Remote-SSH 入口），然后添加并连接一台 SSH 主机。

如果你更习惯直接记命令，也可以把它理解成连接下面这条 SSH 命令对应的远程环境；在首次连接时，通常还需要输入远程主机的登录密码：

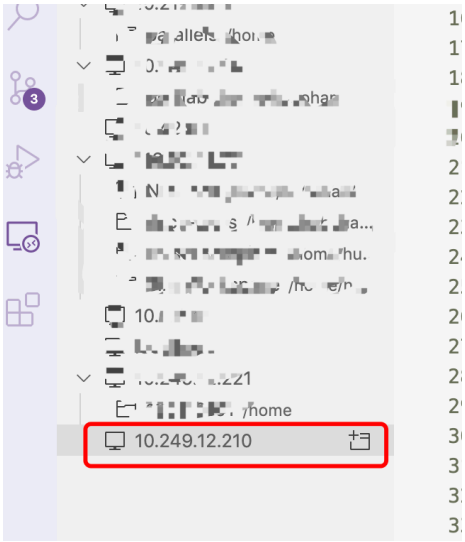
```
ssh 你的学号@10.249.12.98 -p 6666
```

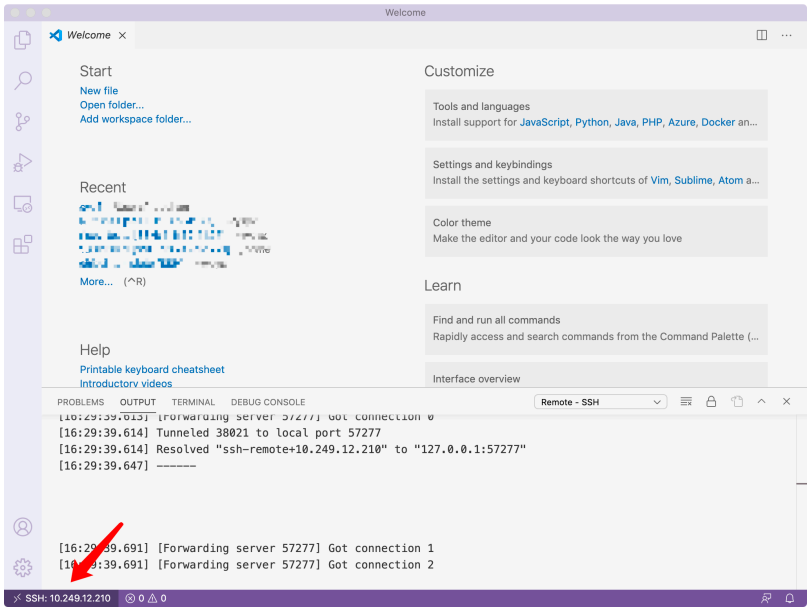
连上后，建议做这 3 件事：

1. 打开实验仓库文件夹
2. 打开内置终端
3. 在终端里执行 `pwd`、`ls`、`git status` 看看环境是否正常

如果你需要更详细的逐步截图说明，请看远程实验平台使用指南。







2.3 在 VSCode 里最常用的功能

对本课程来说，最常用的是下面这些：

- **资源管理器**：看文件目录、打开代码
- **全局搜索**：快速查函数、变量、字符串
- **终端**：执行 `git`、`make qemu`、`make grade`
- **Source Control**：图形化提交 Git

如果你刚开始不熟，先记住一句话：

> **写代码看左边，跑命令看下面，提交 Git 看侧边栏的 Source Control。**

3. Git（本课程最重要的工具）

Git 是版本控制工具。对本课程来说，它至少有 3 个作用：

- **切换实验分支**，例如 `util`、`syscall`、`lock`、`pgtbl`
- **保存自己的代码进度**，防止改坏后回不去
- **同步课程更新**，把官方仓库的新内容合并到自己的仓库

Git 远程仓库统一使用 HTTPS

- `HTTPS` 不需要额外配置 SSH 密钥，更适合第一次接触 Git 的同学。
- `SSH` 依赖本机密钥配置，初学者更容易遇到 `Permission denied (publickey)` 一类问题。

3.1 同步上游仓库

当课程组更新了实验代码，或者你要切换到新的实验分支时，会用到这一节。

3.1.1 检查远程仓库设置

```
git remote -v
```

查看当前仓库配置了哪些远程仓库，以及它们对应的网址。

理想情况下你会看到两个远程仓库：

- `origin`：你自己的仓库
- `upstream`：课程提供的官方仓库

3.1.2 完成远程仓库设置

如果没有 `upstream`，执行：

```
git remote add upstream https://gitee.com/ftutorials/xv6-oslabs-hitsz.git
```

添加一个名为 `upstream` 的课程官方仓库。

再执行一次：

```
git remote -v
```

再次检查是否添加成功。

3.1.3 同步上游仓库更改

在这里，我们以将上游仓库的 `syscall` 分支同步到本地的 `syscall` 分支为例。输入以下命令，以获取上游更改、切换到本地 `syscall` 分支、完成与上游仓库 `syscall` 分支的合并，并最后上传到自己的远程仓库：

```
git fetch --all
git checkout syscall
git merge upstream/syscall --no-edit
git push origin
```

这些命令按顺序的作用分别是：先获取远程更新；再切到本地 `syscall` 分支；接着把上游仓库的 `syscall` 分支合并进来；最后把合并后的结果推送到你自己的远程仓库。

> **检查工作区更改**
>
> 切换分支之前，请先保存工作区中的修改。如果切换时提示 `error: Your local changes to the following files would be overwritten by checkout`，请先执行 `git status` 检查工作区，并先 `commit` 你的更改。

如果你是第一次切到这个分支，也可能需要：

```
git checkout --track origin/syscall
```

这条命令会新建本地 `syscall` 分支，并让它跟踪远程的 `origin/syscall` 分支。

> **初次 checkout?**
>
> 如果你此前从未 checkout 过该分支，Git 可能会因为 `origin` 和 `upstream` 都存在同名分支，而不知道该跟踪哪一个。此时执行上面的 `git checkout --track origin/syscall` 即可：它会创建本地 `syscall` 分支，并让它跟踪 `origin/syscall`。

如果在 `git merge upstream/syscall` 这一步提示存在冲突，说明你和课程实验在同一个文件的同一个位置都做了修改，Git 无法自动完成合并。如果发生冲突，请参照 3.3.3 合并冲突更改 节处理。

3.2 将代码发布至自己的远程仓库

如果你的代码是直接从课程实验仓库 clone 而来，而不是先创建自己的远程仓库、再进行 clone，那么这一节会带你把代码发布到自己的远程仓库中。

> **这么做有什么好处?**
>
> 如果你会在不同的电脑上工作，比如实验室电脑、自己的电脑和远程环境之间来回切换，那么远程仓库可以帮助你同步自己的代码进度，也能在误操作时起到额外的备份作用。

3.2.1 设立自己的远程仓库（以 Gitee 为例）

登录 Gitee，右上角可以看到一个加号，点击下拉菜单中的“新建仓库”。



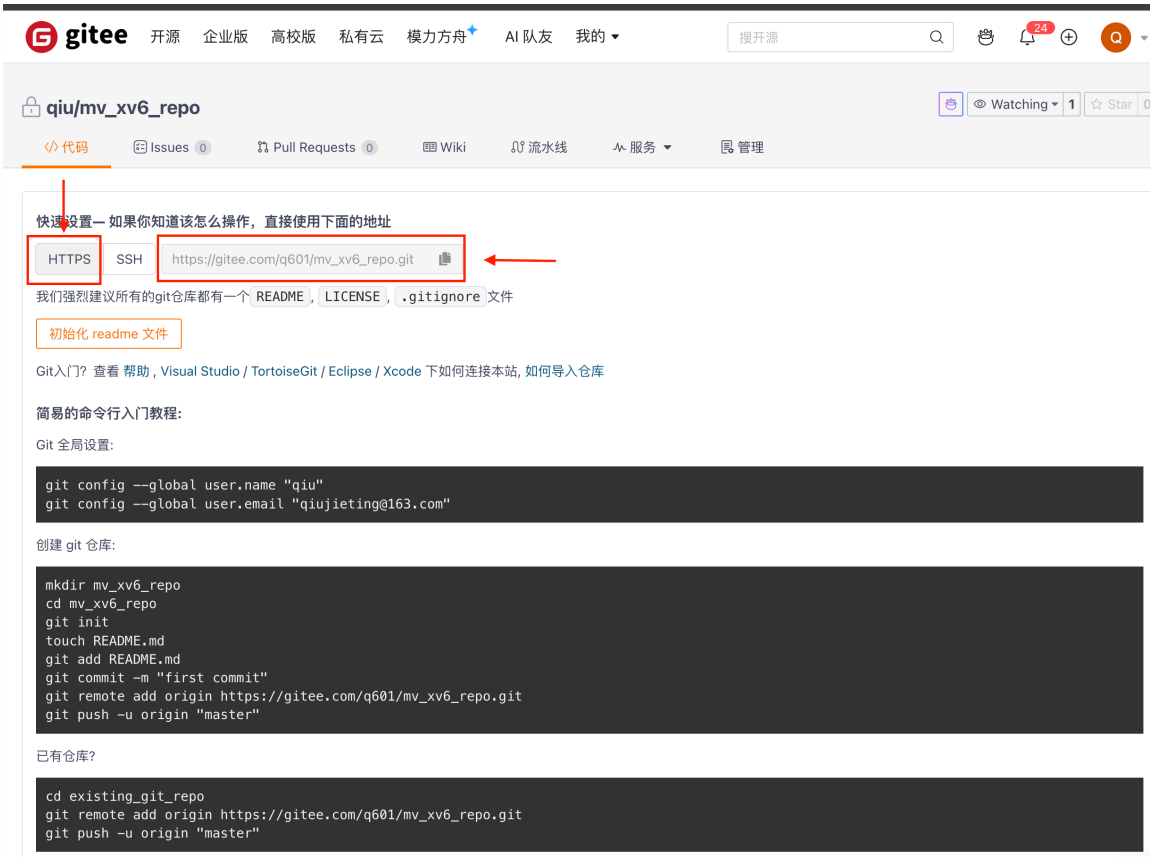
进入后，为自己的仓库设置一个名字。你可以选择公开仓库或者私有仓库；完成后点击“创建”，即可创建一个空的远程仓库。



> **请不要初始化仓库**
>
> 你的仓库已经在本地，所以这里不要再初始化仓库，也不要选择模板或分支模型。我们的目标是创建一个空的远程仓库，然后把本地仓库推送上去。

仓库创建完成后，会进入仓库主页。请复制仓库的 **HTTPS** 地址，例如：

```
https://gitee.com/你的用户名/你的仓库名.git
```



3.2.2 清理本地工作树

在命令行中，先执行：

```
git status
```

这条命令用于检查当前工作区是否还有未提交的修改。

如果工作树是干净的，就可以直接继续下一步；如果本地还有尚未提交的更改，请先参照 3.3.1.1 Commit 或 3.3.2 使用 VSCode 内建的图形化界面完成操作 先完成一次提交。

3.2.3 修改远程仓库地址并推送

在命令行中，输入以下命令：

```
git remote set-url origin https://gitee.com/你的用户名/你的仓库名.git
```

这条命令会把当前仓库的 `origin` 地址改成你自己的远程仓库地址。

将其中的地址替换成你在 3.2.1 中复制下来的 HTTPS 仓库地址。随后，可以输入 `git remote -v` 检查远程仓库地址是否已经修改成功。

完成后，切换到你想推送的分支，并上传到远程。这里以 `syscall` 分支为例：

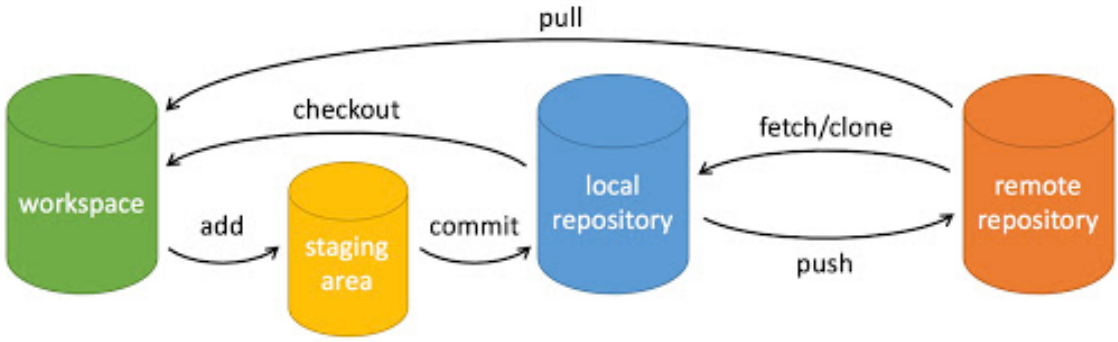
```
git push -u origin syscall
```

这条命令会把本地 `syscall` 分支推送到远程 `origin`，并建立跟踪关系；以后再推送同一分支时，通常只需要执行 `git push`。

3.3 常用 git 操作

你可以选择使用命令行完成 `commit` / `push` / `pull` / `merge` 等操作，也可以通过 VSCode 内建的图形化界面完成这些操作。

Git 工作过程如下图所示：



说明：

- `workspace`：工作区
- `staging area`：暂存区 / 缓存区
- `local repository`：本地仓库
- `remote repository`：远程仓库

以下是 Git 的官方说明：

https://git-scm.com/book/zh/v2

> **什么是 Commit?**
>
> 你可以把 `Commit` 看成一个个小的里程碑，它们记录了你的代码进度。每个 `Commit` 都有独特的哈希值，也会记录是谁、在什么时候提交了这次修改，因此非常适合用来回溯、比较和管理代码版本。

> **什么是 Push / Pull?**
>
> 通过 `Commit`，我们可以在本地完成版本控制；而为了在不同设备之间同步，或者把代码保存到远程仓库中，还需要 `Push` / `Pull`。`Push` 是把本地的提交上传到远程仓库，`Pull` 则是把远程仓库中的更新同步到本地。

> **什么是 Merge?**
>
> 当两个分支都发生了修改，或者你要把上游仓库的更新合并进本地分支时，就会用到 `Merge`。如果两边改动互不冲突，Git 往往能自动合并；如果冲突了，就需要你手动决定最终保留哪一版。

3.3.1 使用命令行完成操作

3.3.1.1 Commit

```
git add 你要保存更改的文件们
# 或者，git add --all 以添加所有更改
git commit -m "本次 Commit 你做了什么"
```

- `git add`：把修改加入暂存区，告诉 Git“这些内容要进入下一次提交”；例如 `git add .` 表示把当前文件夹及其子文件夹中的修改一起加入暂存区。
- `git commit -m ...`：把暂存区中的修改保存成一次提交，并写上这次提交的说明。

建议每次 `Commit` 都使用有效的提交信息，以便自己后续追踪代码进度。

3.3.1.2 push

`push` 可以将自己的进度同步到远程仓库。

```
git push
```

这条命令会把本地已经提交的内容推送到当前跟踪的远程分支。

3.3.1.3 pull

`pull` 可以将远程仓库的进度同步到本地。

```
git pull
```

这条命令会把远程分支的最新更新拉到本地，并尝试自动合并。

3.3.1.4 merge

`merge` 一般用于分支之间的合并，也可以用于把上游远程分支合并到本地分支。

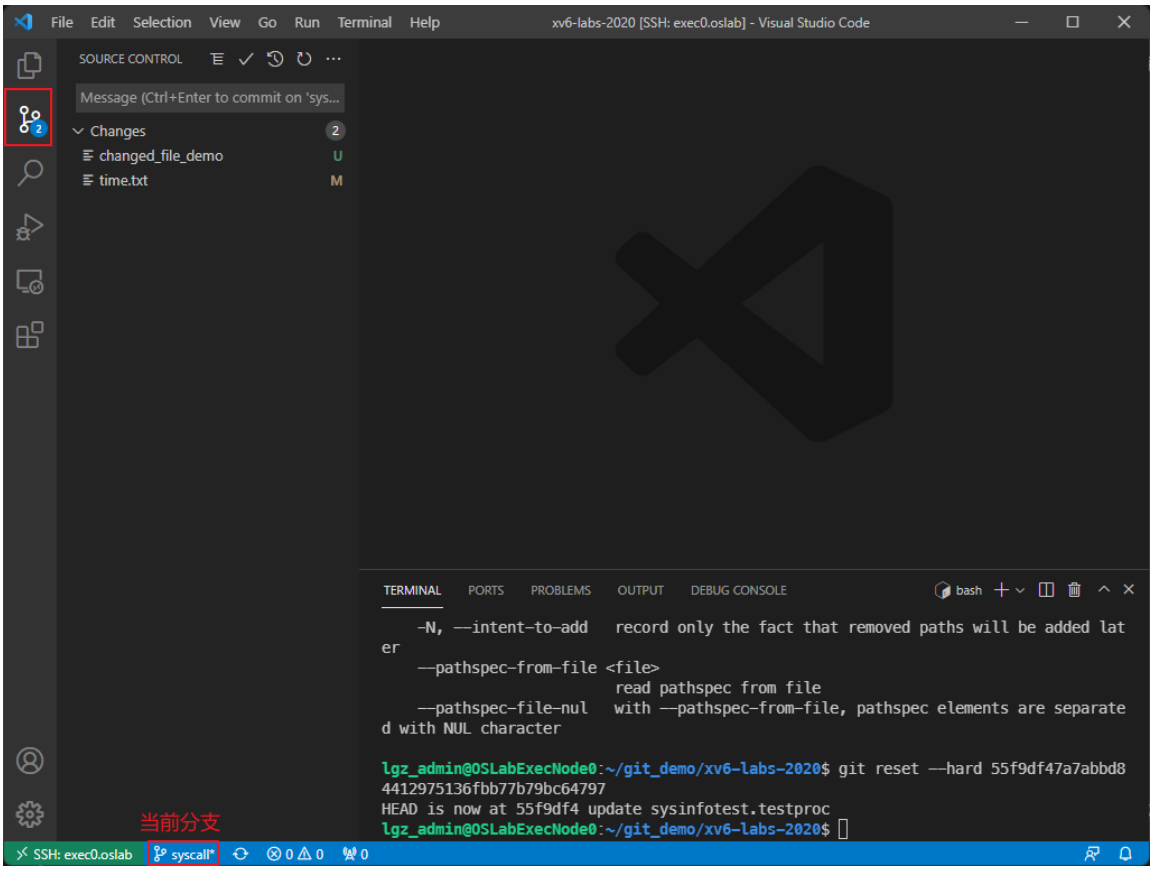
```
git checkout 你想合并的本地分支
git merge 远程仓库名称/分支名称
```

- `git checkout ...`：先切到你准备接收合并结果的本地分支；
- `git merge ...`：再把另一个分支的修改合并进当前分支。

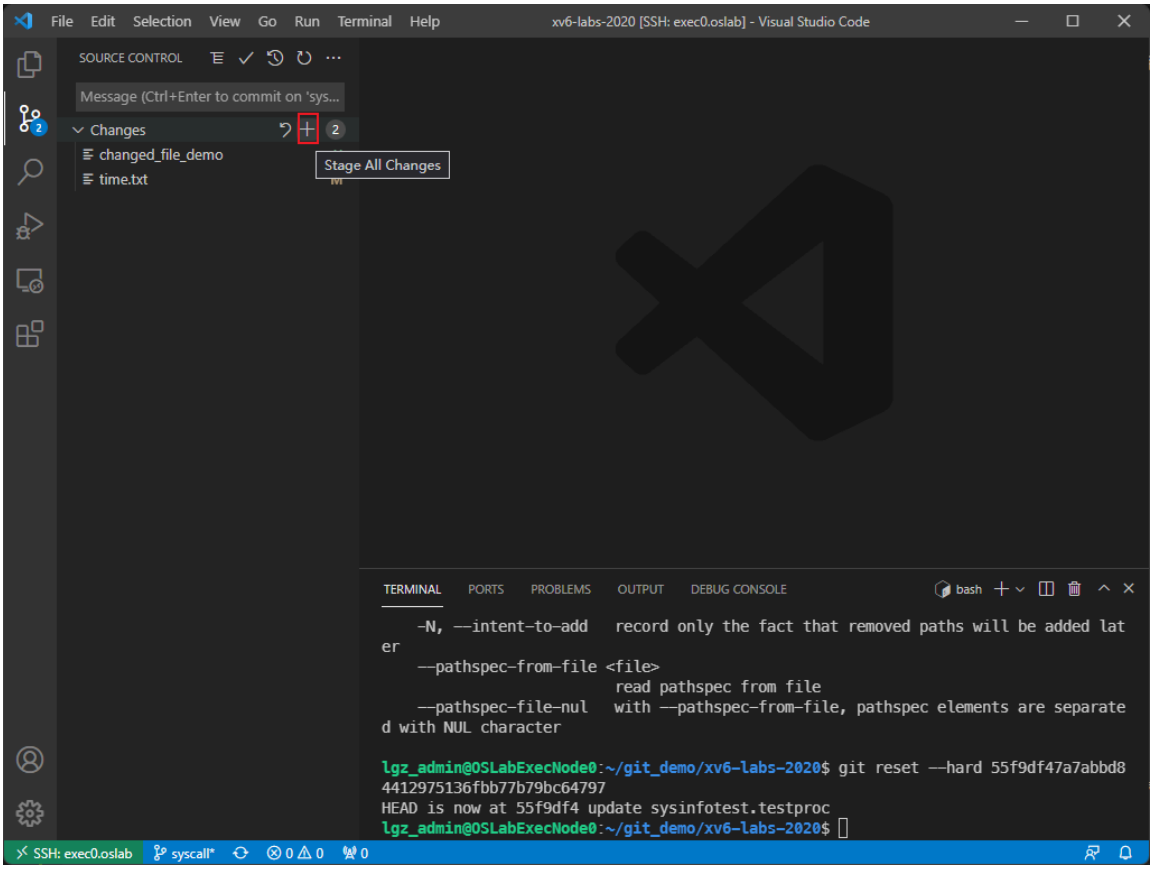
> **仓库权限**
>
> 你可以从公共仓库拉取，但只能推送到自己有权限的仓库。如果当前 `origin` 还不是你自己的远程仓库，请先参照 3.2 完成设置。

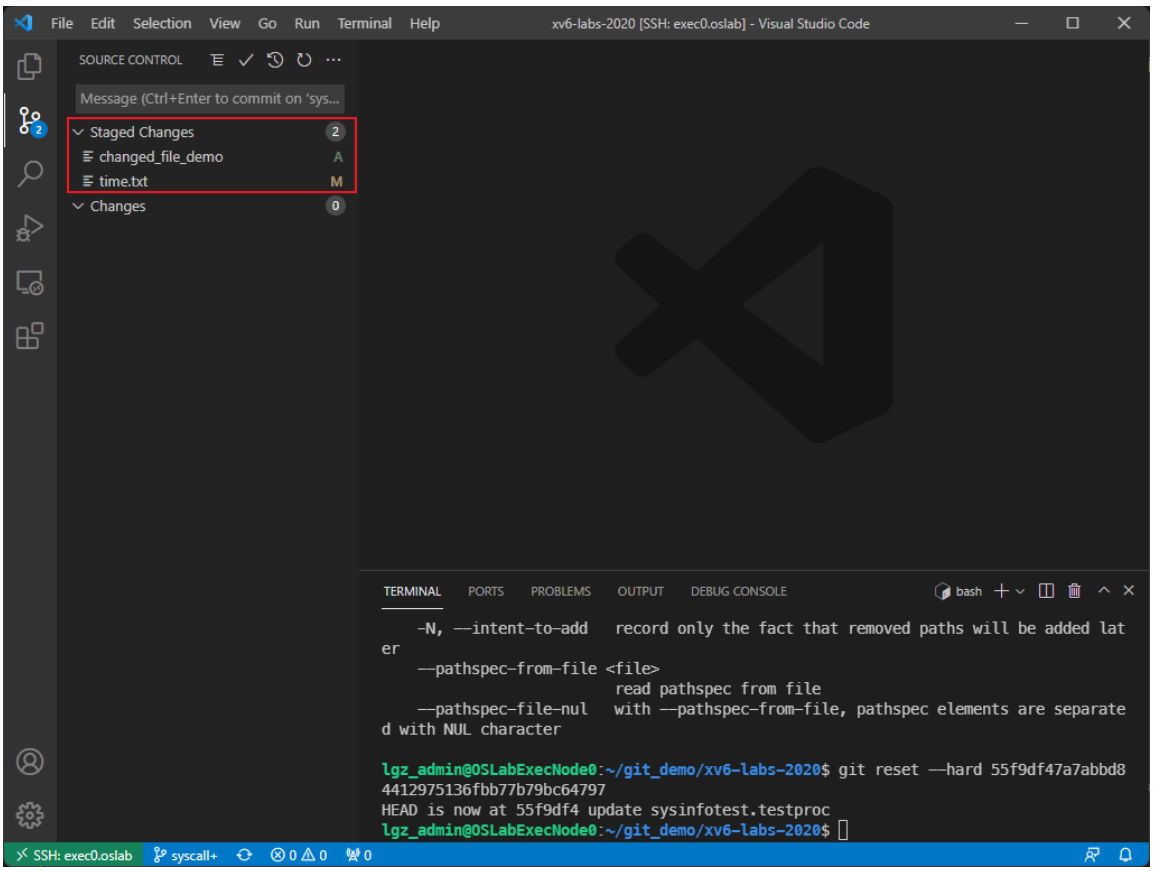
3.3.2 使用 VSCode 内建的图形化界面完成操作

VSCode 的版本控制界面在左侧边栏中。界面下方会显示当前所处分支，点击后可以切换分支；右侧的圆形双箭头表示与远程仓库同步。右侧 `Changes` 显示工作区中被修改的文件，而 `Staged` 则显示已经准备加入下一次 `Commit` 的修改。下图中，就有两个文件处于被修改状态：

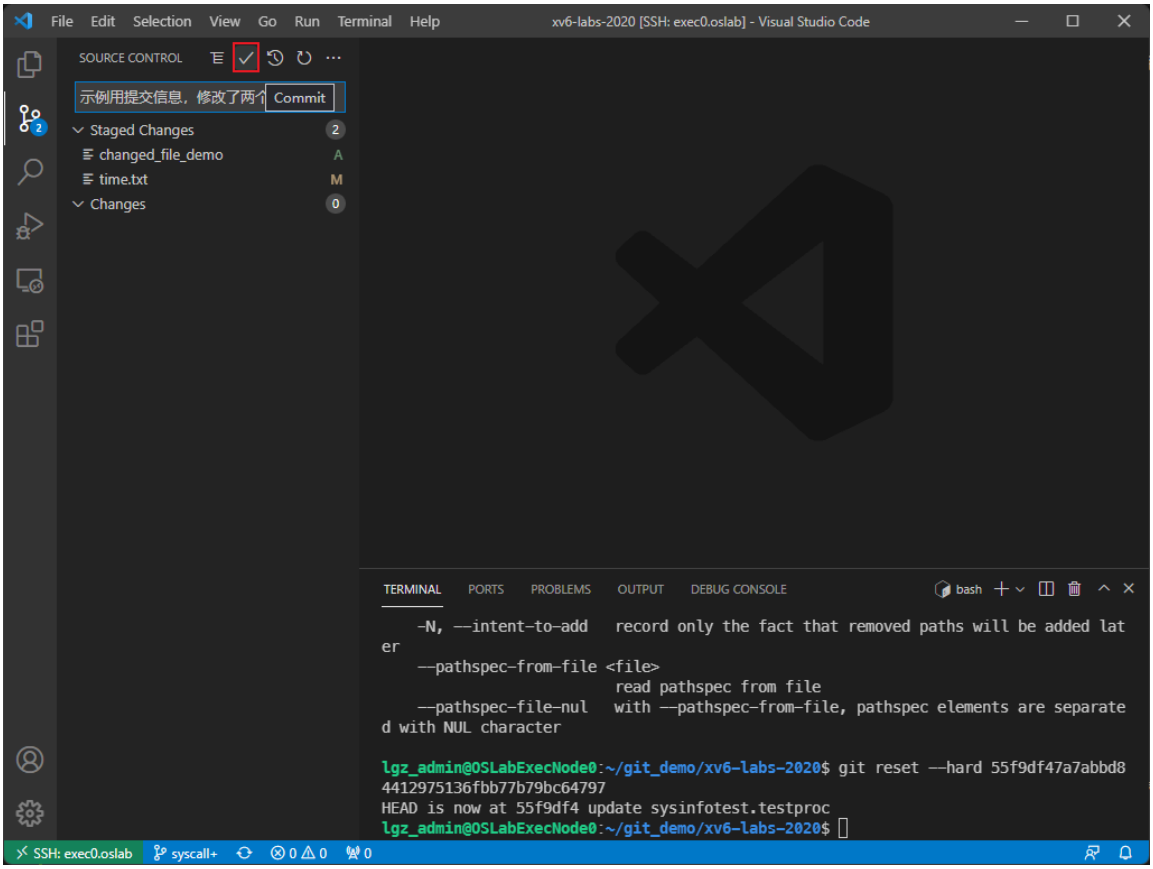


鼠标移到 `Changes` 或下面列出的文件名上，点击右侧的 `+`，即可把文件加入 `Staged` 区：

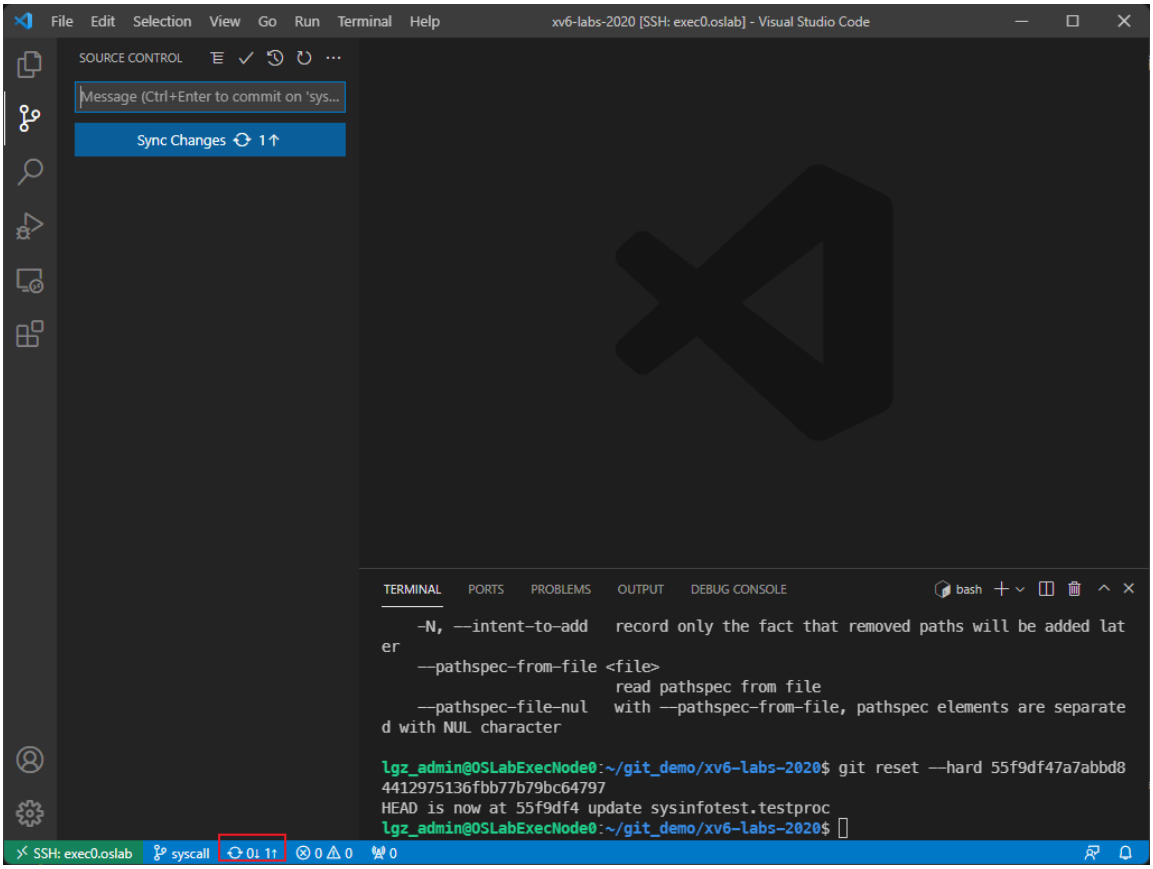




此时，这些文件已经准备好被 `Commit`。在上方输入框中填写提交信息，并点击提交按钮即可：



如果已经绑定远程仓库，那么在界面下方的同步按钮处，通常可以看到有多少提交需要拉取、多少提交需要推送。点击这个按钮，或者点击界面上方的 `Sync Changes`，就可以完成同步：

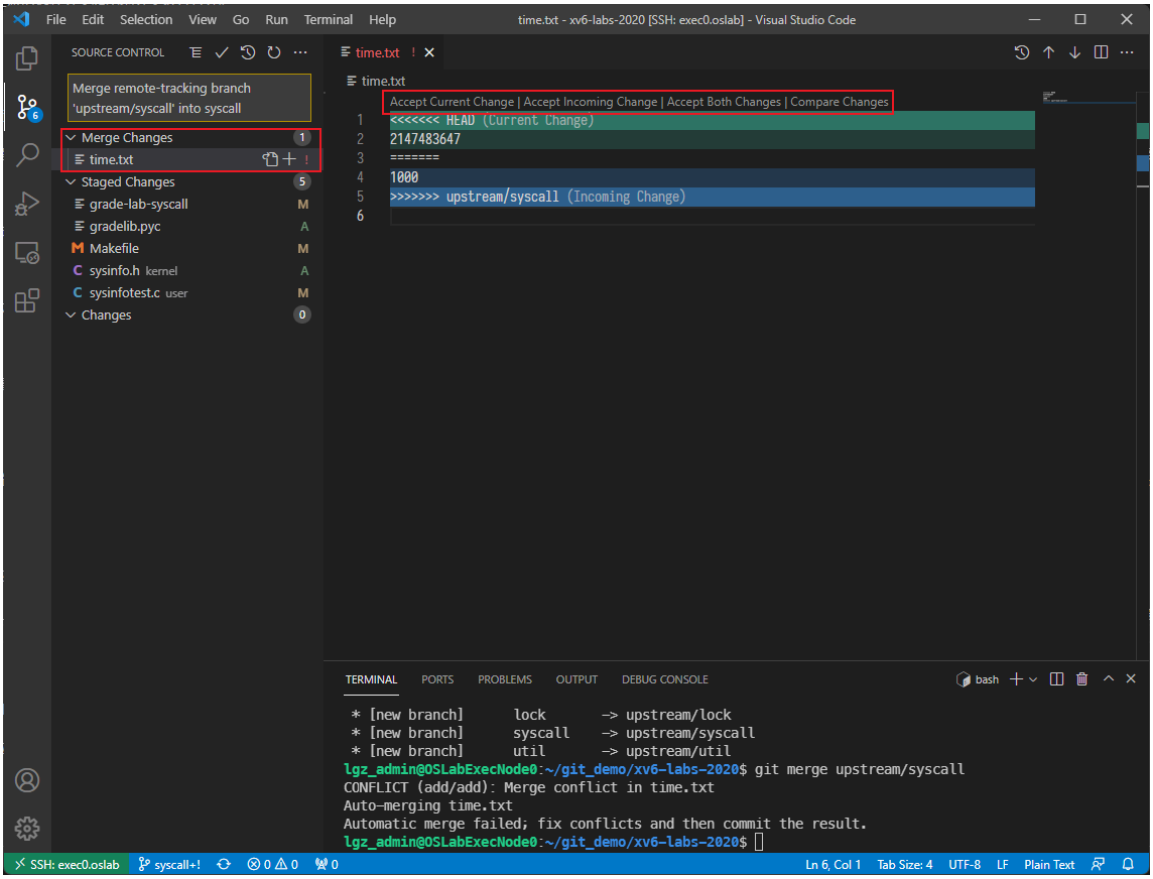


这样，你的代码就保存到远程仓库中了。

> **仓库权限**
>
> 你可以从公共仓库拉取，但只能推送到自己有权限的仓库。如果当前 `origin` 还不是你自己的远程仓库，请先参照 3.2 完成设置。

3.3.3 合并冲突更改

如果有两个 `Commit` 的内容出现冲突，那么在 `merge` 时就可能需要手动合并冲突。一般来说，推荐优先在 VSCode 中处理。例如，下图中本地 `syscall` 的修改与远程 `upstream/origin` 的修改发生了冲突：



从左侧红框中的 **Merge Changes** 区域可以看到发生冲突的文件。点开文件后，右侧会出现冲突视图：上方通常表示当前（本地）的修改，下方通常表示远程的修改。根据实际情况，可以从上方几个按钮中选择合适的那个。

例如，如果你想保留自己的修改、丢弃远程那一侧的修改，通常就选择 `Accept Current Change`。完成后，像平时提交代码一样，将处理后的文件加入 `Stage`，再执行 `Commit` 和 `Push`，就完成了这次冲突合并。

如果你更习惯使用命令行，也可以手动打开冲突文件，找到 `<<<<<<<`、`=======`、`>>>>>>>` 这些冲突标记，整理成最终版本后，执行：

```
git add 冲突文件
git commit -m "解决合并冲突"
```

这里的含义是：先把已经处理好的冲突文件重新加入暂存区，再提交一次“冲突已解决”的新提交。

如果你只是想放弃这次合并，也可以执行：

```
git merge --abort
```

这条命令会尽量放弃当前这次合并，并把仓库恢复到合并开始前的状态。

3.4 HTTPS 和 SSH 有什么区别?

Git 远程仓库常见有两种写法：

```
https://gitee.com/你的用户名/你的仓库名.git
git@gitee.com:你的用户名/你的仓库名.git
```

它们的区别可以简单理解为：

- `HTTPS`：直接使用账号密码、令牌或网页授权进行认证，**本课程统一推荐这种方式**。
- `SSH`：依赖本机 SSH 密钥配置；如果密钥、账户或权限没有配好，就容易报错。

如果你只是想顺利完成实验，建议统一使用 `HTTPS`。

如果你现在的 `origin` 是 SSH 地址，可以改成 HTTPS：

```
git remote set-url origin https://gitee.com/你的用户名/你的仓库名.git
git remote -v
```

- `git remote set-url origin ...`：把 `origin` 从 SSH 地址改成 HTTPS 地址；
- `git remote -v`：再次检查远程仓库地址是否已经修改成功。

> **选读**
>
> 如果你已经熟悉 SSH 密钥、公钥绑定和远程认证流程，也可以继续使用 SSH；但这不是本课程的统一要求。

4. 一句话建议

如果你只记住 4 件事，就记住这 4 件：

1. **写代码主要用 VSCode**
2. **每次做完一点就 `git commit`**
3. **切换分支前先 `git status`**
4. **不能 push 时先检查 `git remote -v`，优先使用 HTTPS**

Linux开发环境基础知识

先说结论

做实验时，你不需要一下子学会所有 Linux 命令。先掌握下面这些就够用了： **登录环境、切换目录、查看文件、运行命令、看报错、会复制删除、会用管道和重定向** 。

推荐学习方式

一边看，一边在终端里自己敲一遍。Linux 不是靠背会的，而是靠用会的。

1. 如何进入 Linux

常见方式有 3 种：

- **远程实验平台**： 推荐用 VSCode 连接，具体见远程实验平台使用指南
- **实验室/个人虚拟机**： 启动虚拟机后登录即可
- **WSL2**： Windows 同学可直接打开 Ubuntu/WSL 终端

如果你只是想尽快开始做实验， **优先推荐 VSCode + 终端** 这种方式。

2. 先理解 3 个最重要的概念

2.1 当前目录

终端里每条命令，都是在“当前目录”下执行的。

```
pwd
```

- `pwd` 的作用：显示你当前所在的位置
- 如果命令运行结果不对，第一件事通常就是先执行一次 `pwd`

2.2 路径

Linux 用路径表示文件位置，常见两种：

- **绝对路径**： 从根目录 `/` 开始写，例如 `/home/学号/xv6-oslabs-hitsz`

- **相对路径**：从当前目录开始写，例如 `./kernel`、`../user`

3 个特别常见的写法：

- `.`：当前目录
- `..`：上一级目录
- `~`：家目录

例子：

```
cd ~
pwd
cd ..
pwd
```

2.3 文件和目录是一棵树

你可以把 Linux 文件系统理解成一棵树：

- `/` 是根
- 每个目录下面还可以继续有目录和文件
- 你平时最常待的地方通常是自己的家目录 `~`

3. 最常用命令

下面这些命令，建议先学会。

3.1 目录与文件

命令	作用	常见用法
`pwd`	查看当前目录	`pwd`
`ls`	查看目录内容	`ls`、`ls -lah`
`cd`	切换目录	`cd ~`、`cd ..`、`cd -`
`mkdir`	创建目录	`mkdir lab1`、`mkdir -p a/b/c`
`touch`	创建空文件	`touch a.txt`

命令	作用	常见用法
`cp`	复制文件/目录	`cp a b`、`cp -r dir1 dir2`
`mv`	移动/重命名	`mv a.txt b.txt`
`rm`	删除文件/目录	`rm a.txt`、`rm -r dir`
`cat`	直接查看文件内容	`cat README.md`
`less`	分页查看文件	`less README.md`

> ⚠ **关于 `rm`**
>
> **请谨慎使用 `rm`。** 在 Linux 中，`rm` 删除后通常 **没有回收站**，删错了往往很难恢复。拿不准时，先用 `ls` 确认目标文件，再执行删除命令。

3.2 搜索与查看

命令	作用	常见用法
`find`	查找文件	`find . -name "*.c"`
`grep`	查找文字	`grep -n "fork" kernel/proc.c`
`head`	看文件开头几行	`head README.md`
`tail`	看文件最后几行	`tail -n 20 log.txt`

3.3 进程与系统状态

命令	作用	常见用法
`top`	查看系统进程	`top`
`ps aux`	查看当前所有进程	`ps aux`

命令	作用	常见用法
`kill`	结束进程	`kill pid`、`kill -9 pid`
`df -h`	查看磁盘空间	`df -h`
`free -h`	查看内存使用	`free -h`

4. 终端里最好先会的快捷键

- `Tab`：自动补全命令或文件名
- 连按两次 `Tab`：显示可补全项
- `↑ / ↓`：翻历史命令
- `Ctrl + C`：中止当前命令
- `Ctrl + L`：清屏
- `Ctrl + U`：删除当前行光标前的内容

如果某个程序“卡住了”，先试试 `Ctrl + C`。

5. 实验里最常见的操作流程

假设你已经进入实验目录：

```
cd ~/xv6-oslabs-hitsz
ls
git status
make qemu
```

建议养成这个习惯：

1. 先 `pwd` 看自己在哪
2. 再 `ls` 看当前目录里有什么
3. 再执行真正的命令

这样能避免很多“命令没错，但目录错了”的问题。

6. 管道与重定向

这一部分实验里会经常见到，但只要先会最常用的几种即可。

6.1 管道 `|`

管道的意思是： **把前一个命令的输出，交给后一个命令继续处理** 。

例子：

```
ls | grep kernel
ps aux | grep qemu
cat README.md | grep xv6
```

你可以简单理解成“把两条命令接起来用”。

6.2 输出重定向 `>` 和 `>>`

- `>`：覆盖写入文件
- `>>`：追加写入文件

例子：

```
echo hello > a.txt
echo world >> a.txt
cat a.txt
```

6.3 输入重定向 `<`

把文件内容作为命令输入：

```
wc -l < a.txt
```

6.4 错误输出重定向 `2>`

有些命令报错信息不会进普通输出，而是进入“错误输出”。

```
make 2> err.log
```

如果想把正常输出和报错都保存下来：

```
make > build.log 2>&1
```

7. 环境变量：先会最常用的两个

环境变量可以理解成“系统记住的一些配置”。

7.1 查看环境变量

```
env
echo $HOME
echo $PATH
```

最常用的两个：

- `HOME`：你的家目录
- `PATH`：系统去哪些目录里找命令

7.2 临时修改环境变量

```
export PATH=$PATH:/test
echo $PATH
```

这类修改通常只对 **当前终端** 有效。

7.3 永久生效怎么做

如果指导书明确要求你配置环境变量，通常做法是把命令写进配置文件：

- `bash` 用户常见是 `~/.bashrc`
- `zsh` 用户常见是 `~/.zshrc`

修改完成后执行：

```
source ~/.bashrc
```

如果你用的是 `zsh`，就改成：

```
source ~/.zshrc
```

> ⚠ **环境变量不要乱改**
>
> 如果实验没有明确要求，就不要随意修改 `PATH` 等关键变量。改错后可能导致命令无法使用。

8. 本课程高频命令补充

前面的内容足够你“会用终端”。下面这些是 **本课程实验里高频出现** 的命令，建议也一起掌握。

8.1 Git 最小集合

虽然 Git 的完整用法在实验实用工具里有更详细的说明，但至少建议先会下面这些：

命令	作用
`git clone 仓库地址`	下载代码仓库
`git status`	查看当前有没有改动
`git branch`	查看本地分支
`git checkout 分支名`	切换到指定分支
`git add 文件名`	把改动加入暂存区
`git commit -m "说明"`	保存一次版本
`git pull`	拉取远程更新
`git push`	把本地提交推到远程

对本课程来说，最常见的场景就是：

```
git status
git add 你改过的文件
git commit -m "完成 Lab1 的 sleep"
```

8.2 编译、运行、测试

XV6 实验里最常见的是这些命令：

命令	作用
`make qemu`	编译并启动 xv6
`make grade`	运行实验测试
`make clean`	清理旧编译结果和部分中间文件
`./grade-lab-util sleep`	运行某个分项测试

命令	作用
`./grade-lab-syscall yield`	运行指定测试脚本

你可以把 `make` 理解成“按 Makefile 里写好的规则自动做事”。

- `make qemu`：最常用，表示编译并运行
- `make grade`：最常用，表示跑测试
- `make clean`：当环境有点乱、旧文件影响结果时经常有用

> **如何退出 xv6**
>
> 运行 `make qemu` 后，如果你想退出 xv6，一般按：`Ctrl + a`，再按 `x`。

8.3 脚本、权限与退出码

实验里经常会直接运行脚本，比如 `test.sh`、`start.sh`。

命令	作用
`chmod +x test.sh`	给脚本增加可执行权限
`./test.sh`	运行当前目录下的脚本
`bash test.sh`	用 bash 解释器运行脚本
`echo $?`	查看上一条命令是否成功

说明：

- `./test.sh` 前面的 `./` 表示“运行当前目录里的这个文件”
- 如果提示 `Permission denied`，常常是因为没有执行权限，需要先 `chmod +x`
- 通常 `echo $?` 输出 `0` 表示上一条命令成功，非 `0` 表示失败

8.4 远程连接与环境安装时常见的命令

如果你使用远程实验平台或自行部署环境，下面这些命令也很常见：

命令	作用
`ssh 用户名@IP -p 端口`	连接远程 Linux 主机
`sudo apt-get update`	更新软件包列表
`sudo apt-get install 包名`	安装软件
`unzip 文件名.zip`	解压 zip 文件
`source ~/.bashrc`	让配置立即生效

例如远程实验平台常见写法：

```
ssh 你的学号@10.249.12.98 -p 6666
```

8.5 Lab5 文件系统实验里常见的命令

Lab5 不是 xv6，而是用户态文件系统实验，会频繁用到这些命令：

命令	作用
`mkdir`	建目录
`touch`	建空文件
`ls`	查看目录内容
`cat`	查看文件内容
`fusermount -u ./tests/mnt`	卸载 FUSE 文件系统

如果 Lab5 测试后提示挂载点忙、文件系统没卸载干净，第一反应通常就是检查是否需要执行：

```
fusermount -u ./tests/mnt
```

9. 不会某个命令时怎么办

先按这个顺序排查：

1. `pwd`：确认自己在哪个目录
2. `ls`：确认文件在不在
3. `命令 --help`：先看帮助
4. `man 命令`：看更完整说明
5. 看报错的 **最后一行**，通常最有用

例如：

```
ls --help
man grep
```

10. 建议你自己练一遍

如果下面这些命令你都能独立敲出来，就已经足够开始做实验了：

```
cd ~
mkdir linux-test
cd linux-test
touch a.txt
mkdir dir1
cp a.txt dir1/
ls -lah
pwd
find . -name "*.txt"
echo hello > a.txt
cat a.txt
echo 'echo hello linux' > run.sh
chmod +x run.sh
./run.sh
git status
```

如果你还能看懂并使用 `git checkout`、`make qemu`、`make grade`、`./test.sh`、`fusermount -u` 这些命令，就已经足够覆盖本课程实验的大部分 Linux 使用场景。

GDB调试指南

这页能帮你做到什么

这页只做一件事： **快速比较命令行 GDB 和 VSCode 图形化调试** ，帮助你决定先看哪一篇。

Lab1 建议

对 `Lab1` 来说， **更推荐优先使用命令行 GDB** 。因为它更容易看清 xv6 的启动流程、调用栈、寄存器和基础执行过程，后面继续做实验也更顺手。

1. 两种调试方式怎么选

对比项	命令行 GDB	VSCode 图形化调试
上手难度	略高，需要记几条常用命令	更低，按钮和面板更直观
观察能力	更强，适合看调用栈、汇编、寄存器、内存	更适合看断点、单步、变量和源码位置
适合场景	看 xv6 启动流程、定位底层问题、做 `Lab1`	第一次入门、快速上手、减少命令负担
推荐人群	想真正看懂执行过程的同学	想先把调试流程跑通的同学

2. 简单建议

- 如果你现在在做 `Lab1`，建议先看 GDB 命令行调试指南
- 如果你想使用图形化界面进行调试，可以先看 VSCode 图形化界面调试指南
- 如果你已经会基础调试，后面还想继续看系统调用过程，再看 系统调用进阶（VSCode）

3. 开始前先确认

- 已经完成实验环境准备，可参考 实验平台
- 如果使用远程平台，先完成 远程实验平台使用指南
- 初学调试时建议单核运行，通常使用 `CPUS=1` 更稳定

GDB 命令行调试指南

> **这页能帮你做到什么**
>
> 看完这页后，你应该能完成三件事： **启动 xv6 的命令行调试** 、 **用几条最常用的 GDB 命令定位问题** 、 **调试一个用户态程序** 。

> **先记住一条主线**
>
> 对本课程来说，GDB 不需要一下子学很多。先掌握这条主线就够用了： **下断点** → **运行到断点** → **看变量 / 调用栈 / 汇编** → **单步继续** 。

为了帮助同学们上手，我们录制了两个演示视频，建议按需观看：

1. 【使用 GDB 调试 xv6 启动流程（1）】 https://www.bilibili.com/video/BV1zAt6e3EfU/?share_source=copy_web&vd_source=a822dcda3537564ccdd0bb45aa0afe33
2. 【使用 GDB 调试 xv6 启动流程（2）】 https://www.bilibili.com/video/BV1MrtUe7ELC/?share_source=copy_web&vd_source=a822dcda3537564ccdd0bb45aa0afe33

1. 开始前准备

1.2 安装 `gdb-dashboard`

原始 GDB 的命令行界面比较朴素，推荐安装 gdb-dashboard 插件。

先备份原有配置（如果没有这个文件，报错可以忽略）：

```
mv ~/.gdbinit ~/.gdbinit.bak
```

下载课程组维护的配置文件：

```
wget -P ~ https://gitee.com/ftutorials/gdb-dashboard/raw/master/.gdbinit
```

让 GDB 信任各目录下的 `.gdbinit`：

```
grep -qxF 'set auto-load safe-path /' ~/.gdbinit || echo 'set auto-load safe-path /' >> ~/.gdbinit
```

如果你想让界面高亮更清楚，可以额外安装 `pygments`：

```
pip install --no-cache-dir pygments
```

> **远程实验平台说明**
>
> 远程实验平台通常已经具备所需环境，`pygments` 一般不需要你再手动安装。

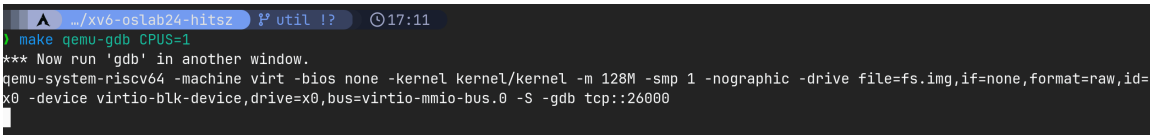
2. 最快上手流程

2.1 第一个终端：启动 QEMU 并等待 GDB 连接

在 `xv6-oslabs-hitsz` 目录下执行：

```
make qemu-gdb CPUS=1
```

这里使用 `CPUS=1`，是为了让调试更稳定，也更容易看清执行过程。



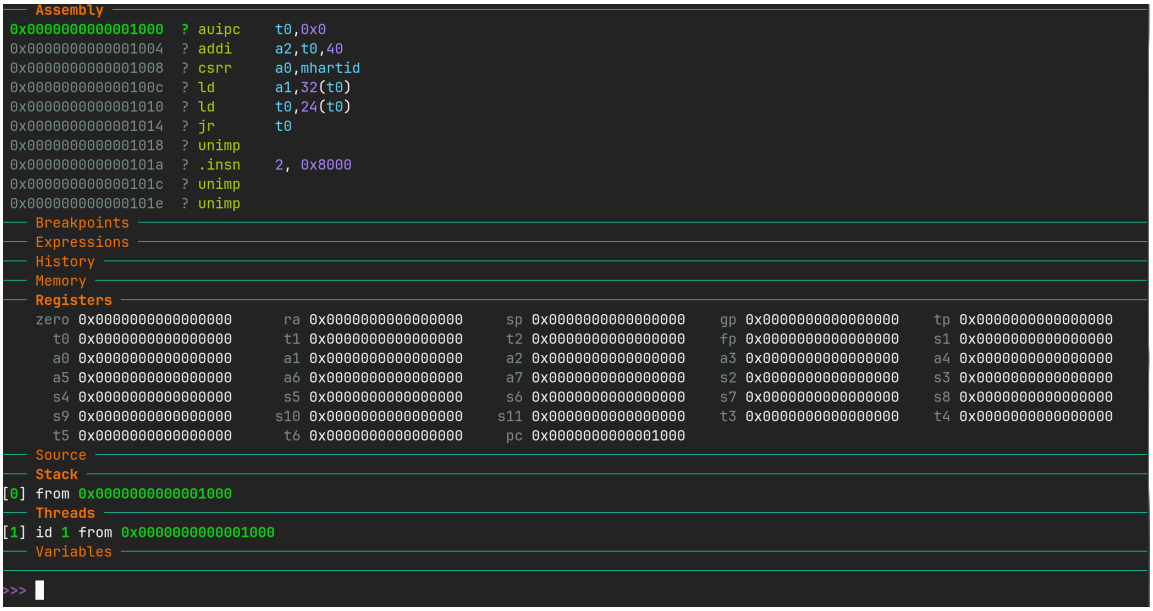
> **为什么建议单核调试?**
>
> 多核情况下，不同 CPU 会同时推进执行。刚开始学习 GDB 时，更容易看乱。对大多数实验问题来说，单核已经足够。

2.2 第二个终端：启动 GDB

在 **同一个仓库目录** 下新开一个终端，执行：

```
make gdb
```

启动后通常会看到类似下面的界面：



> **远程实验平台特别提醒**
>
> 如果你使用的是远程实验平台，`make qemu-gdb CPUS=1` 和 `make gdb` 需要在 **同一计算节点** 上执行。否则会出现端口连接超时问题。
>
> 最简单的做法是：
>
> 1. 先打开第一个终端，登录远程实验平台；
> 2. **紧接着再打开第二个终端**，再次登录远程实验平台；
> 3. 在第一个终端运行 `make qemu-gdb CPUS=1`，在第二个终端运行 `make gdb`。
>
> 登录后也可以先看命令行提示符里的主机名，例如 `学号@comp0:~$`、`学号@comp1:~$`。只有两个终端里的 `comp?` 相同，才说明它们处在同一计算节点上。更详细的说明可参考 实验环境问题 FAQ。

2.3 第一次建议先敲这几条命令

```
b main
c
bt
p cpus
x/i $pc
q
```

这几条命令分别完成：

- `b main`：在 `main` 函数打断点
- `c`：继续运行到断点
- `bt`：看函数调用栈

- `p cpus`：打印变量
- `x/i $pc`：查看当前程序计数器对应的汇编指令
- `q`：退出 GDB

如果你第一次能顺利跑完这 6 条命令，就已经完成入门了。

3. 先会这些最常用命令

先熟悉下面这些命令，后面调试 xv6 时基本都会用到：

命令	完整写法	作用	最常见场景
`help 命令`	`help 命令`	查看命令帮助	忘记命令语法时
`b 位置`	`break 位置`	设置断点	想在某个函数或某一行停下时
`c`	`continue`	继续运行	从当前断点继续跑时
`n`	`next`	源码单步，不进入函数	想按行排查，但先不进入函数时
`s`	`step`	源码单步，会进入函数	想跟进函数内部时
`ni` / `si`	`nexti` / `stepi`	汇编单步	调试系统调用、陷入与返回时
`finish`	`finish`	运行到当前函数返回	已经进函数，但想直接回到调用处时
`p 表达式`	`print 表达式`	打印变量或表达式	如 `p p->pid`
`x/格式 地址`	`examine /格式 地址`	查看指定地址的内存内容	如 `x/10i $pc`、`x/16x 0x80000000`
`i r`	`info registers`	查看寄存器值	想看 `pc`、`sp`、`a0` 等寄存器时

命令	完整写法	作用	最常见场景
`bt`	`backtrace`	查看调用栈	想知道当前是怎么走到这里时
`q`	`quit`	退出 GDB	结束本次调试时

3.1 `help` 命令

不会某条命令时，先试：

```
help n
help break
```

```
>>> help
List of classes of commands:

aliases -- User-defined aliases of other commands.
breakpoints -- Making program stop at certain points.
data -- Examining data.
files -- Specifying and examining files.
internals -- Maintenance commands.
obscure -- Obscure features.
running -- Running the program.
stack -- Examining the stack.
status -- Status inquiries.
support -- Support facilities.
text-user-interface -- TUI is the GDB text based interface.
tracepoints -- Tracing of program execution without stopping the program.
user-defined -- User-defined commands.

Type "help" followed by a class name for a list of commands in that class.
Type "help all" for the list of all commands.
Type "help" followed by command name for full documentation.
Type "apropos word" to search for commands related to "word".
Type "apropos -v word" for full documentation of commands related to "word".
Command name abbreviations are allowed if unambiguous.
```

例如 `help n` 会告诉你 `n` 是 `next` 的缩写，并说明它的用途。

```
>>> help n
next, n
Step program, proceeding through subroutine calls.
Usage: next [N]
Unlike "step", if the current source line calls a subroutine,
this command does not enter the subroutine, but instead steps over
the call, in effect treating it as a single source line.
```

3.2 单步执行怎么选

- `n`：按源码走一步，但 **不进入** 函数
- `s`：按源码走一步， **会进入** 函数
- `ni`：按汇编走一步，不进入调用
- `si`：按汇编走一步，可以进入调用

如果你在看 xv6 的启动流程、系统调用、陷入 / 返回、汇编过程，通常 `ni` / `si` 更有帮助。

3.3 观察状态的三件套

最常用的观察命令通常是：

```
p 变量名
bt
info registers
```

如果你还想看当前位置的汇编和栈附近的内容，也很常用：

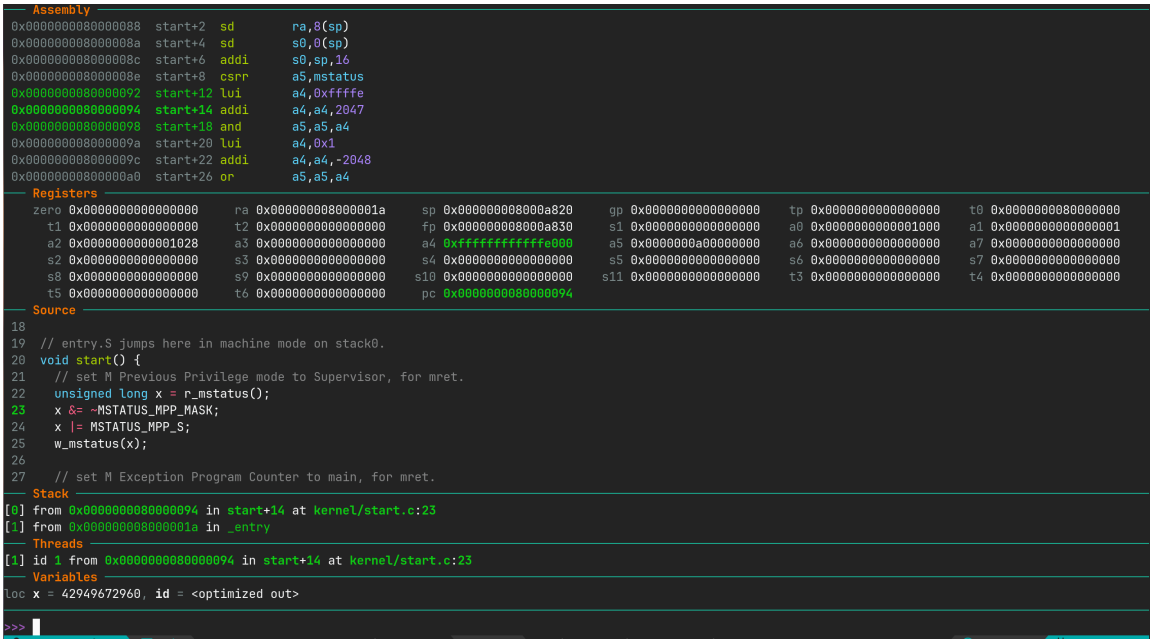
```
x/10i $pc
x/16x $sp
```

你可以把它们理解成三类问题：

- `p`：值对不对
- `bt`：怎么走到这里的
- `info registers` / `x`：机器当前到底在干什么

4. `gdb-dashboard` 够用即可

`gdb-dashboard` 会把源码、汇编、寄存器、栈等信息集中显示在一个界面里。



对本课程来说，最常看的区域通常是：

- `assembly`：当前汇编位置
- `source`：当前源码位置
- `registers`：寄存器
- `stack`：调用栈
- `variables`：当前作用域变量

4.1 几条够用的 dashboard 命令

```
dashboard
dashboard history
dashboard -layout assembly source registers stack variables
dash -layout !
tui focus src
tui disable
```

它们的作用分别是：

- `dashboard`：刷新 dashboard 布局
- `dashboard history`：显示 / 隐藏 `history` 区域
- `dashboard -layout ...`：只保留你想看的区域
- `dash -layout !`：恢复默认布局
- `tui focus src`：专注查看源码窗口
- `tui disable`：退出 TUI 模式，回到 dashboard

如果你刚开始用，不必折腾太多个性化配置，默认布局已经够用。

5. 一套典型的调试方法

如果你已经知道“程序大概在哪一段出问题”，推荐按这个顺序调：

1. 在怀疑位置打断点
2. `c` 跑到断点
3. 用 `p` 看关键变量
4. 用 `bt` 看是谁调用到这里的
5. 必要时用 `n` / `s` / `ni` / `si` 单步推进
6. 用 `x/i $pc` 看当前汇编

例如：

```
b kernel/proc.c:exit
c
bt
p p->pid
p p->state
x/10i $pc
```

如果你现在不知道问题在哪，就先多用 `bt` 和 `p`；如果你已经知道就是某几条指令不对，就多用 `ni` / `si` 。

6. 课程里最常见的几个问题

6.1 `make gdb` 连不上 QEMU

先检查这三件事：

1. 第一个终端是否还停在 `make qemu-gdb CPUS=1`
2. 第二个终端是否在 **同一个仓库目录** 下执行 `make gdb`
3. 如果你在远程实验平台上，两个终端是否在 **同一计算节点** 上

多数“连不上”的问题，都是这三项里的某一项没有满足。

6.2 内核断点能停下，用户态断点没反应

这通常不是 GDB 坏了，而是 **用户程序的符号还没有导入** 。

对用户态程序调试，常见做法是：

```
add-symbol-file user/_ls
b user/ls.c:main
```

如果你还没导入符号，GDB 很难把地址和 `user/ls.c` 这样的源码位置对应起来。

6.3 不知道该用源码单步还是汇编单步

可以直接这样判断：

- 你想理解“这段 C 代码怎么走” → 先用 `n` / `s`
- 你想看“陷入、返回、寄存器、跳转到底怎么变” → 先用 `ni` / `si`

对操作系统实验来说，很多关键问题都发生在“源码视角不够看清”的地方，所以别怕看汇编。

7. 常用调试技巧

7.1 暂停程序

调试过程中，如果程序一直在跑，可以直接在 GDB 中按 `Ctrl + C` 暂停。

7.2 条件断点和监视点

这些不是一开始必须会，但一旦会了，能省很多时间。

- `break 位置 if 条件`：条件断点
- `cond 编号 条件`：给已有断点加条件
- `watch 表达式`：表达式变化时停下
- `watch -l 地址`：某个地址的内容变化时停下

例如：

```
break kernel/proc.c:123 if p->pid == 3
watch p->state
```

7.3 历史与脚本

GDB 的命令历史通常保存在 `.gdb_history` 文件里（一般在你启动 GDB 的目录下）。

这对本课程有两个直接用途：

- 回看你上次是怎么调的
- 整理成 `commands.gdb` 脚本，交实验或复现过程时很方便

> **两个实用提醒**
>
> 1. 退出 GDB 后，历史才会写回 `.gdb_history`
> 2. 推荐用 `q` 退出 GDB。`q` 是一条 GDB 命令，会和前面的调试命令一起保存在 `.gdb_history` 中；`Ctrl + D` 只是向终端发送结束输入，不是 GDB 命令，所以不会出现在历史里。

如果你想把一套调试过程保存成脚本，可以这样做：

1. 从 `.gdb_history` 找到你需要的命令
2. 新建一个文本文件，比如 `commands.gdb`
3. 把这些命令贴进去
4. 下次启动 GDB 后执行：

```
source commands.gdb
```

这也是 Lab1 里导出调试命令脚本最直接的做法。

7.4 调试用户态程序

下面以调试 `ls` 为例：

1. 分别启动 `make qemu-gdb CPUS=1` 和 `make gdb`
2. 在运行 `make gdb` 的那个终端里，看到 `(gdb)` 提示符后，输入 `c`，让系统先继续运行
3. 等运行 `make qemu-gdb CPUS=1` 的窗口里出现 xv6 shell 后，在 GDB 终端里按 `Ctrl + C` 暂停
4. 继续在 GDB 终端里导入用户程序符号：

   ```
   add-symbol-file user/_ls
   ```

5. 仍然在 GDB 终端里，在 `main` 入口打断点：

   ```
   b user/ls.c:main
   c
   ```

6. 切换到 xv6 shell 所在的窗口，输入：

   ```
   ls
   ```

7. 再切回运行 `make gdb` 的那个终端，继续调试这个用户程序

> ⚠ **为什么有时断点会先停在别的地方？**
>
> GDB 是按地址打断点的，而 `ls` 的某些地址可能与别的用户程序（例如 `sh`）的地址重合。因此第一次停下时，未必就是你想要的 `ls`。如果信息明显不对，继续执行一次 `c`，往往就能到真正的断点位置。

8. 最后记住这张清单

如果你只打算先学会最核心的部分，请优先掌握下面这些：

1. `make qemu-gdb CPUS=1` + `make gdb`
2. `b`、`c`、`n`、`s`、`p`、`bt`
3. `x/10i $pc`、`info registers`
4. `source commands.gdb`
5. `add-symbol-file user/_程序名`

做到这一步，就已经足够覆盖本课程大多数命令行调试场景了。

VSCode 图形化界面调试指南

这页能帮你做到什么

看完这页后，你应该能完成三件事： **在 VSCode 中启动 xv6 图形化调试** 、 **用按钮完成断点 / 单步 / 变量查看** 、 **在需要时切回调试控制台补充 GDB 命令** 。

先记住一条主线

对本课程来说，图形化调试也不需要一下子学很多。先掌握这条主线就够用了： **下断点 → 运行到断点 → 看变量 / 调用栈 → 必要时在控制台补一条 GDB 命令** 。

为了帮助同学们上手，我们也录制了两个演示视频，建议按需观看：

1. 【1. VSCode 调试 xv6 内核代码】 https://www.bilibili.com/video/BV1ZB4y1E7X5?share_source=copy_web&vd_source=a822dcda3537564ccdd0bb45aa0afe33
2. 【2. VSCode 调试 xv6 用户代码】 https://www.bilibili.com/video/BV1i14y1Y7ZZ?share_source=copy_web&vd_source=a822dcda3537564ccdd0bb45aa0afe33

下面按步骤完成配置。

1. 设置GDB信任

在终端中，输入：

```
grep -qxF 'set auto-load safe-path /' ~/.gdbinit || echo 'set auto-load safe-path /' >> ~/.gdbinit # 使gdb可以信任所有目录的.gdbinit
```

这条命令会让 GDB 信任各目录下的 `.gdbinit` ，这样后续调试脚本才能正常加载。

2. 获取专属GDB端口号

接下来，获取你专属的GDB端口号。在控制台输入 `expr $(id -u) % 5000 + 25000` 并回车，你应该能得到一串数字：

```
ldap_example@OSLabExecNode0:~$ expr $(id -u) % 5000 + 25000
25001    // ← 这个
```

请记好这串数字，后面会用到。

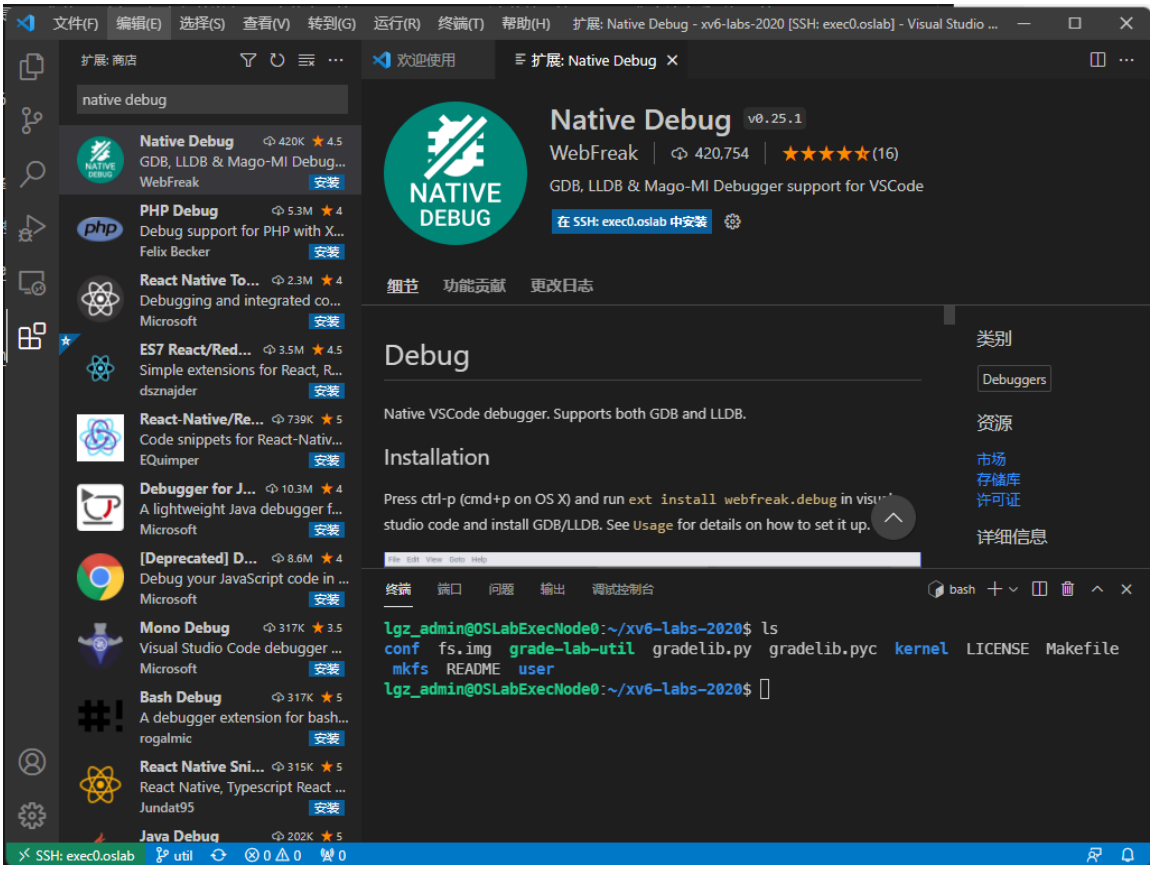
3. 设置VSCode

首先确认 **VSCode 当前打开的工作区就是 xv6 仓库根目录**，不要在外面再多套一层文件夹。按下 `Ctrl+`` 打开终端，输入 `ls`，你应该会看到类似下面的内容：

```
ldap_example@OSLabExecNode0:~/xv6-oslabs-hitsz$ ls
conf  fs.img  grade-lab-util  gradelib.py  gradelib.pyc  kernel  LICENSE
Makefile  mkfs  README  user
```

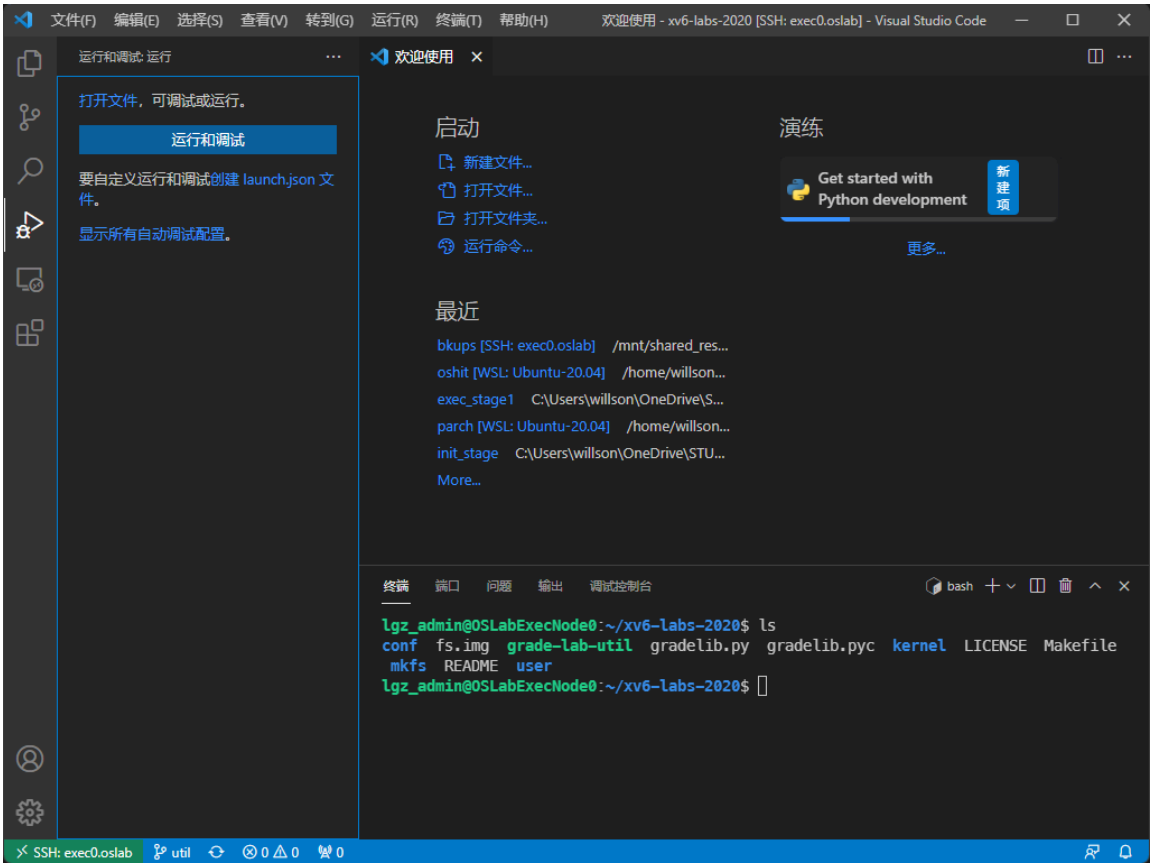
如果看到的不是这组文件，请重新打开工作区，并直接选择 xv6 所在目录。

接着在 VSCode 左侧扩展栏搜索并安装 `Native Debug` 插件：

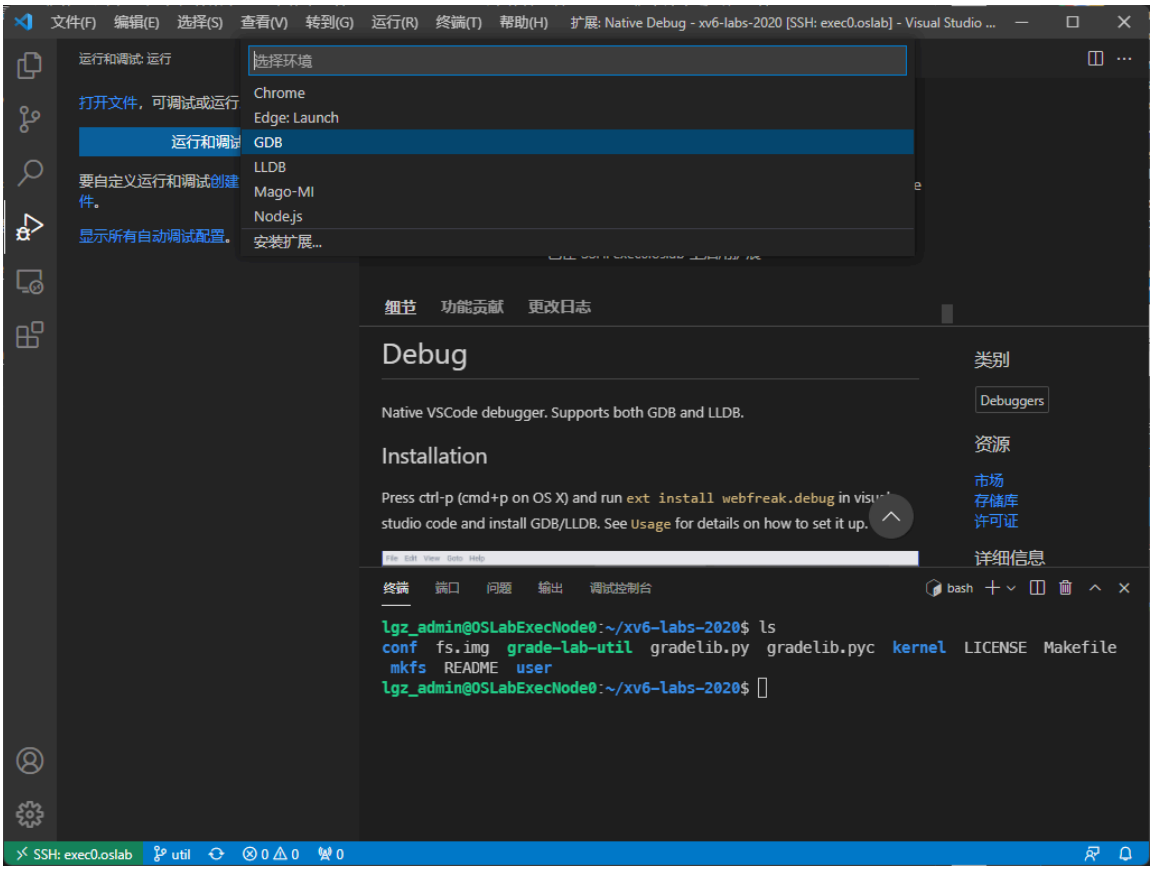


> **插件推荐**
>
> 同时也推荐安装 VSCode 的 C/C++ 插件，这样可以获得自动补全、跳转定义等常用功能。

在VSCode左侧，点击“运行和调试”选项，如图：



选择 `创建launch.json文件` ，选择GDB。



这时会打开一个名为 `launch.json` 的文件。将其中内容替换为下面的配置：

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "type": "gdb",
            "request": "attach",
            "name": "Attach to gdbserver",
            "executable": "${workspaceRoot}/kernel/kernel",
            "gdbpath": "gdb-multiarch",
            "remote": true,
            "target": "127.0.0.1:此前你得到的GDB端口号",
            "cwd": "${workspaceRoot}",
            "setupCommands": {
                "text": "source ${workspaceFolder}/.gdbinit"
            }
        }
    ]
}
```

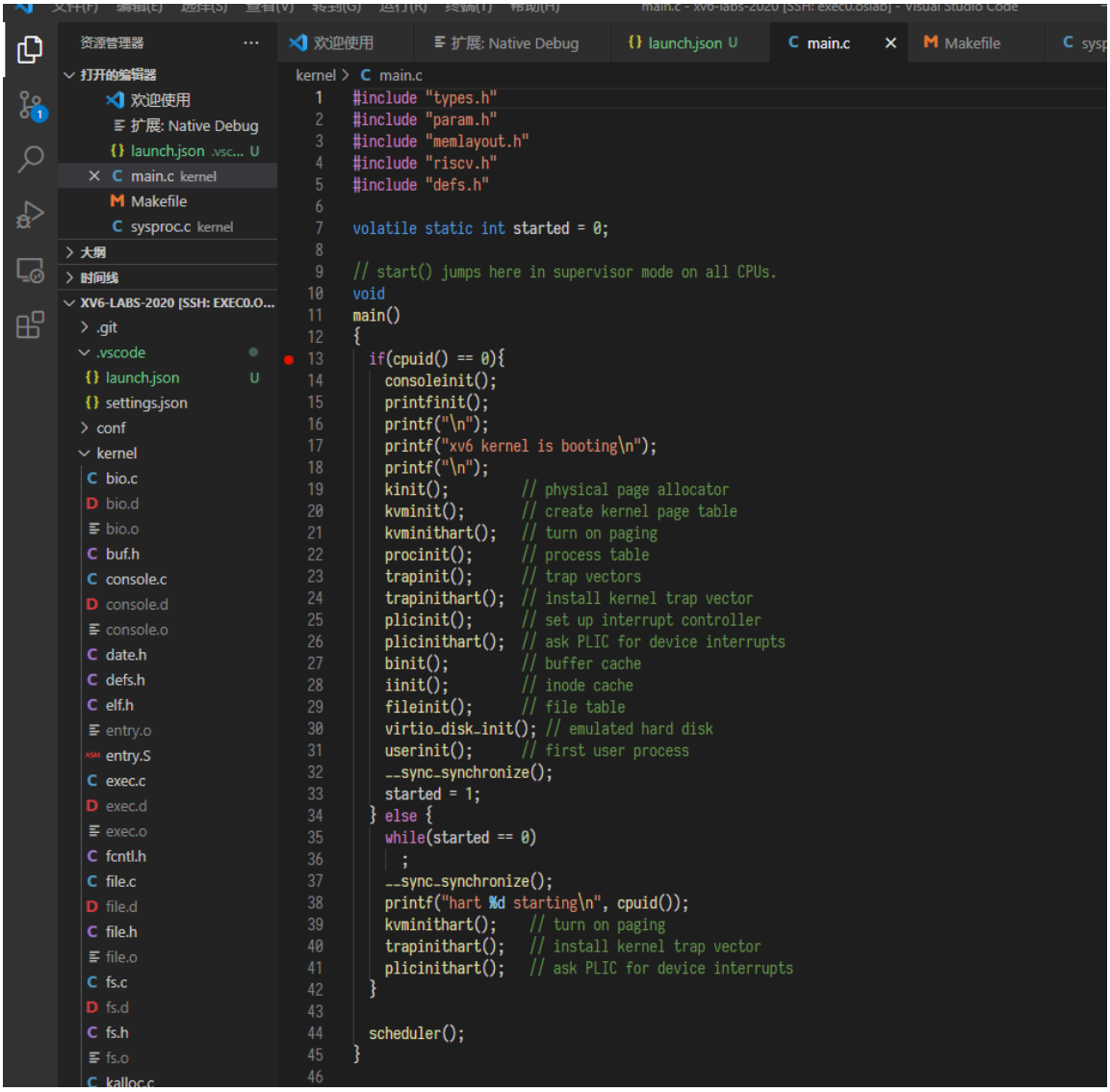
其中，第十行 `"target": "127.0.0.1:此前你得到的GDB端口号",` 里第二个冒号后的中文，需要替换成你刚才通过 `expr $(id -u) % 5000 + 25000` 得到的端口号，例如 `25001` 。保存即可。

到这里，VSCode 侧的基本配置就完成了。

4. 基础使用教学

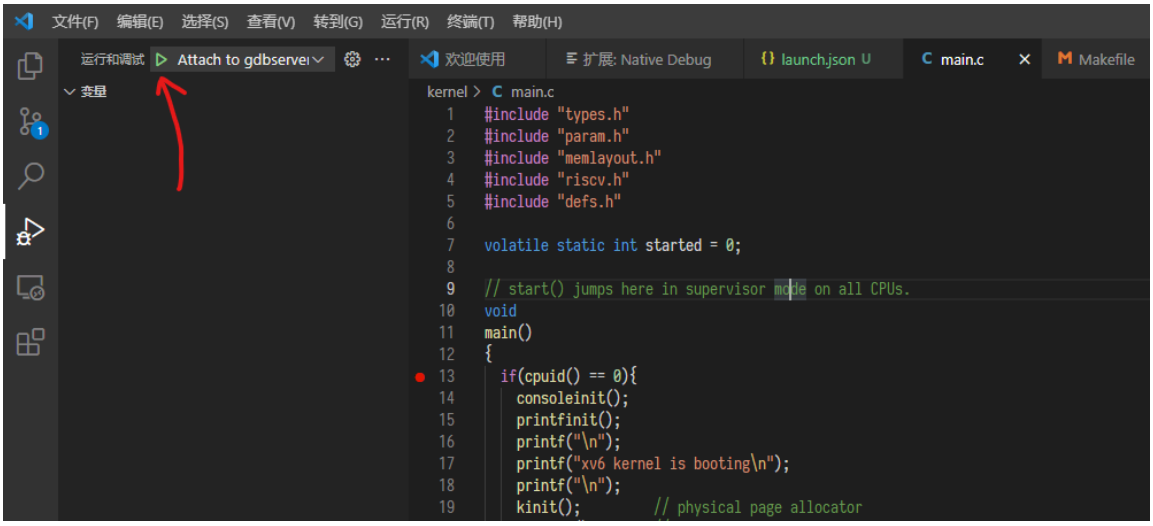
> **默认是内核调试**
>
> 默认加载的是 xv6 内核符号。若要调试用户程序，可参考下方 4.4.4 用户态程序调试 。

为演示起见，我们先在 `kernel/main.c` 第 13 行打一个断点：

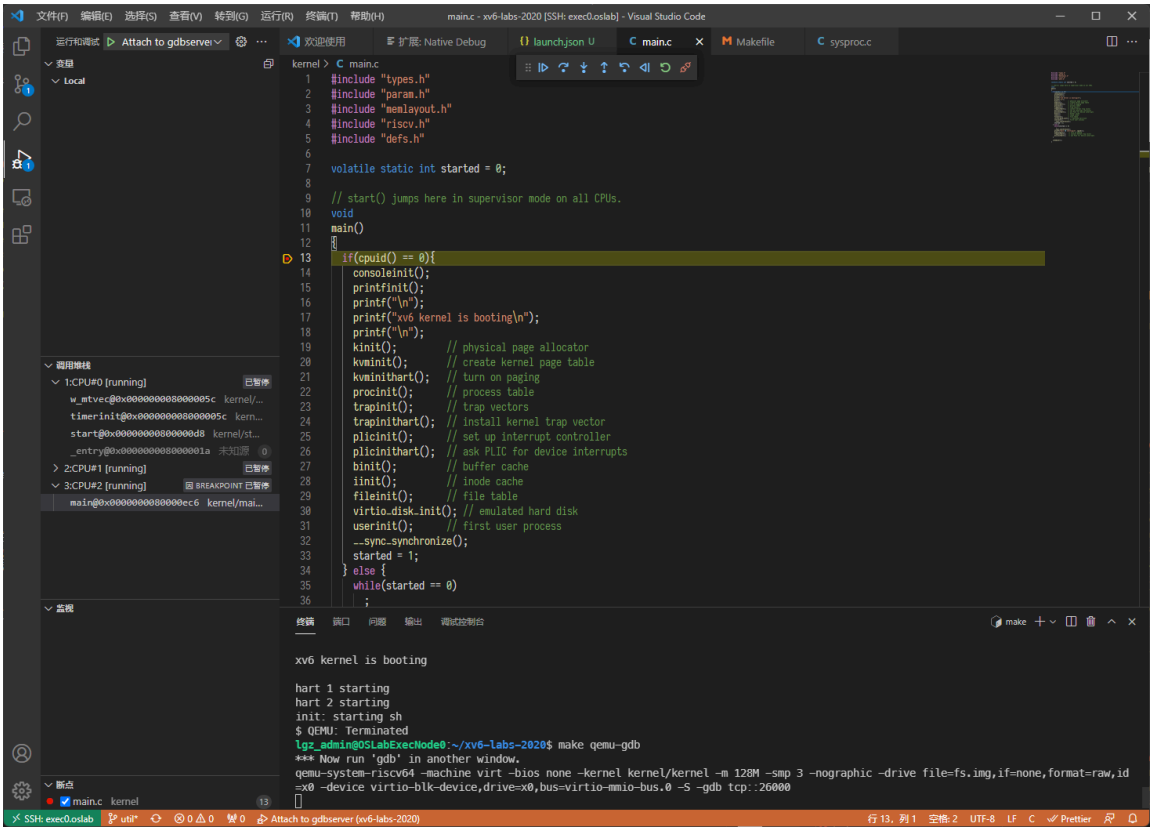


断点可以直接点击行号左侧设置。

然后在终端执行 `make qemu-gdb CPUS=1` ，再回到 VSCode 按下 `F5` ，或者点击左侧“运行和调试”并选择左上角绿色三角（`Attach to gdb`）：



现在，内核已经成功运行，并且停在断点处：



如果能看到这个状态，说明当前配置已经可以正常使用。

上方几个最常用的按钮依次是：运行、单步跳过（下一行）、单步调试（步入函数）、单步跳出（跳出函数）和断开：



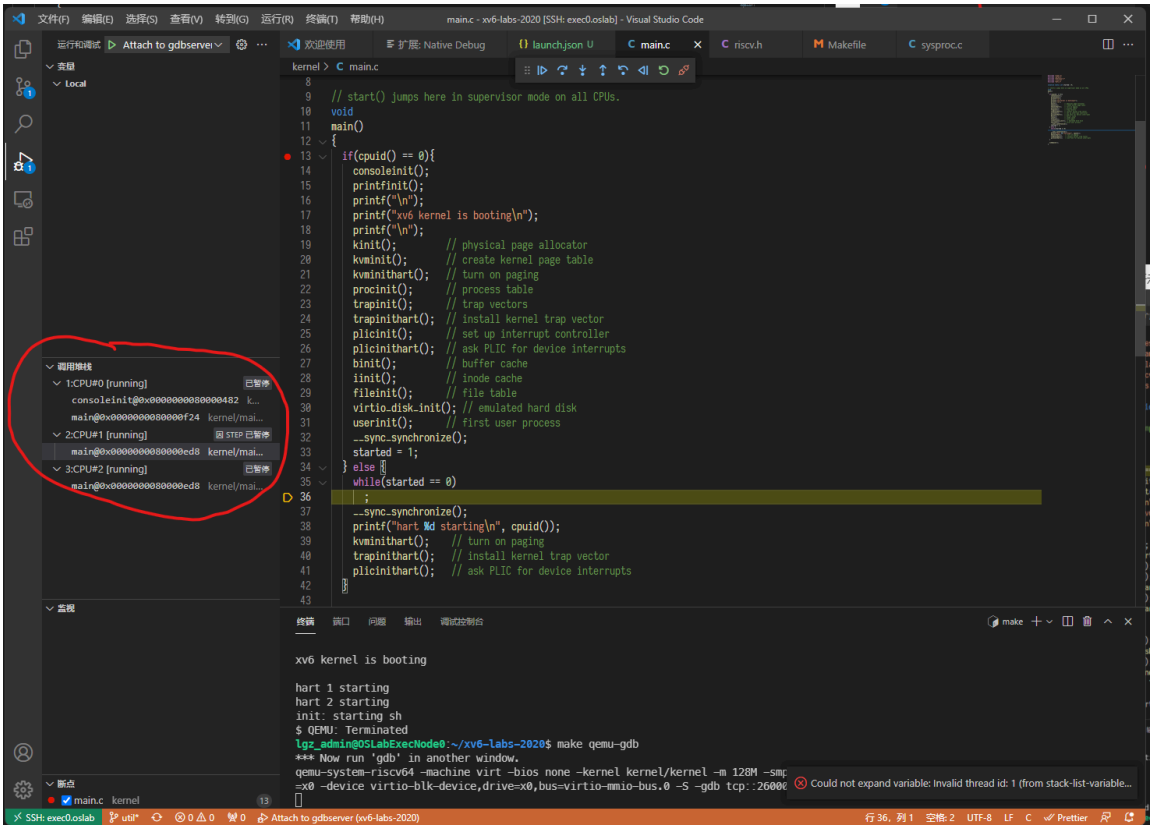
结束调试时

每次调试结束后，建议先使用红色按钮 **断开** GDB，再在 QEMU 所在终端按 `Ctrl-A` `X` 退出 QEMU。

调试期间不要改源码

调试运行中的程序时，不要一边运行一边修改源码。如果需要改代码，请先停止调试，修改后再重新启动。

如果你发现“单步”时程序跳得很快，通常是因为当前启用了多核。左侧线程列表里会看到多个 CPU 同时运行：



单核更适合入门调试

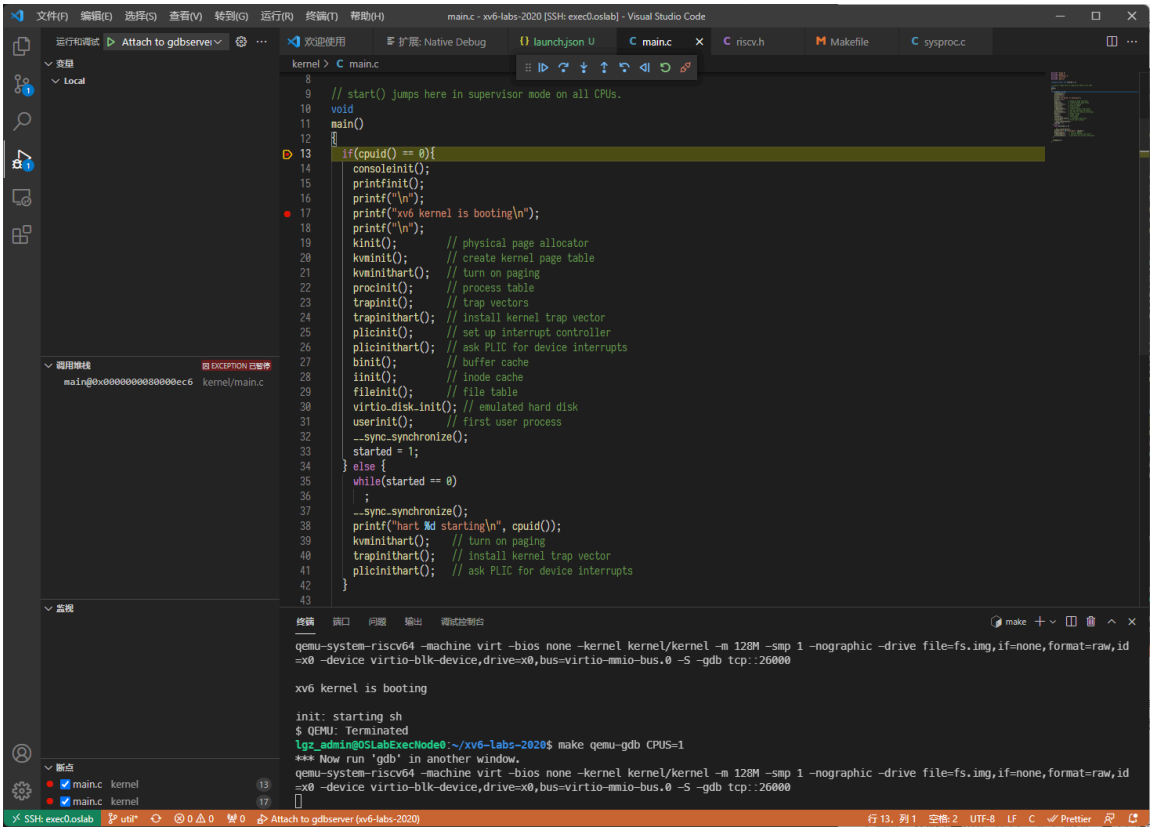
如果你想先把执行过程看清楚，建议用 `make qemu-gdb CPUS=1` 以单核模式启动。

> **但不要把单核当成最终目标**
>
> 某些 lab 需要正确处理多核竞争。`CPUS=1` 主要用于调试；如果代码只能在单核下运行，提交时仍可能丢分。

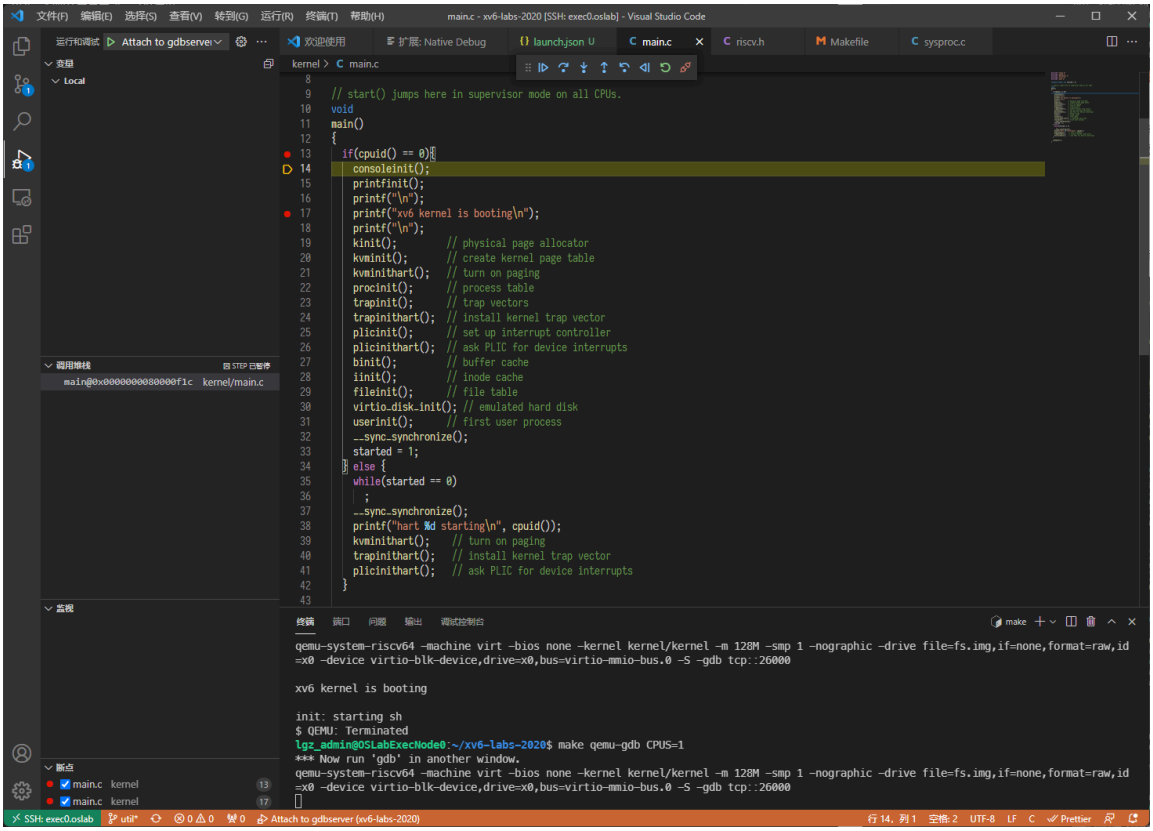
后续示例默认按单核模式演示。

4.1 单步跳过

左侧第二个按钮是单步跳过（向下一行）。点击后，程序会按源码向下执行一行，但不会进入函数。下面先看初始状态：

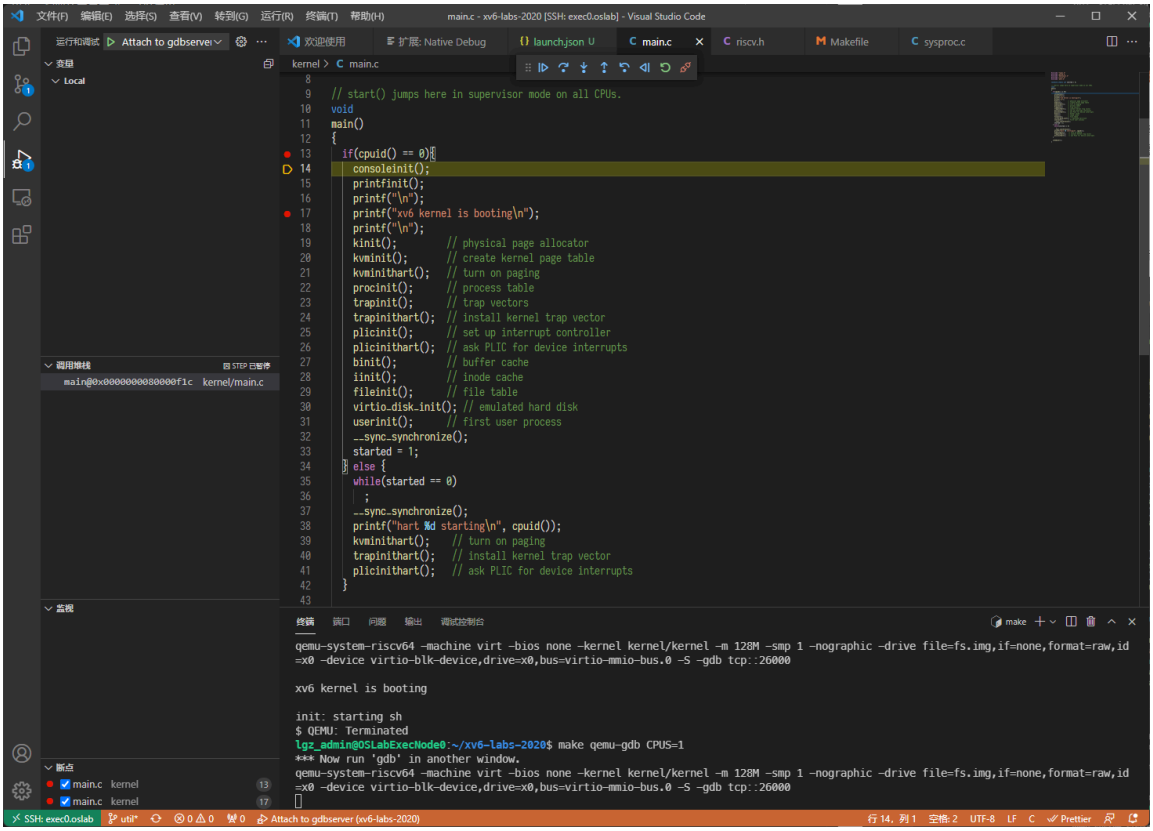


按下单步跳过按钮，则程序向下执行一行：

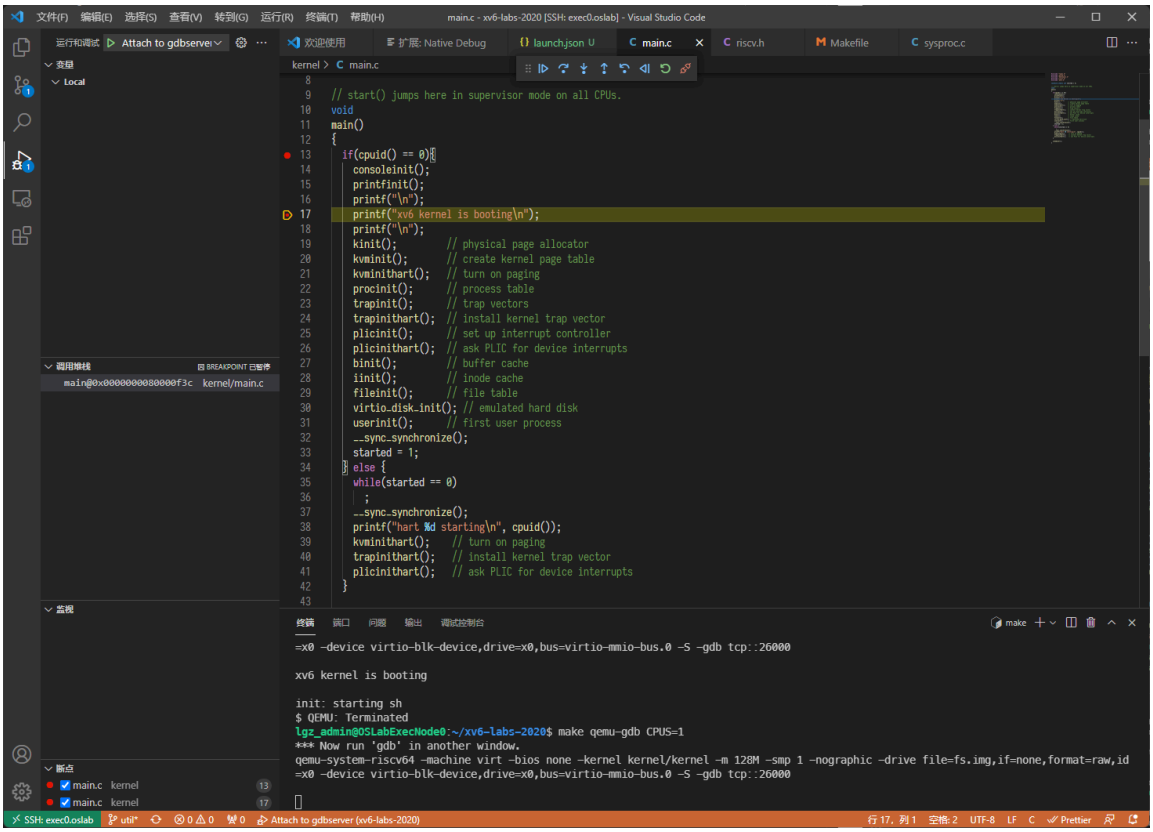


4.2 断点功能

调试运行中也可以继续加断点。点击行号左侧即可设置，程序会在运行到该位置时自动停下。以下示例中，第 17 行有一个尚未触发的断点：



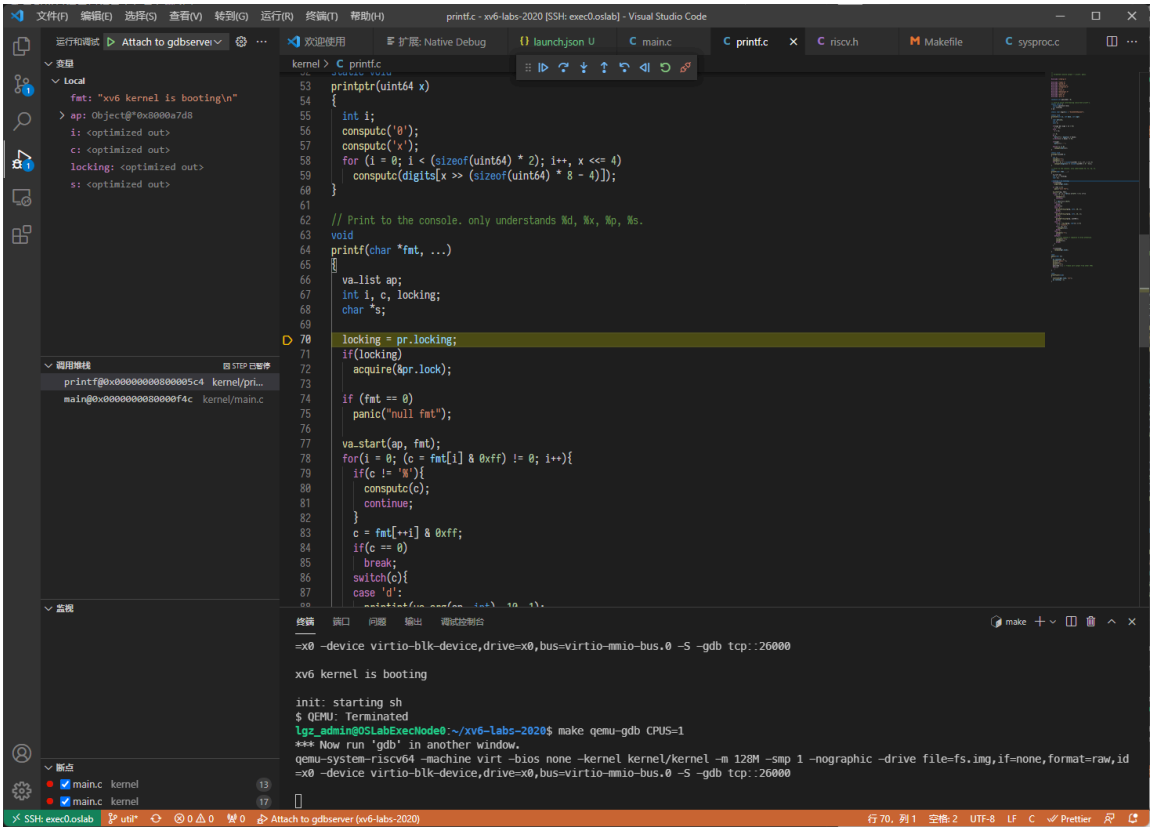
点击“运行”(左侧第一个按钮)，程序将自动运行到下一个断点并停止：



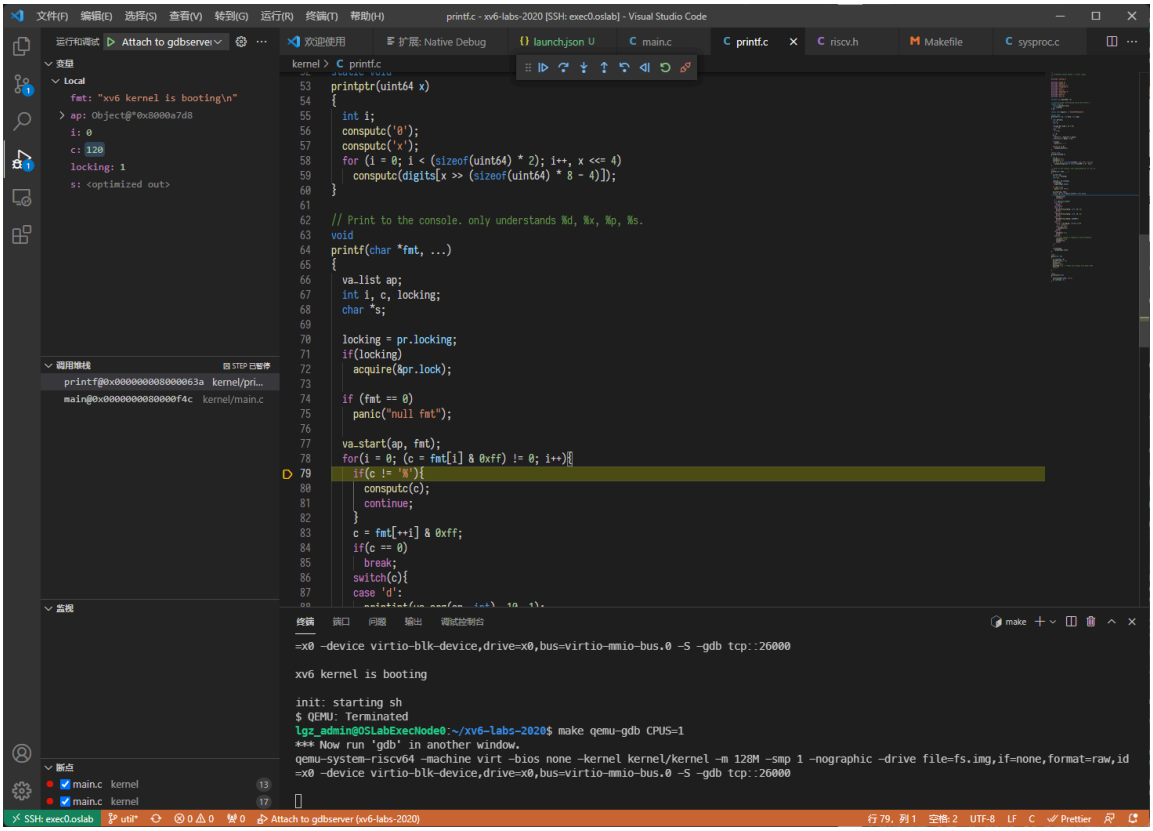
从输出可见其输出了一个回车，并停止在了该语句执行之前。

4.3 单步调试（步入函数）、单步跳出与局部变量

接下来从一个 `printf` 调用继续往下看。点击单步调试按钮（左侧第 3 个），调试器会进入 `printf` 函数的第一条可执行语句：

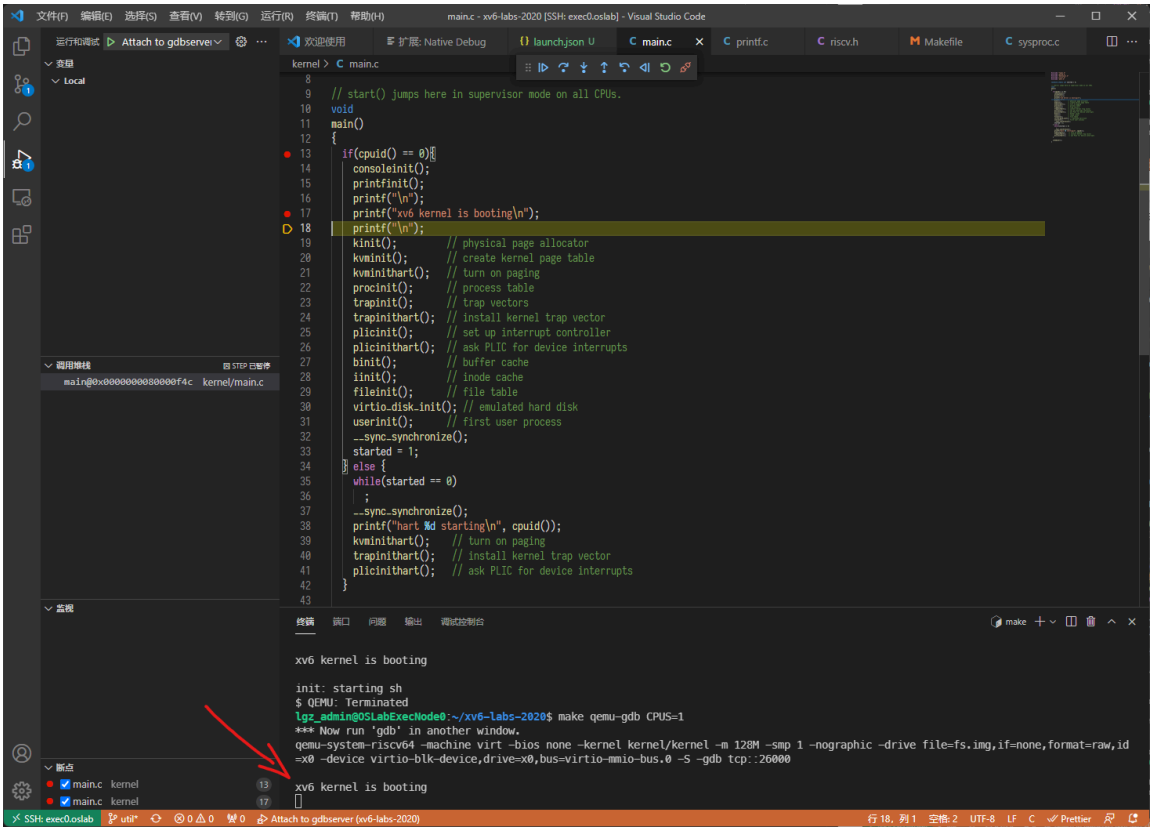


此时左上角会显示当前作用域的局部变量。`<optimized out>` 表示这个变量当前还无法直接显示，常见原因是它尚未初始化，或者当前位置还没有生成可见值。继续单步后，部分变量会变得可见：



复杂的变量（如结构体）可以展开以查看内容。

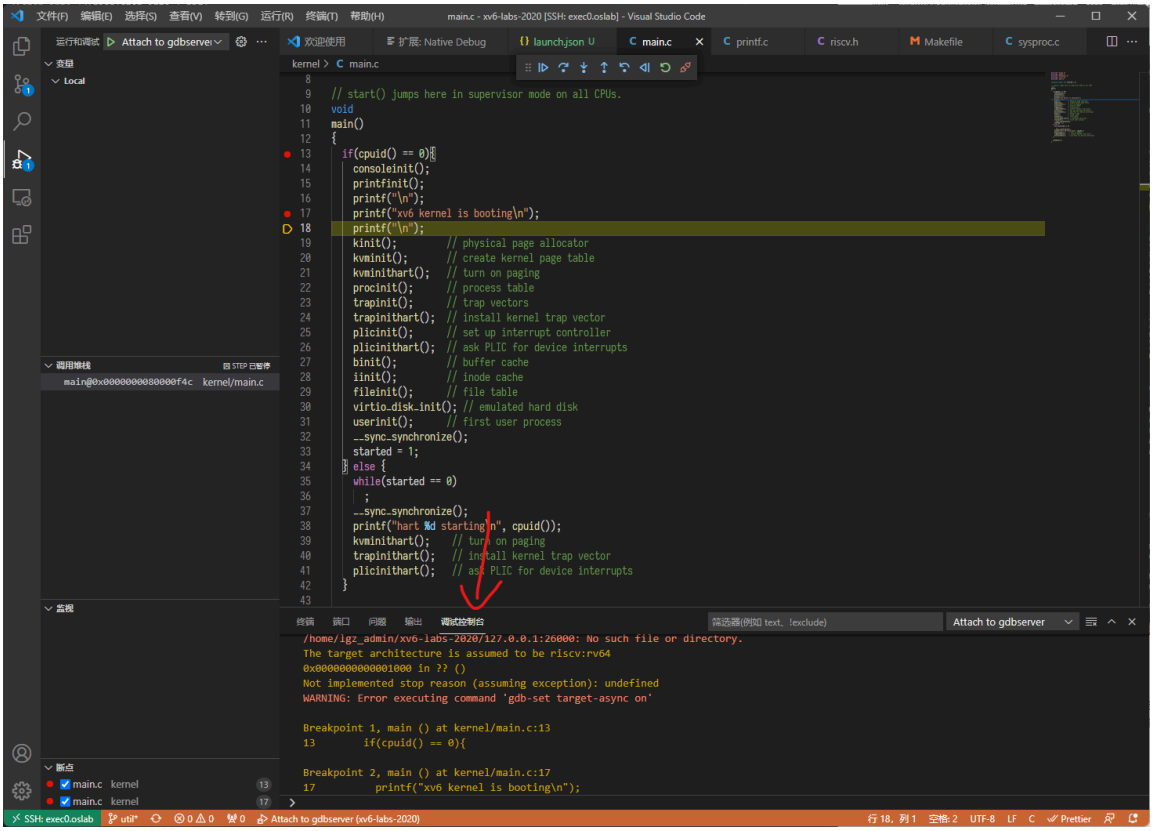
如果当前函数已经看清楚了，可以点击单步跳出（左侧第 4 个按钮）直接返回调用点。下面这张图表示函数已经执行完成，调试器重新停在函数外：



4.4 需要时再看的进阶技巧

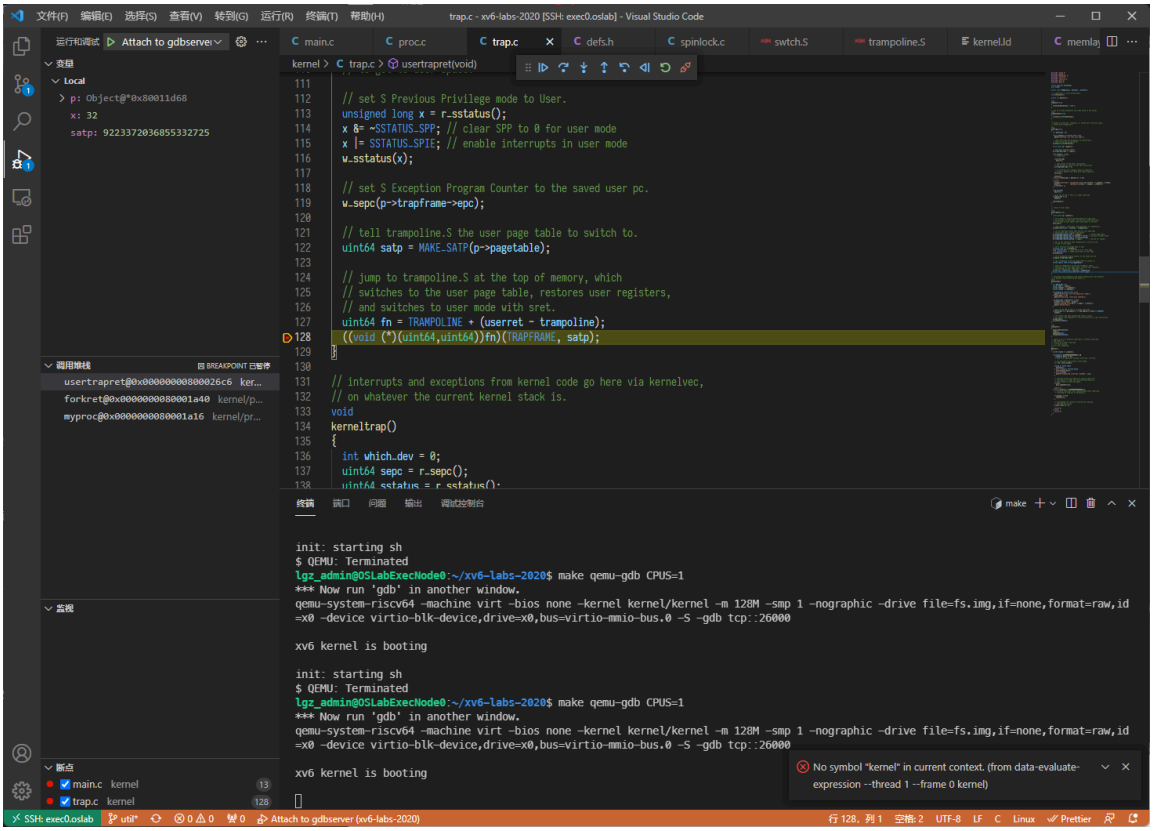
前面的内容已经足够覆盖大多数实验调试需求。如果你还想继续看汇编、寄存器、内存，或者切到用户程序继续调试，可以继续看这一节。

以下内容主要通过调试控制台完成：



4.4.1 单步汇编调试、汇编显示

我们以Trampoline处的调试作为示例。我们可以进入userret处：



单步跳过该指令，进入trampoline的userret。通过 `disas $pc, $pc+20` ，可以显示接下来20byte的汇编指令，可以发现我们确实进入了userret处：

```
disas $pc, $pc+20
Dump of assembler code from 0x3ffffff090 to 0x3ffffff0a4:
=> 0x0000003ffffff090:  csrw    satp,a1
   0x0000003ffffff094:  sfence.vma
   0x0000003ffffff098:  ld      t0,112(a0)
   0x0000003ffffff09c:  csrw    sscratch,t0
   0x0000003ffffff0a0:  ld      ra,40(a0)
End of assembler dump.
```

或者使用 `display/10i $pc` 可以让gdb持续自动显示从pc起的10条指令。

```
display/10i $pc
1: x/10i $pc
=> 0x3ffffff090:        csrw    satp,a1
   0x3ffffff094:        sfence.vma
   0x3ffffff098:        ld      t0,112(a0)
   0x3ffffff09c:        csrw    sscratch,t0
   0x3ffffff0a0:        ld      ra,40(a0)
   0x3ffffff0a4:        ld      sp,48(a0)
   0x3ffffff0a8:        ld      gp,56(a0)
   0x3ffffff0ac:        ld      tp,64(a0)
   0x3ffffff0b0:        ld      t0,72(a0)
   0x3ffffff0b4:        ld      t1,80(a0)

si

0x0000003ffffff094 in ?? ()
=> 0x0000003ffffff094:  73 00 00 12     sfence.vma
1: x/10i $pc
=> 0x3ffffff094:        sfence.vma
   0x3ffffff098:        ld      t0,112(a0)
   0x3ffffff09c:        csrw    sscratch,t0
   0x3ffffff0a0:        ld      ra,40(a0)
   0x3ffffff0a4:        ld      sp,48(a0)
   0x3ffffff0a8:        ld      gp,56(a0)
   0x3ffffff0ac:        ld      tp,64(a0)
   0x3ffffff0b0:        ld      t0,72(a0)
   0x3ffffff0b4:        ld      t1,80(a0)
   0x3ffffff0b8:        ld      t2,88(a0)
```

为什么不能直接 `b uservec`

这里直接 `b uservec` 通常不可行，因为 `uservec` 位于 trampoline 映射区域。更稳妥的方式，是先结合地址和反汇编定位到对应位置。

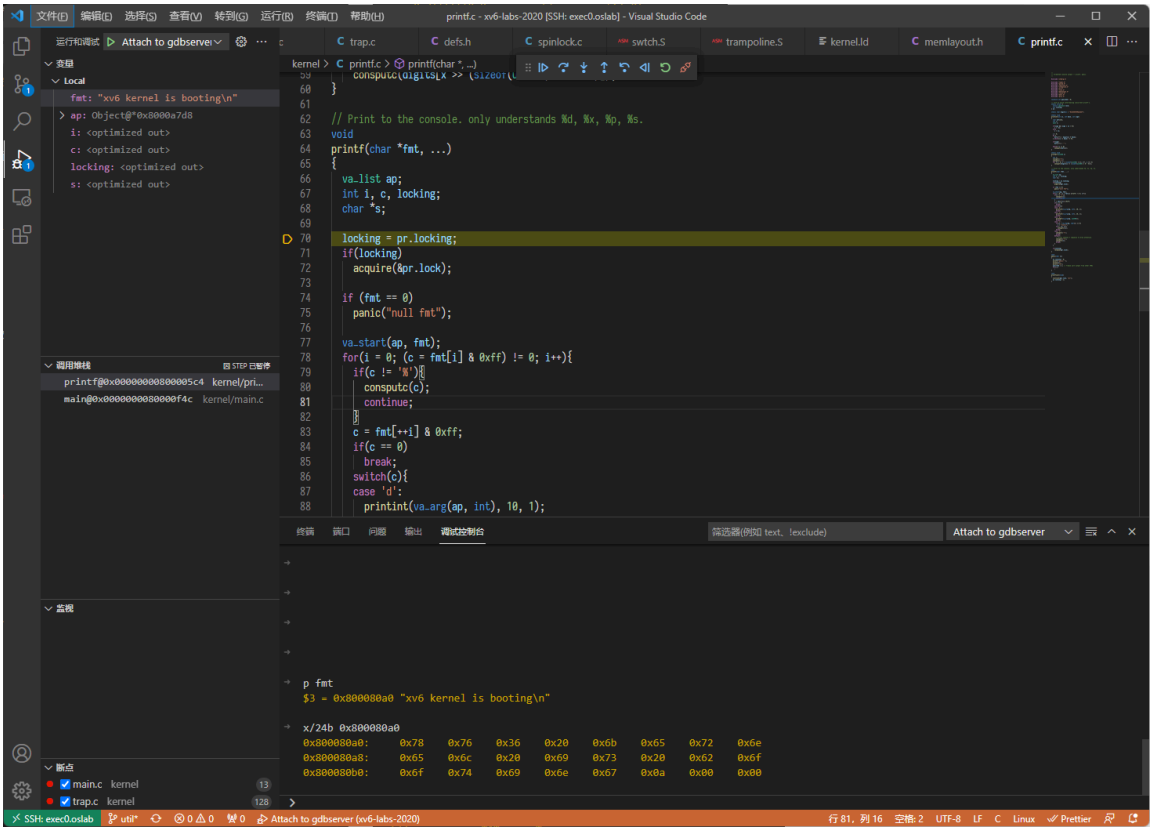
4.4.2 寄存器内容分析

使用 `info reg` 可以查看当前 CPU 的寄存器值。

```
info reg
ra             0x800026e6       0x800026e6 <usertrapret+150>
sp             0x3fffffdfe0     0x3fffffdfe0
gp             0x0      0x0
tp             0x0      0x0
t0             0x80000000       2147483648
t1             0x0      0
t2             0x0      0
fp             0x3fffffdff0     0x3fffffdff0
s1             0x0      0
a0             0x3fffffe000     274877898752
a1             0x8000000000087f75       -9223372036854218891
a2             0x80007000       2147512320
a3             0x20     32
a4             0x8000000000000000       -9223372036854775808
a5             0x3ffffff090     274877902992
a6             0x0      0
a7             0x0      0
s2             0x0      0
s3             0x0      0
s4             0x0      0
s5             0x0      0
s6             0x0      0
s7             0x0      0
s8             0x0      0
s9             0x0      0
s10            0x0      0
s11            0x0      0
t3             0x0      0
t4             0x0      0
t5             0x0      0
t6             0x0      0
pc             0x3ffffff094     0x3ffffff094
```

4.4.3 内存内容

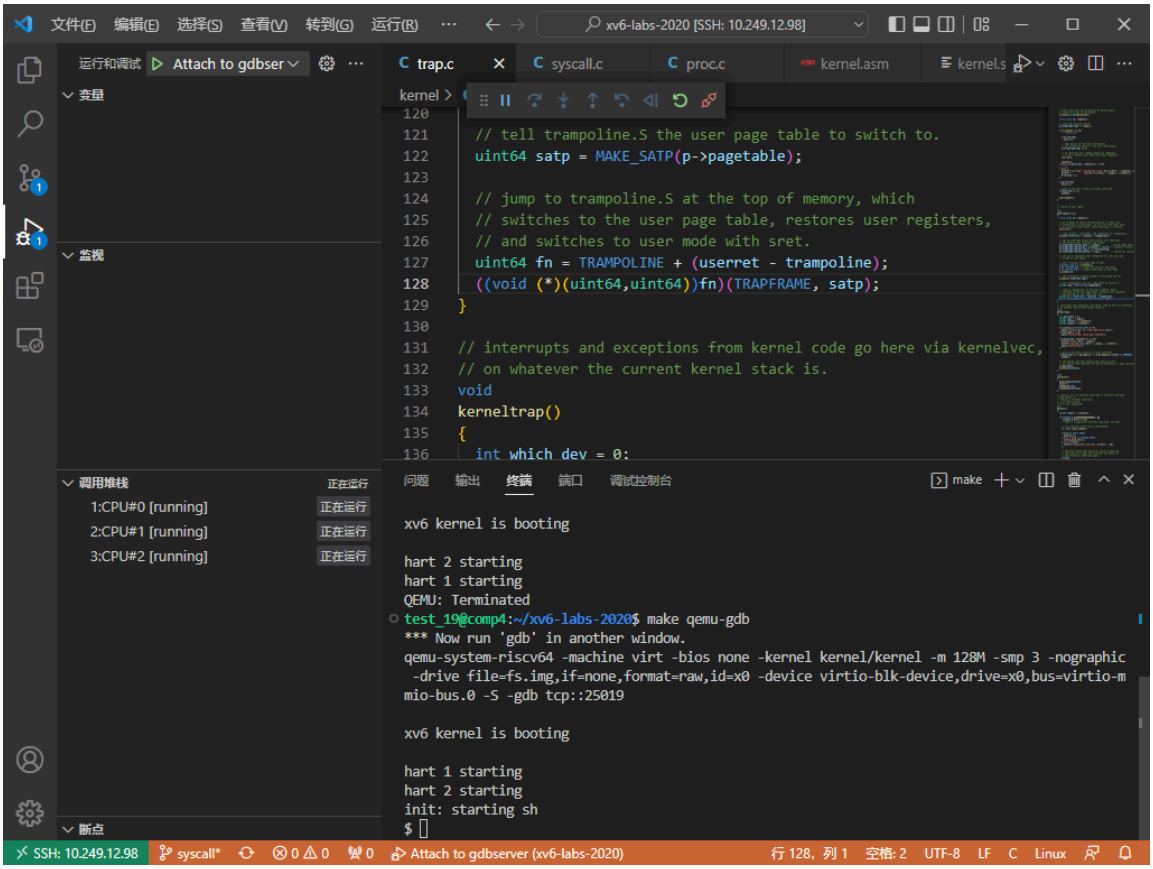
使用 `x/字节数b *地址` 可以查看指定地址附近的内存内容：



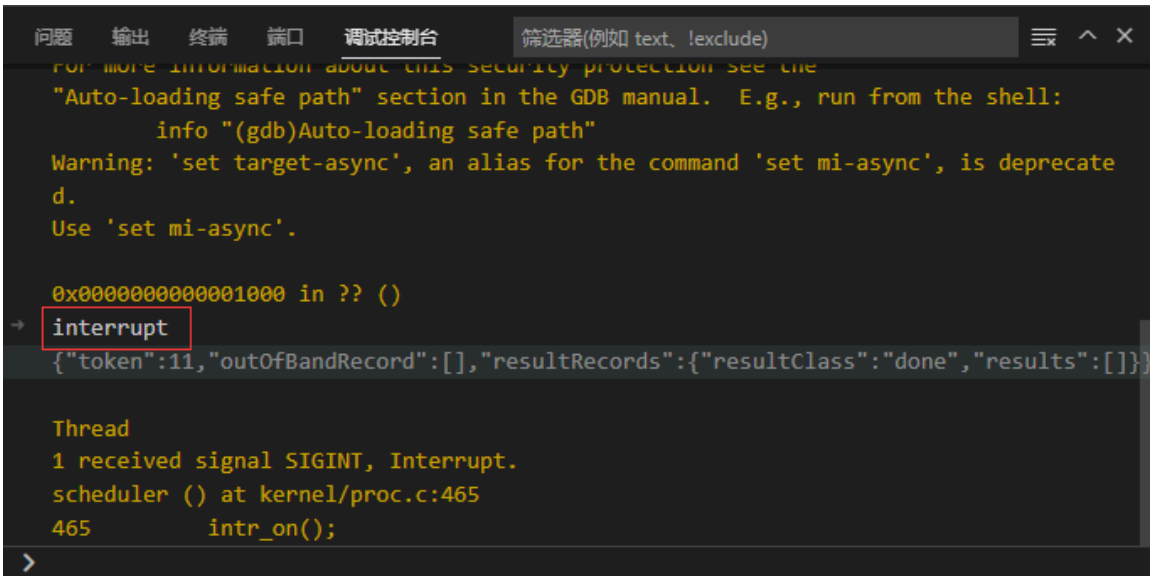
4.4.4 用户态程序调试

xv6 的内核态和用户态不共用页表，调试符号也不同。调试用户程序时，需要把当前符号从内核切换到对应用户程序。下面以调试自带的 `ls` 为例。

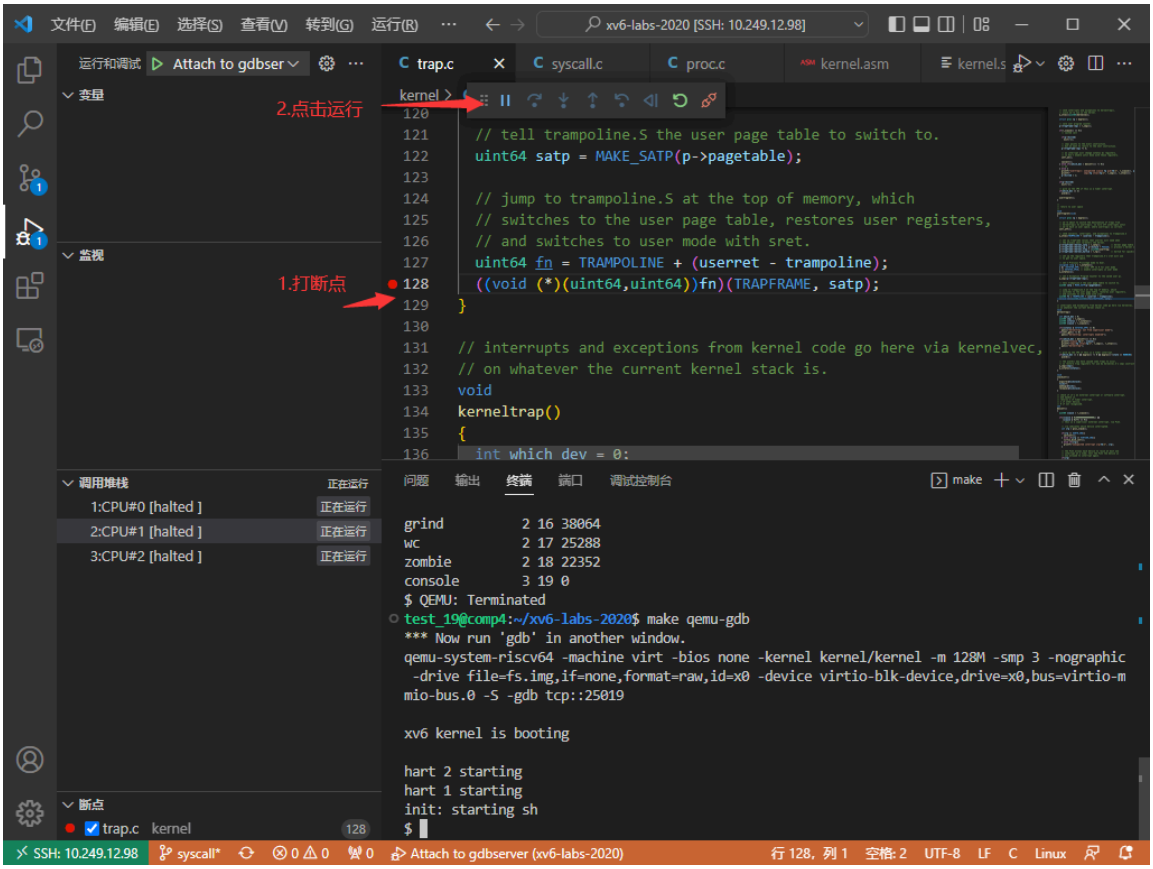
Step1： 先在终端执行 `make qemu-gdb` ，再在 VSCode 中按 `F5` 连接。点击“运行”，让 xv6 正常启动到 shell 提示符 `$` 。



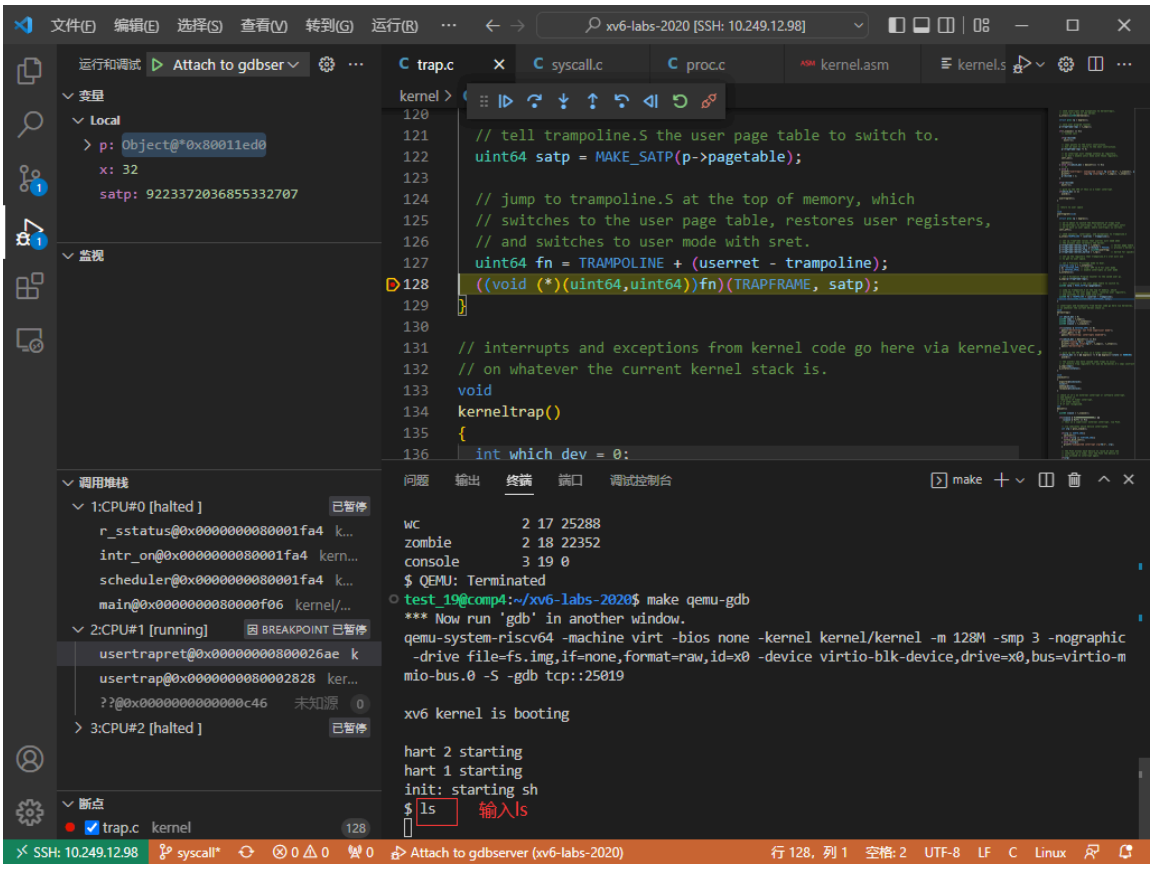
Step2： 在调试控制台输入 `interrupt` ，先把内核停住。



Step3： 在 `kernel/trap.c:128` 打断点，再继续运行。这里是进入 trampoline 切换前最后一行 C 代码。



Step4： 在 xv6 的 shell 中输入 `ls` 。程序会停在 `kernel/trap.c:128` 处。



Step5： 这时需要确定 `ls` 的入口点。常用做法有两种：

1. 用 `readelf` 查看 ELF 入口地址。
2. 直接在 VSCode 中打开 `user/ls.c` ，在 `main()` 打断点。

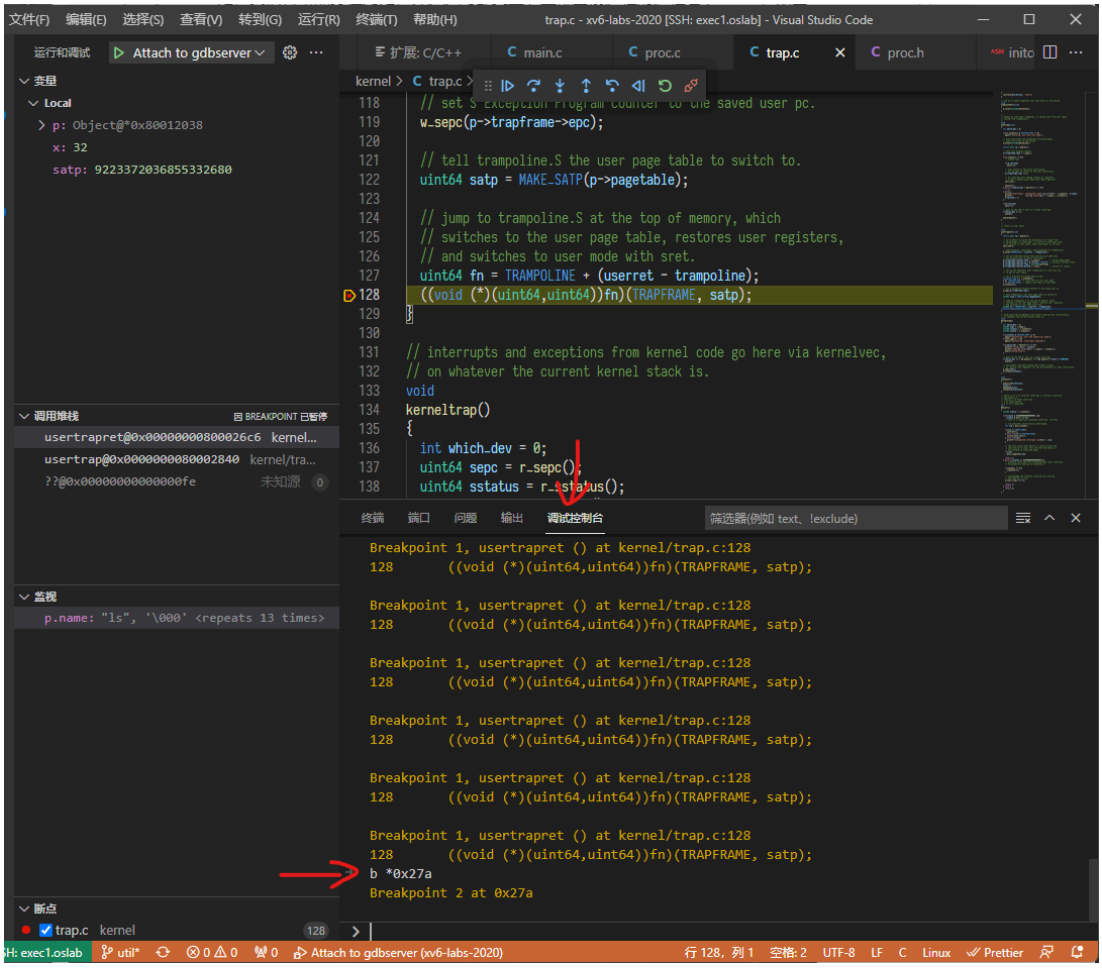
下面分别说明。

方法一：通过 `readelf` 确认入口点

`ls` 的 ELF 文件位于 `user/_ls`：

```
lgz_stu@OSLabExecNode1:~/xv6-oslabs-hitsz$ readelf -h user/_ls
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           RISC-V
  Version:                           0x1
  Entry point address:               0x27a
  Start of program headers:          64 (bytes into file)
  Start of section headers:          25064 (bytes into file)
  Flags:                             0x5, RVC, double-float ABI
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         1
  Size of section headers:           64 (bytes)
  Number of section headers:         18
  Section header string table index: 17
```

看到 `Entry point address: 0x27a` 之后，就可以在调试控制台输入 `b *0x27a`，把断点打在 `ls` 入口处：



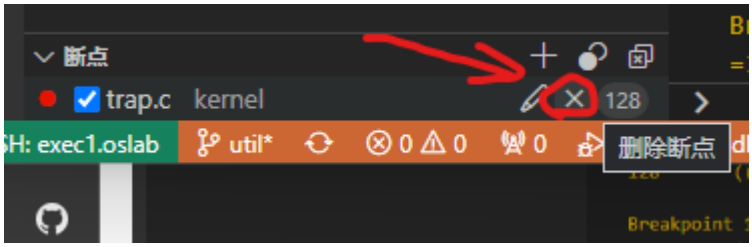
方法二：直接在 `main()` 打断点

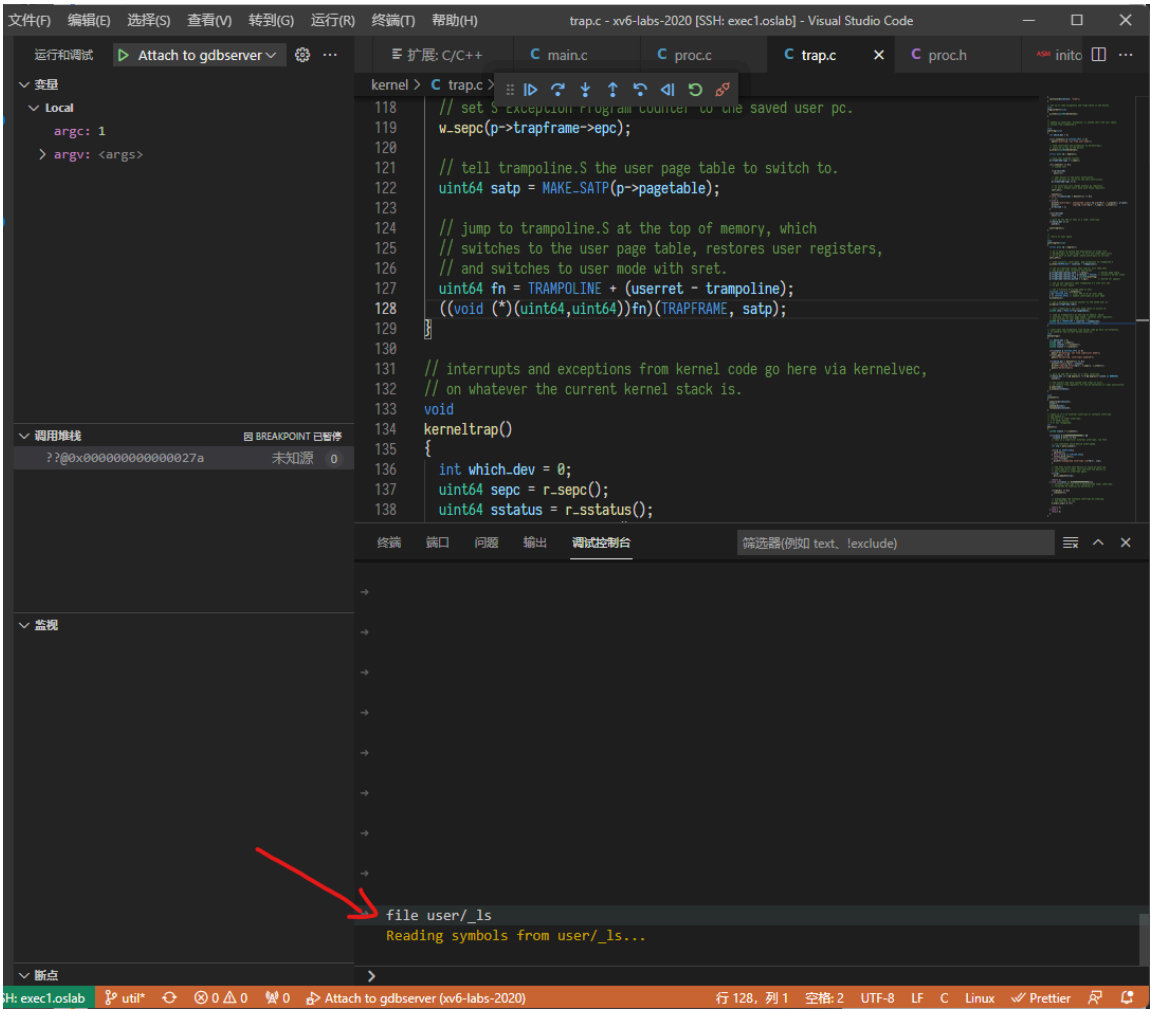
在 VSCode 中打开 `user/ls.c` ，找到 `main()` ，在第 78 行打断点。

```
73  int
74  main(int argc, char *argv[])
75  {
76    int i;
77
78    if(argc < 2){
79      ls(".");
80      exit(0);
81    }
82    for(i=1; i<argc; i++)
83      ls(argv[i]);
84    exit(0);
85  }
86
```

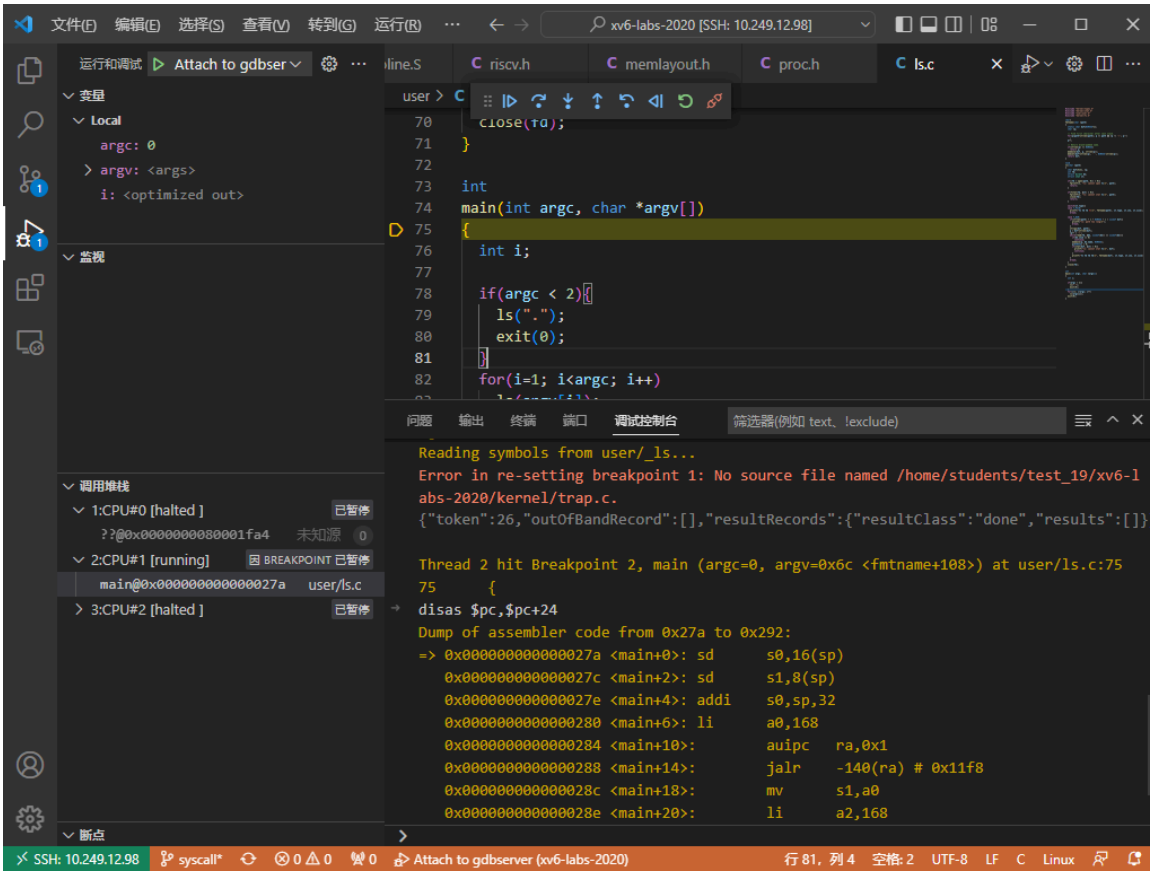
通过这两种方法中的任意一种，都可以把断点放到 `ls` 的入口附近。

Step6： 删除原有内核态断点，并在调试控制台输入 `file user/_ls` ，加载 `ls` 的调试符号：

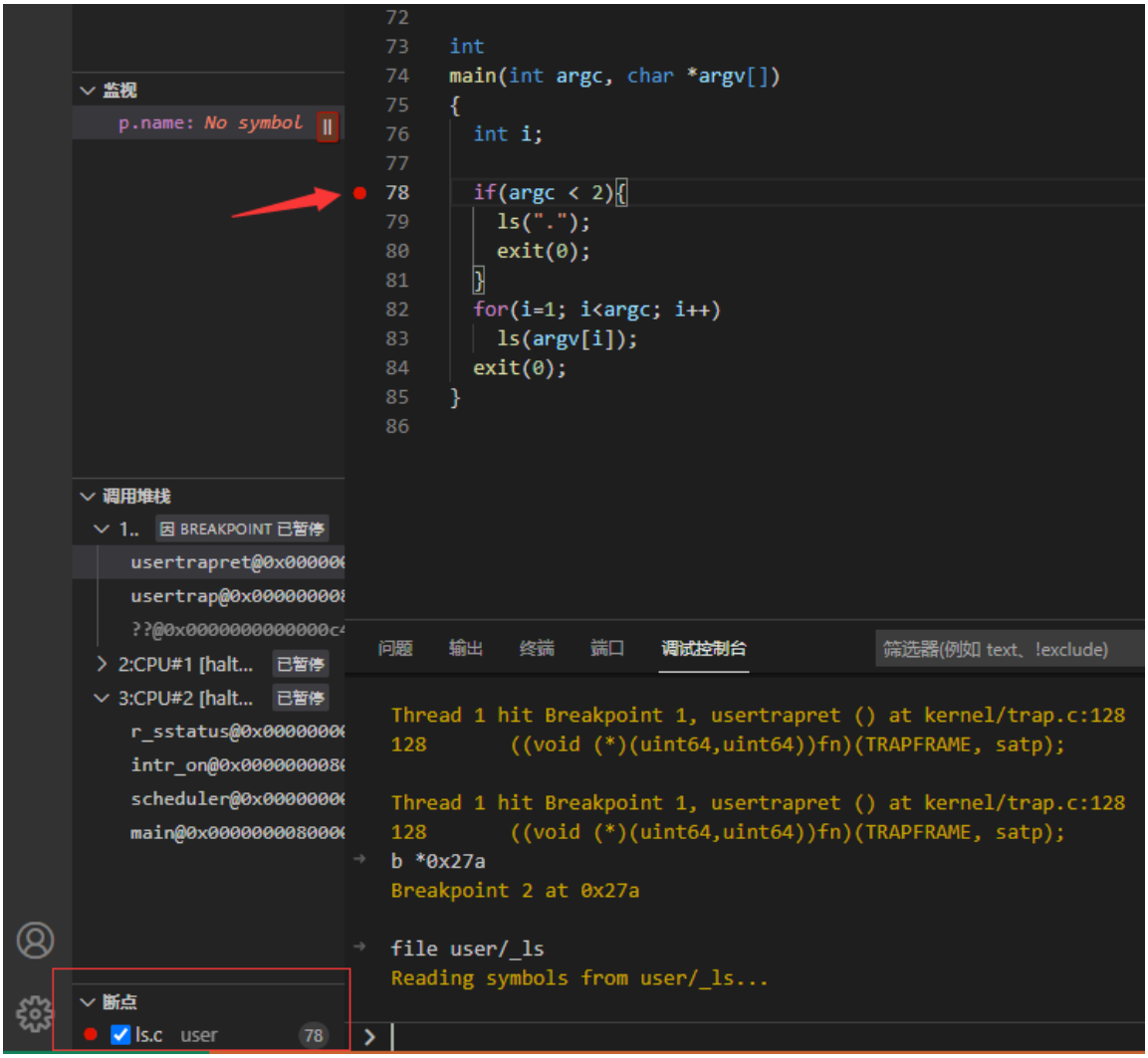




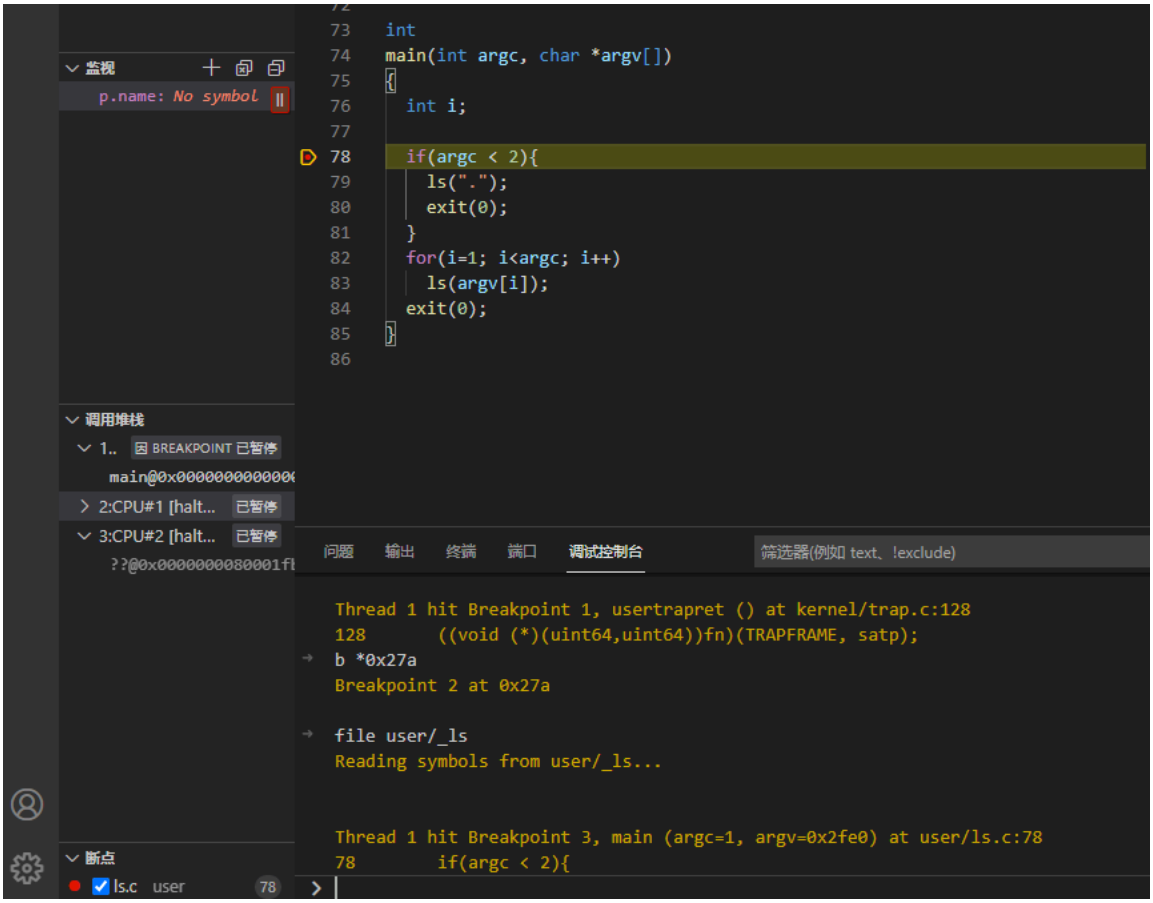
Step7： 点击“运行”。如果符号切换成功，程序会停在 `ls.c` 的 `main()` 附近。



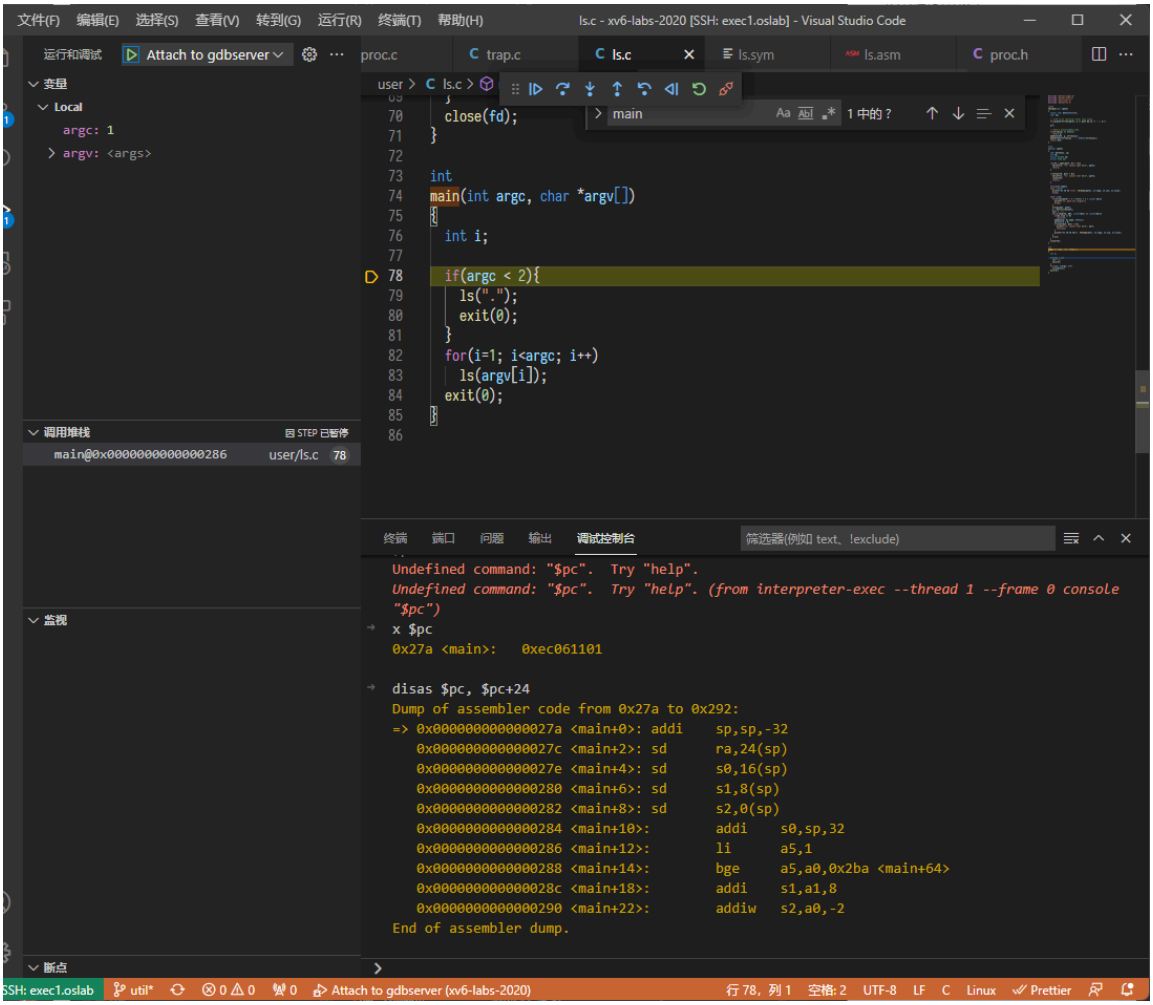
Step8： 如果还没有在 `user/ls.c` 中打断点，可以现在补上。下图示例是在第 78 行打断点。



Step9： 继续运行。程序会停在 `ls` 对应断点处。之后无论是单步、查看变量还是继续运行，操作方式都和内核调试相同。



点击上方的单步调试按钮，可以看到 VSCode 的图形化调试在用户程序里也能正常工作：



此后的调试流程与内核调试基本一致。

4.4.5 其他

其余功能可以按需查阅 GDB 手册。需要注意的是，xv6 运行在 QEMU 上，且使用 RISC-V 架构，个别命令可能出现不支持或行为与预期不完全一致的情况，这属于正常现象。

VSCode 图形化界面调试指南（二）——系统调用

这页能帮你做到什么

看完这页后，你应该能完成三件事： **在 VSCode 中跟踪一次系统调用的执行过程** 、 **观察 `ecall` 前后的寄存器 / 汇编 / 页表信息** 、 **把用户态到内核态的切换过程看清楚** 。

先记住一条主线

这一篇重点不是重新学习断点，而是沿着这条主线看清一次系统调用： **在用户态停住** → **找到 `ecall`** → **看寄存器 / 页表 / 汇编变化** → **进入 `usertrap`** 。

开始前说明

如果你还不熟悉断点、单步、变量查看，建议先看 VSCode 图形化界面调试指南（一） 。

如果你已经能完成基础调试，并想继续看清一次系统调用是怎么从用户态进入内核态、再返回用户态的，这篇会更适合你。

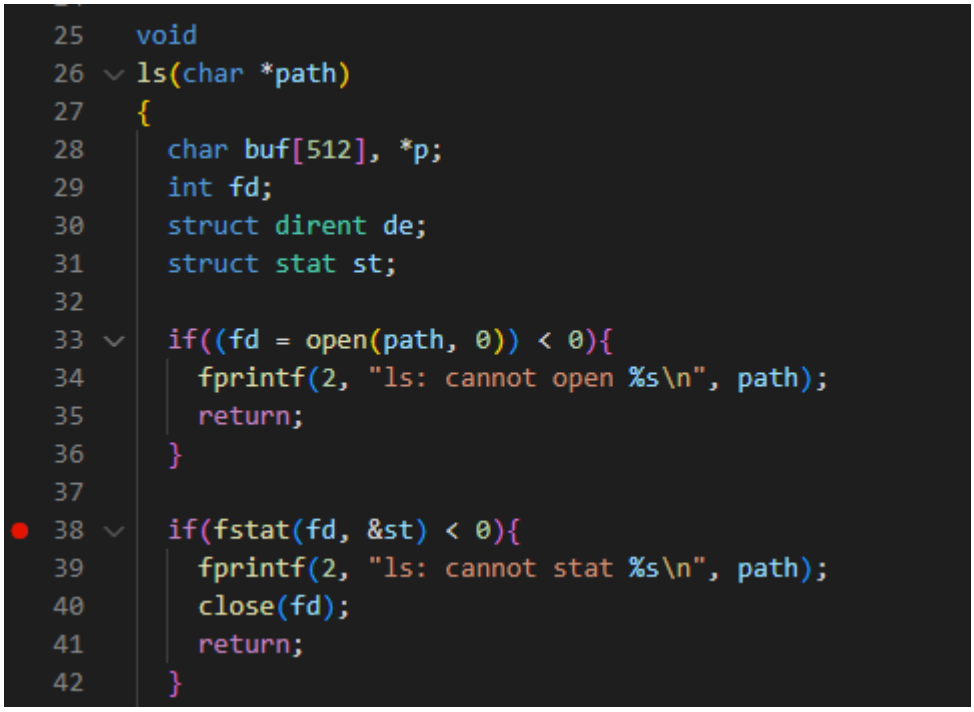
为了给同学们演示如何用 VSCode 图形化调试 xv6 的系统调用过程，我们在 VSCode 图形化界面调试指南（一） 的基础上，又录制了两个进阶演示视频：

1. 【3. VSCode 调试系统调用过程（包含 pagetable 和汇编）】
   https://www.bilibili.com/video/BV12P411J7xq?share_source=copy_web&vd_source=a822dcda3537564ccdd0bb45aa0afe33
2. 【4. VSCode 调试系统调用——从内核到用户，再从用户返回内核】
   https://www.bilibili.com/video/BV1ug411m7ir?share_source=copy_web&vd_source=a822dcda3537564ccdd0bb45aa0afe33

1. ECALL 指令之前的状态

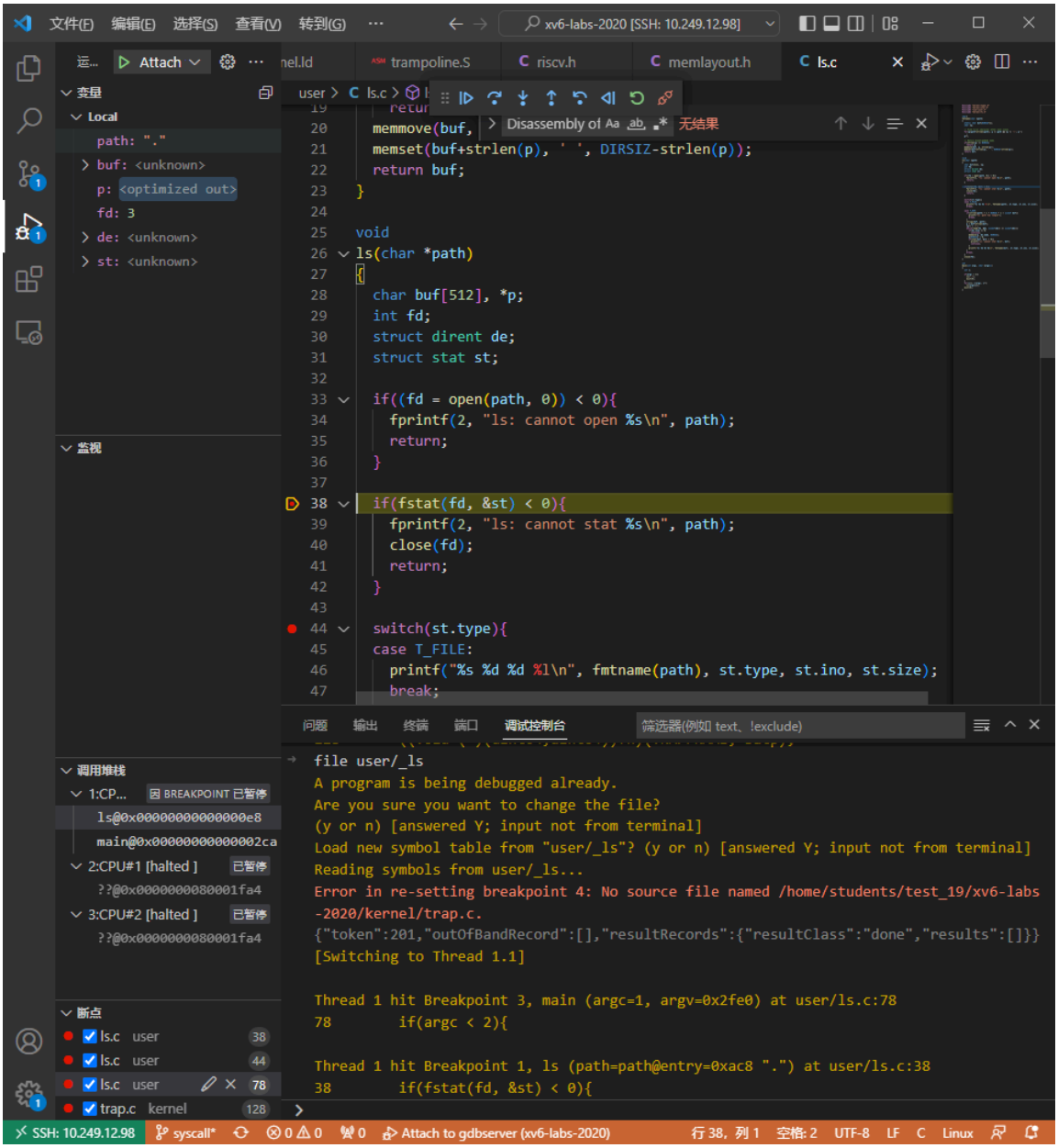
首先，参考用户态程序调试从内核进入用户空间的user/ls.c。

在第38行打上断点。

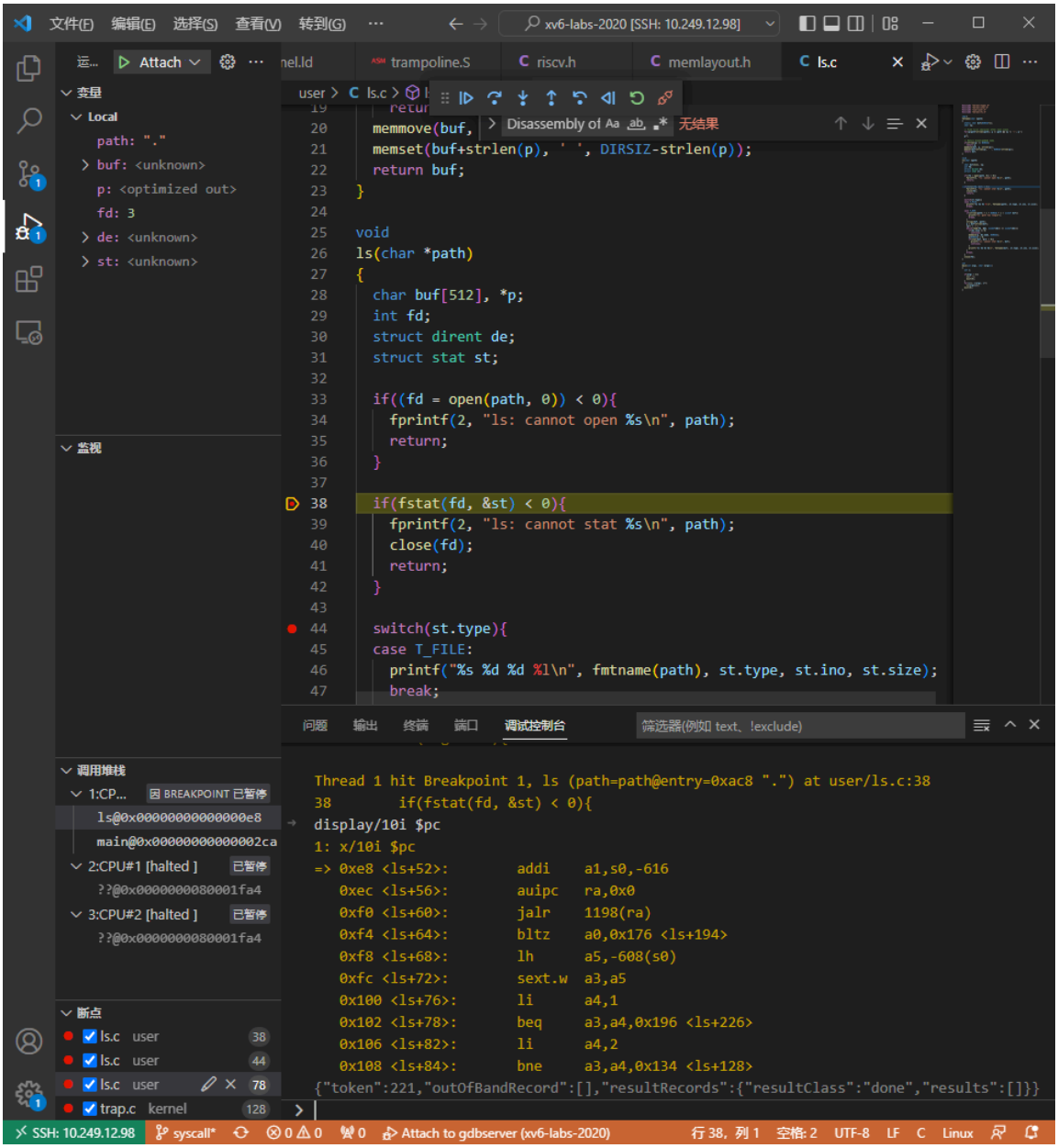


```
25  void
26  ls(char *path)
27  {
28    char buf[512], *p;
29    int fd;
30    struct dirent de;
31    struct stat st;
32
33    if((fd = open(path, 0)) < 0){
34      fprintf(2, "ls: cannot open %s\n", path);
35      return;
36    }
37
38    if(fstat(fd, &st) < 0){
39      fprintf(2, "ls: cannot stat %s\n", path);
40      close(fd);
41      return;
42    }
```

点击“运行”，程序将自动运行到第38行并停止：



在调试控制台输入`display/10i $pc`，让gdb持续自动显示从pc起的10条指令。

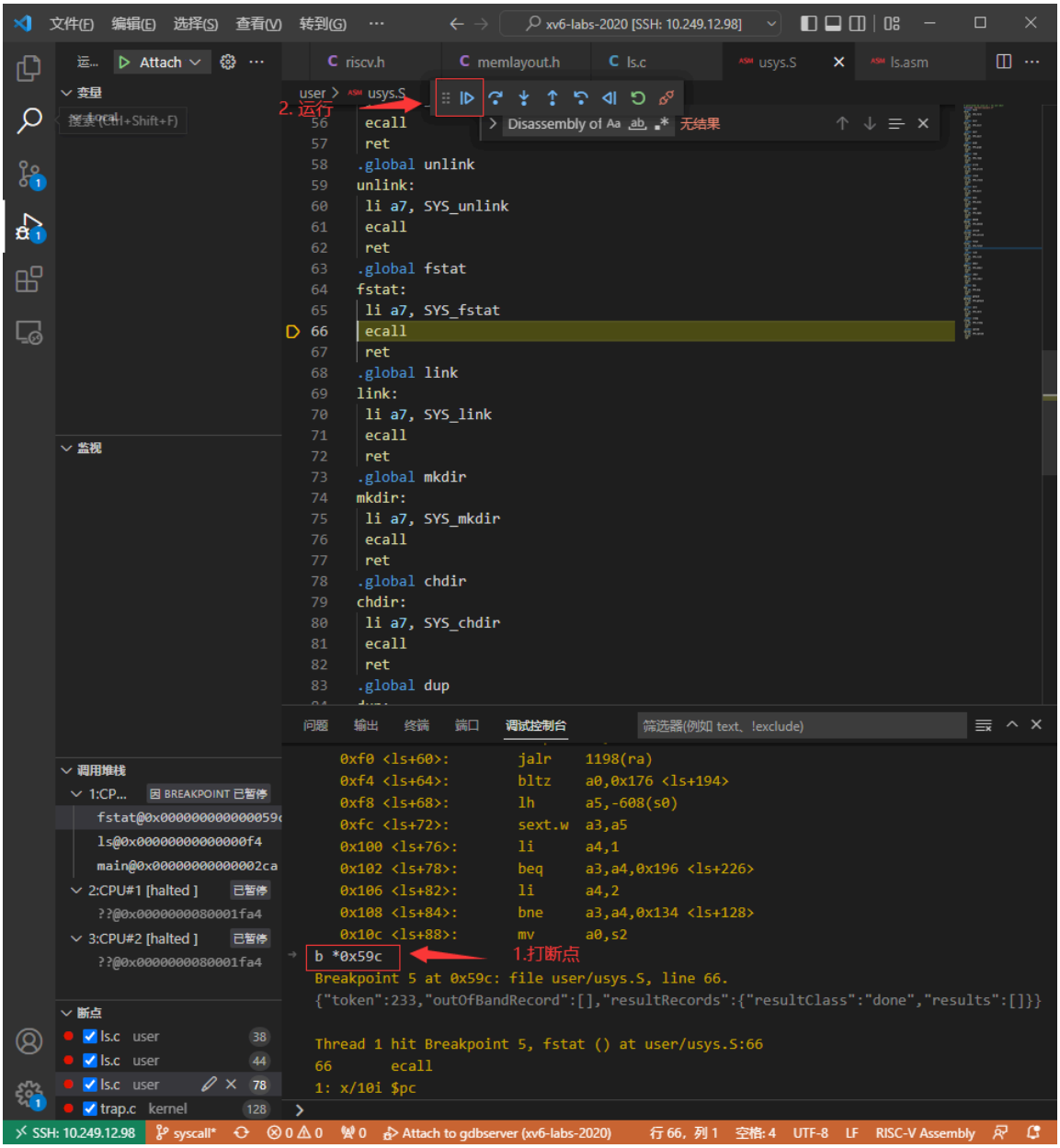


在user/ls.asm反汇编文件中，可以看到ecall指令汇编地址是0x59c。

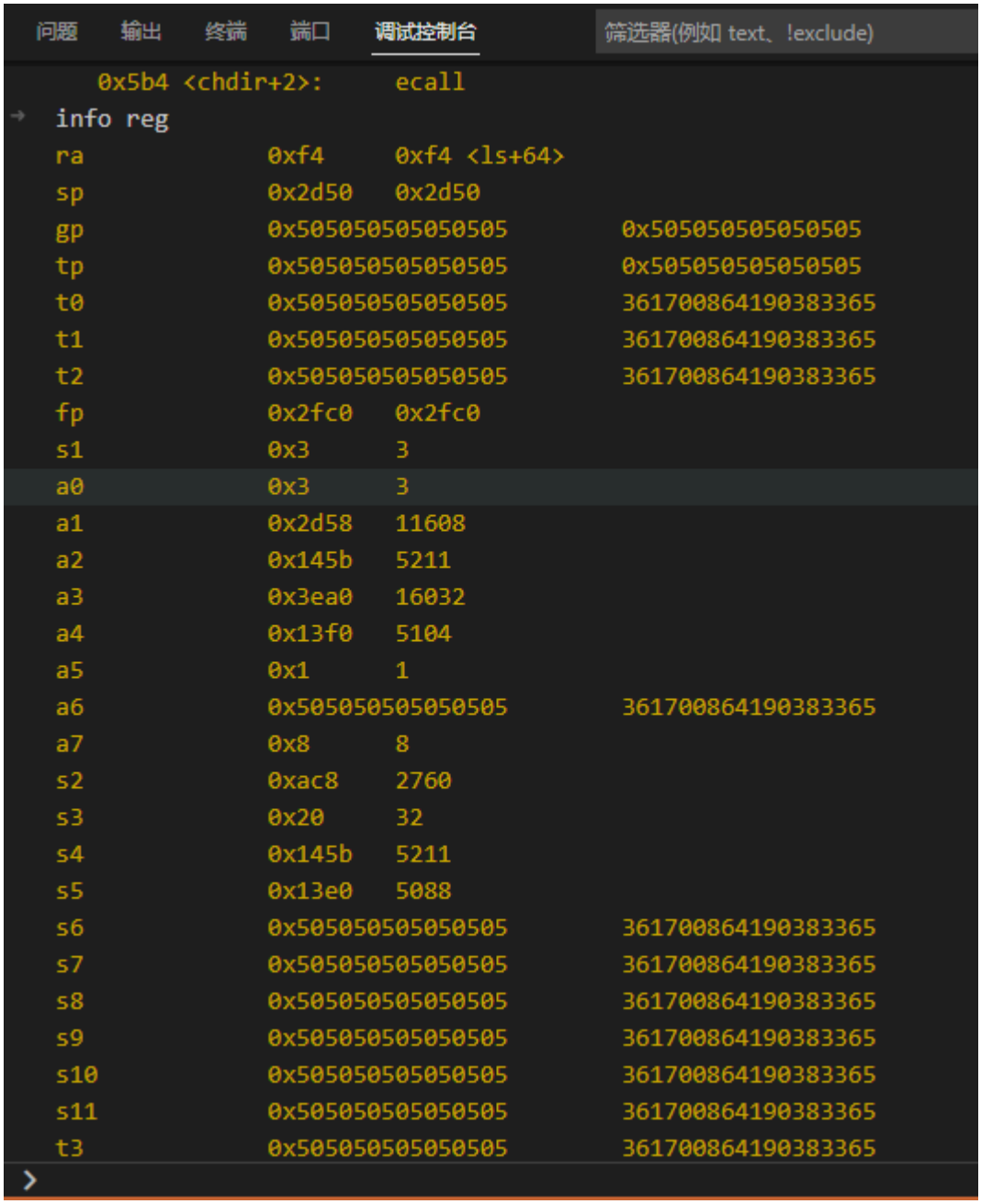
```
868  000000000000059a <fstat>:
869  .global fstat
870  fstat:
871   li a7, SYS_fstat
872   59a: 48a1                 li a7,8
873   ecall
874   59c: 00000073             ecall
875   ret
876   5a0: 8082                 ret
```

接着，输入`b *0x59c`在ecall指令上打断点。再点击“运行”，程序自动停在了ecall指令那一行上。

或者，也可以连续多次输入`si`（stepi命令），单步执行每一条汇编指令，直到程序停止ecall指令处。

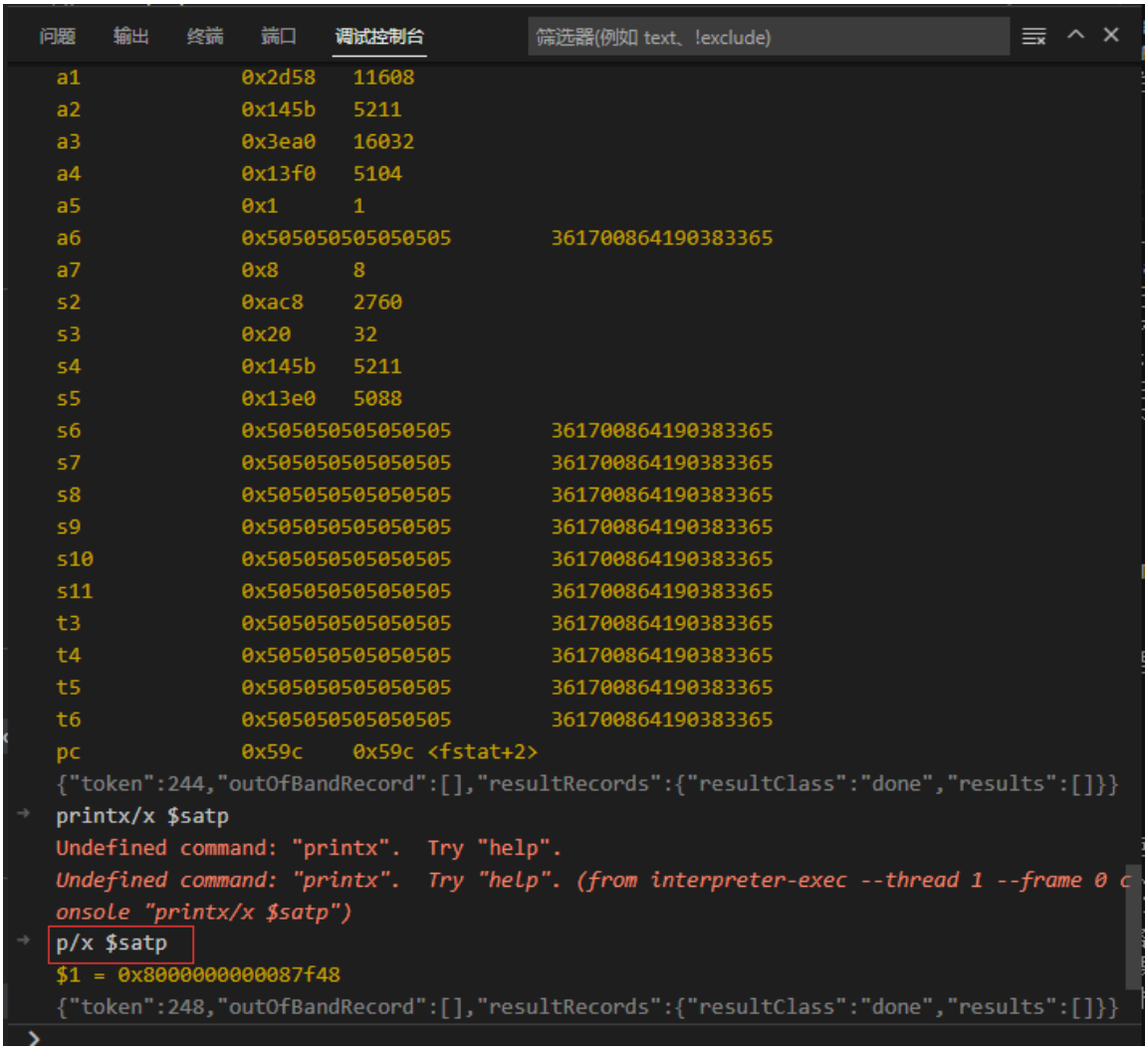


我们还可以输入`info reg`打印全部32个寄存器的值



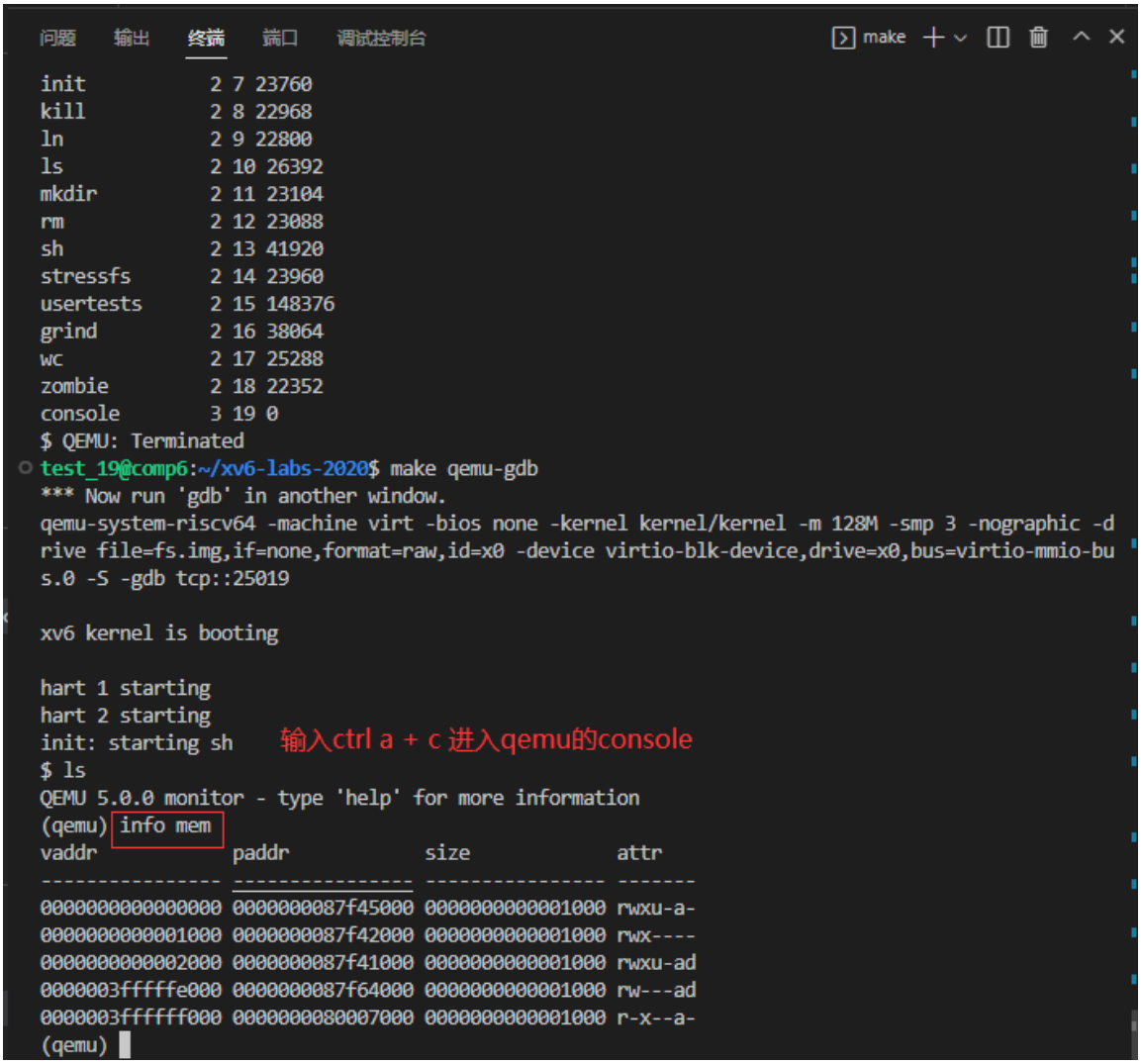
这里最值得先看的，是 `a0` 和 `a1` 这两个寄存器。它们保存了 `ls` 调用 `fstat` 时传入的参数：其中 `a0` 是文件描述符 `fd` ， `a1` 是接收 `stat` 结构体信息的指针。

这时也可以输入 `print/x $satp` 或 `p/x $satp` ，查看 `satp` 寄存器的值，也就是当前页表所在的物理地址。



satp寄存器给出的物理内存地址是0x8000000000087f48。

如果还想进一步看 page table 的映射关系，需要切到 QEMU 界面（注意，是在 **终端** ，不是调试控制台），输入 `Ctrl-A C` 进入 QEMU console，然后再输入 `info mem` 。



上面打印出来的是用户程序`ls`的page table，非常小，只包含了5条映射关系。

page table拓展知识

在上图中，`attr`是PTE（页表，Page Table Entry）的标志位。

- `r`：Readable，可读
- `w`：Writable，可写
- `x`：Executable，可执行
- `u`：User，表明PTE_u标志位是否被设置，用户代码只能访问u标志位设置了的PTE。也就是如果有这个标志位，则说明这个page可以被运行在用户空间的进程访问。
- `a`：Accessed，表明这条PTE是否被使用过。
- `g`：Global
- `d`：Dirty，表明这条PTE是否被写过。

最后两条PTE的虚拟地址非常大（接近虚拟地址的顶端），它们分别是trapframe page和trampoline page（详见xv6 book P24-26）。这两个PTE没有u标志位，用户代码不能访问它们。当进入supervisor mode时，才能够访问。可能有同学注意到，第二条PTE也没有u标志位，这是一个无效的page（guard page），以防止用户程序尝试使用过多的stack page。

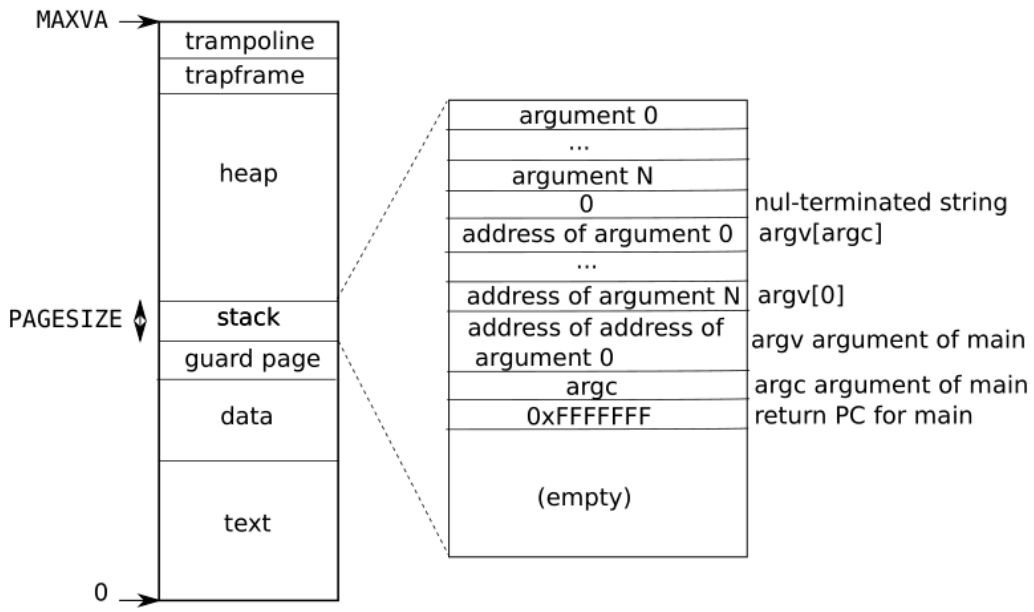


Figure 3.4: A process's user address space, with its initial stack.

我们在kernel/memlayout.h头文件中，也可以看到trapframe的定义。

```
// map the trampoline page to the highest address,
// in both user and kernel space.
#define TRAMPOLINE (MAXVA - PGSIZE)
```

PGSIZE和MAXVA的定义在kernel/riscv.h。

```
#define PGSIZE 4096 // bytes per page

// one beyond the highest possible virtual address.
// MAXVA is actually one bit less than the max allowed by
// Sv39, to avoid having to sign-extend virtual addresses
// that have the high bit set.
#define MAXVA (1L << (9 + 9 + 9 + 12 - 1))
```

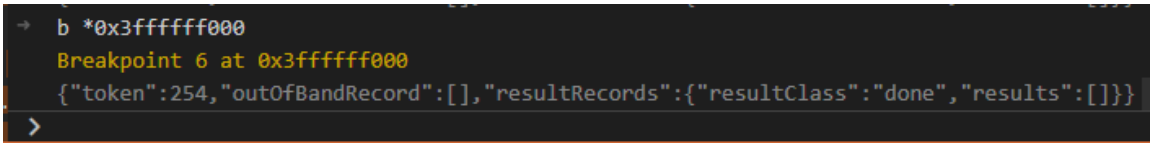
这样，就能计算出TRAMPOLINE的地址是`1<<38 - 4096 = 0x3ffffff000`，这个地址和上面用`info mem`命令查到的最后一条PTE是一样的。也就是，最后一条PTE正是TRAMPOLINE。

2. ECALL指令之后的状态

为了在程序执行到 trampoline.S 中 uservec 段的第一条指令时暂停，您可以在地址 0x3ffffff000 处设置断点。

在 GDB 调试控制台中输入以下命令：

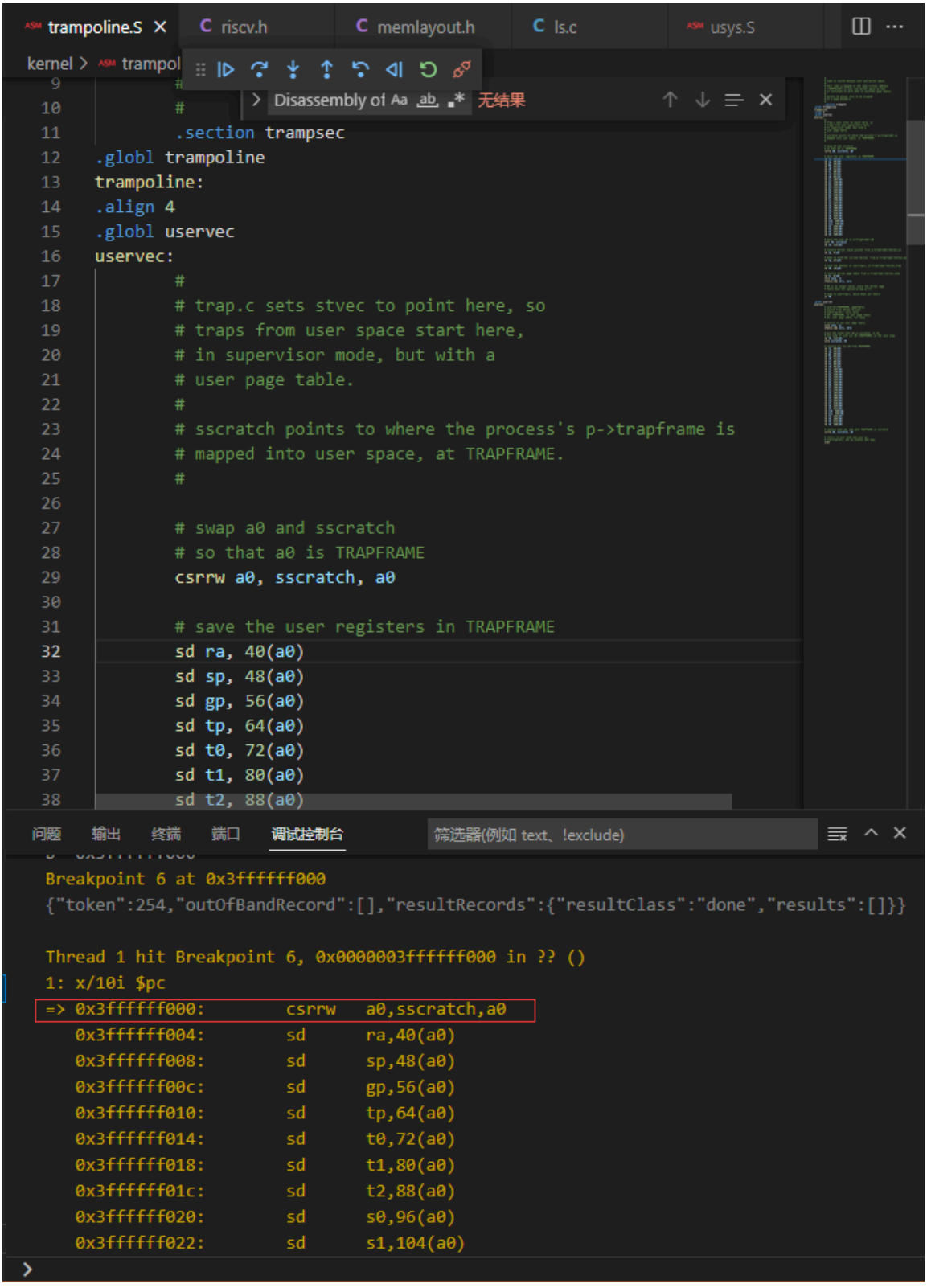
```
b *0x3ffffff000
```



请注意：如果你使用的是`riscv64-unknown-elf-gdb`（而非`gdb-multiarch`或`riscv64-unknown-linux-gnu-gdb`），则可以直接通过`stepi`命令跟踪`ecall`指令进入内核，无需在`0x3ffffff000`地址处设置断点。 这是因为：

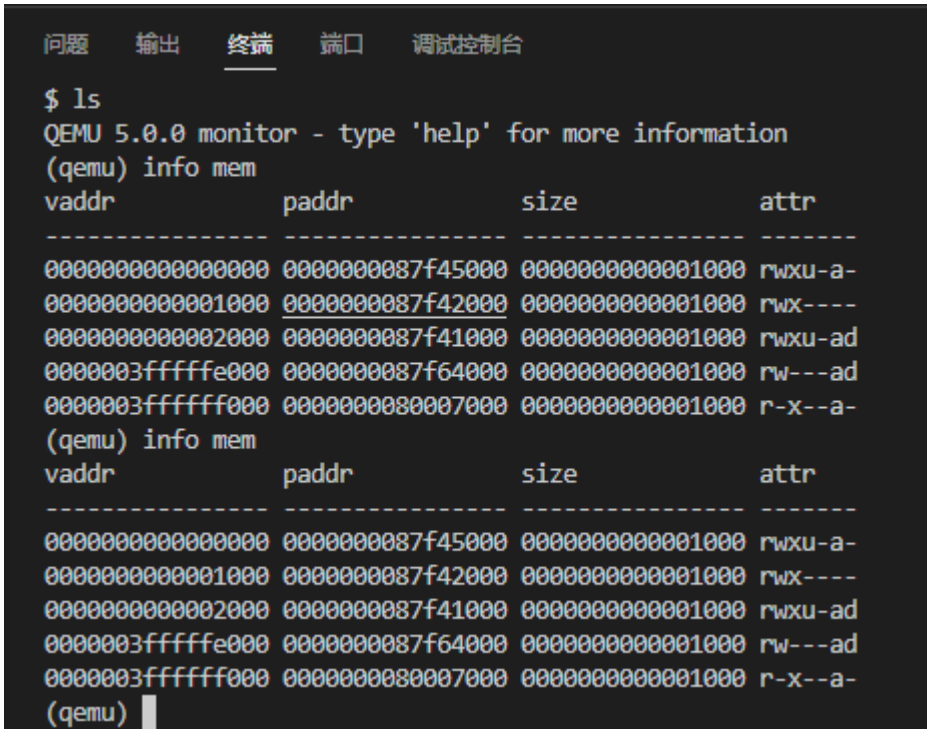
- `riscv64-unknown-elf-gdb`用于调试裸机程序（如操作系统），能够正确处理特权级切换和汇编指令级别的调试。
- `riscv64-unknown-linux-gnu-gdb`则用于调试运行在`Linux`操作系统之上的用户态程序。

点击“运行”，或者输入`si`单步执行ecall指令。



从这里可以看到，已经进入了trampoline。

这时切到 QEMU console，再输入 `info mem`：



可以看出，page table并没有发生改变，也就是说ecall并没有切换page table。

继续观察 `uservec` 的汇编代码。中间有一批 `sd` 指令，主要是在保存用户寄存器；这里先不展开，先抓住更关键的一步：在跳入 `usertrap()` 之前，代码会先切换到 kernel page table。

```
76  # restore kernel page table from p->trapframe->kernel_satp
77  ld t1, 0(a0)
78  csrw satp, t1
79  sfence.vma zero, zero
80
81  # a0 is no longer valid, since the kernel page
82  # table does not specially map p->tf.
83
84  # jump to usertrap(), which does not return
85  jr t0
```

接下来，我们将断点打在 `0x3ffffff08e` 地址上，也就是让程序停留在跳入usertrap()之前。

如何知道第85行 `jr t0` 语句的地址是 `0x3ffffff08e`？

在调试控制台，输入 `x/40i $pc` 显示当前PC后面的40条指令，就可以看到 `jr t0` 对应的地址。

也可以通过查看kernel.asm反汇编文件，在 `0000000080007000 <_trampoline>:` 段，可以看到 `jr t0` 相对uservec第一条指令位置偏移了 `0x8e`，将 `0x3ffffff000` + `0x8e`，就是 `0x3ffffff08e`。

输入 `b *0x3ffffff08e`

```
→ b *0x3ffffff08e
  Breakpoint 7 at 0x3ffffff08e
  {"token":267,"outOfBandRecord":[],"resultRecords":{"resultClass":"done","results":[]}}
>
```

点击”运行“。

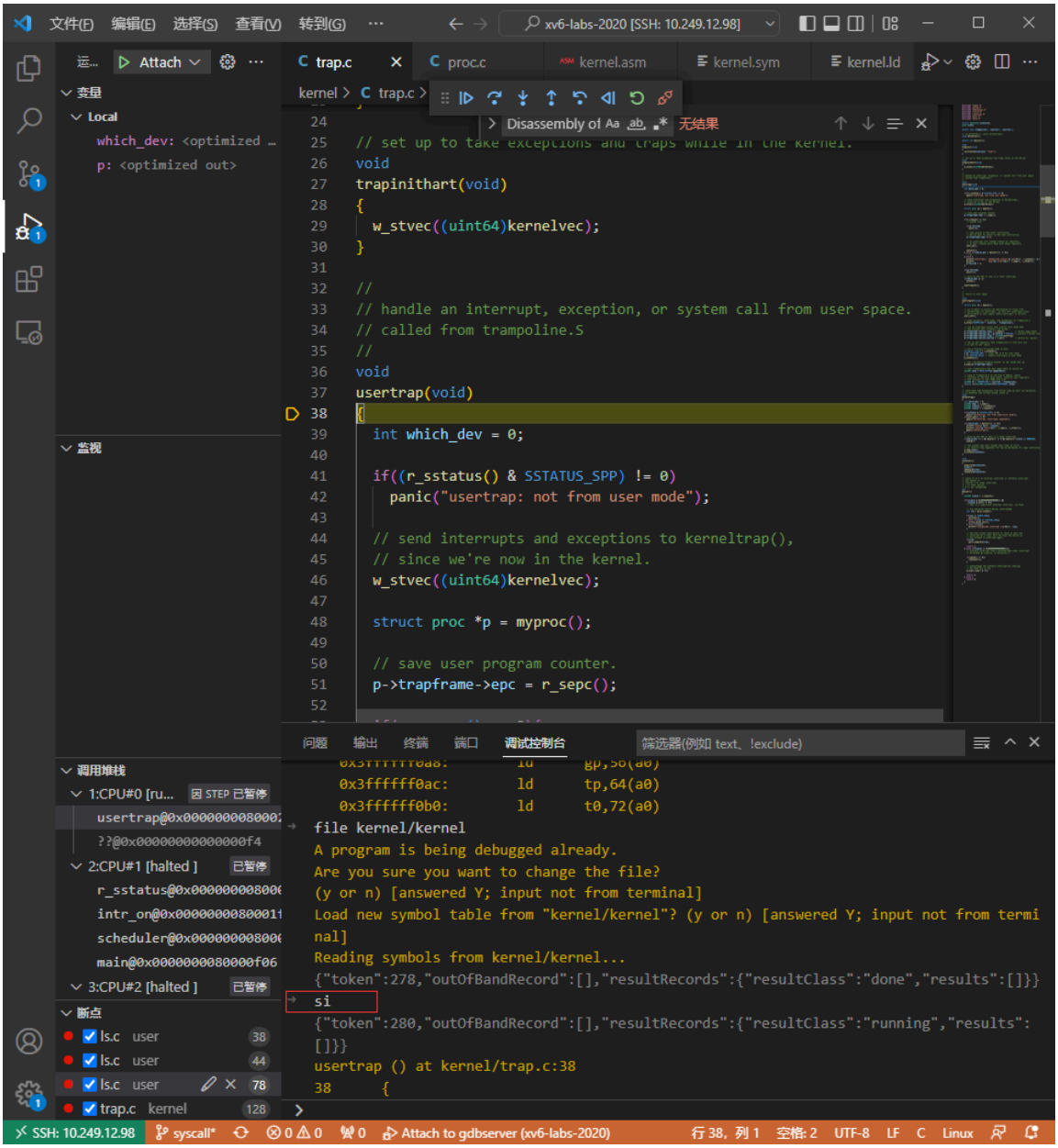
```
问题  输出  终端  端口  调试控制台  筛选器(例如 text、!exclude)
  Breakpoint 7 at 0x3ffffff08e
  {"token":267,"outOfBandRecord":[],"resultRecords":{"resultClass":"done","results":[]}}

  Thread 1 hit Breakpoint 7, 0x0000003ffffff08e in ?? ()
  1: x/10i $pc
  => 0x3ffffff08e:        jr      t0
     0x3ffffff090:        csrw    satp,a1
     0x3ffffff094:        sfence.vma
     0x3ffffff098:        ld      t0,112(a0)
     0x3ffffff09c:        csrw    sscratch,t0
     0x3ffffff0a0:        ld      ra,40(a0)
     0x3ffffff0a4:        ld      sp,48(a0)
     0x3ffffff0a8:        ld      gp,56(a0)
     0x3ffffff0ac:        ld      tp,64(a0)
     0x3ffffff0b0:        ld      t0,72(a0)
>
```

在跳转到内核C语言代码运行之前，加载kernel的调试符号，输入`file kernel/kernel`

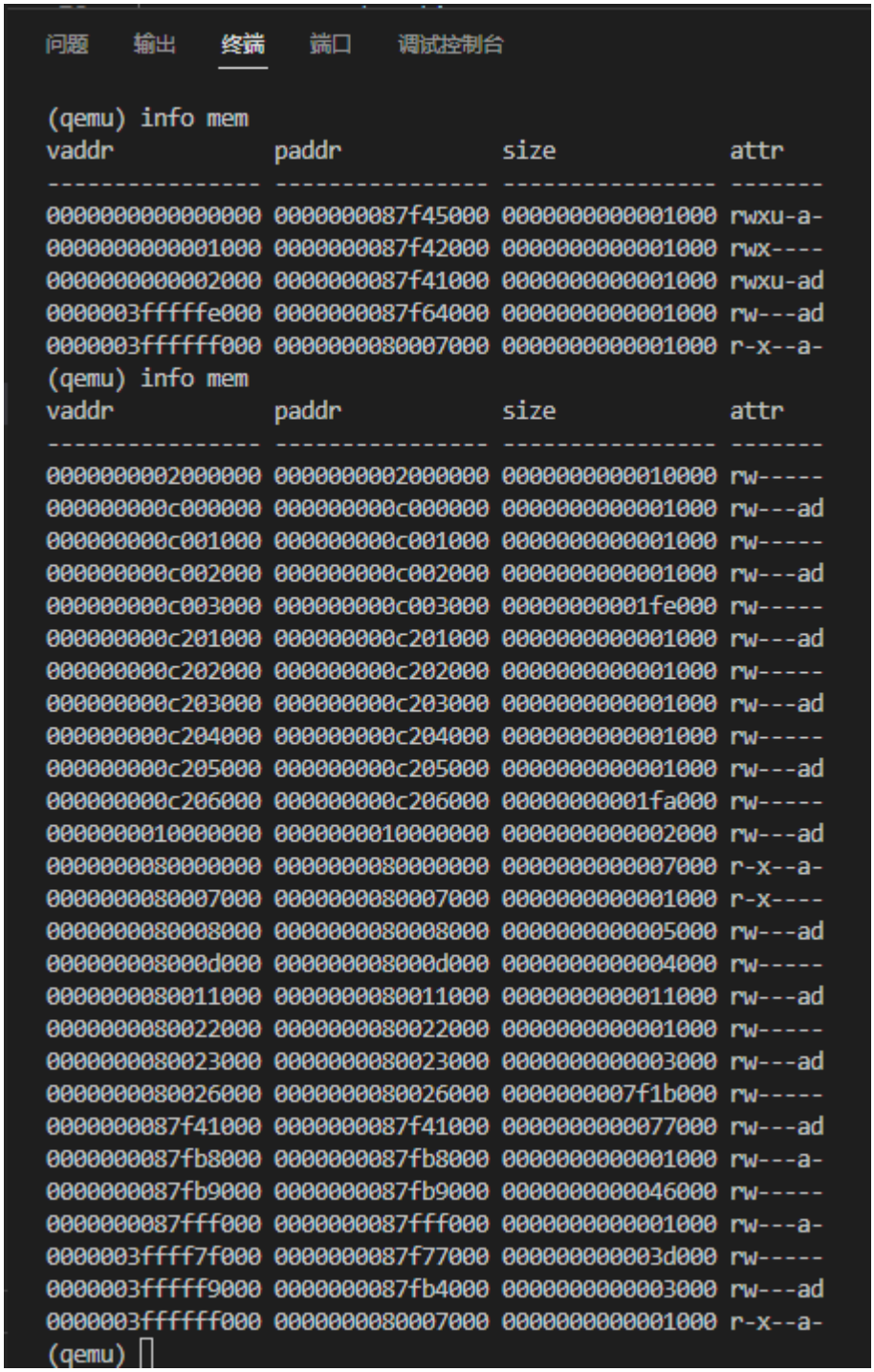
```
→ file kernel/kernel
  A program is being debugged already.
  Are you sure you want to change the file?
  (y or n) [answered Y; input not from terminal]
  Load new symbol table from "kernel/kernel"? (y or n) [answered Y; input not from termi
  nal]
  Reading symbols from kernel/kernel...
  {"token":278,"outOfBandRecord":[],"resultRecords":{"resultClass":"done","results":[]}}
>
```

然后输入 `si` ，按汇编单步执行。



这时可以看到，程序已经进入 `trap.c` 中的 `usertrap` 函数。

这时再回到 QEMU 控制台输入 `info mem` ，观察 page table 的变化：



这时的 page table 已经和之前不同，说明执行流已经从 user page table 切换到了 kernel page table，并准备进入内核中的 C 代码。

`trampoline` 拓展知识

user page table 和 kernel page table 对 `trampoline page` 的映射是相同的（虚拟地址都是 `0x3ffffff000`）。正因为如此，切换页表之后，CPU 仍然能继续在这段代码上执行；否则一旦切页表就可能因为找不到下一条指令而崩溃。这也是 `trampoline page` 的关键作用。

`trapframe` 拓展知识

user page table 中倒数第二个 PTE 的虚拟地址 `0x3ffffffe00` 对应的是 `trapframe page` 。它由内核预先设置好，用来保存当前进程的用户态寄存器。这样一来，内核在进入 C 代码并覆盖寄存器之后，后续仍然可以把这些寄存器恢复回去。

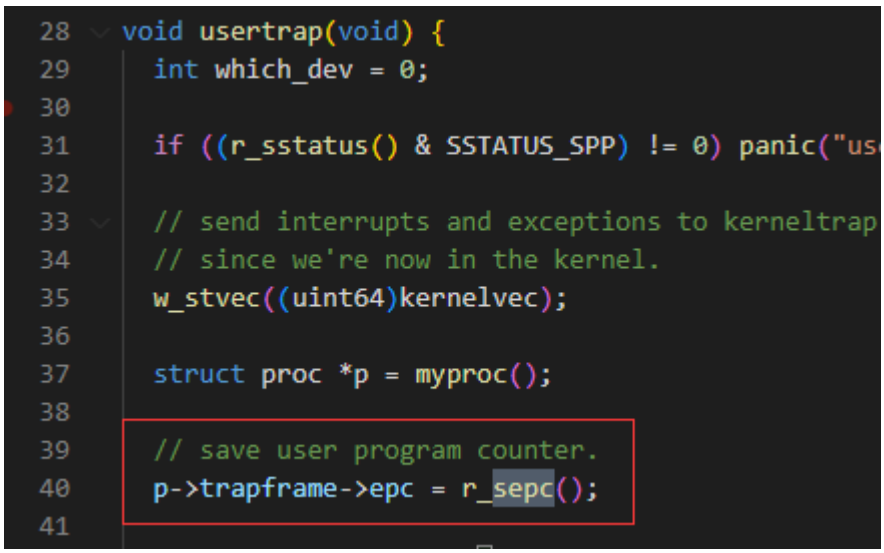
如果你想进一步看里面保存了什么，可以继续查看 `proc.h` 中的 `trapframe` 结构体。

先记住这个结论

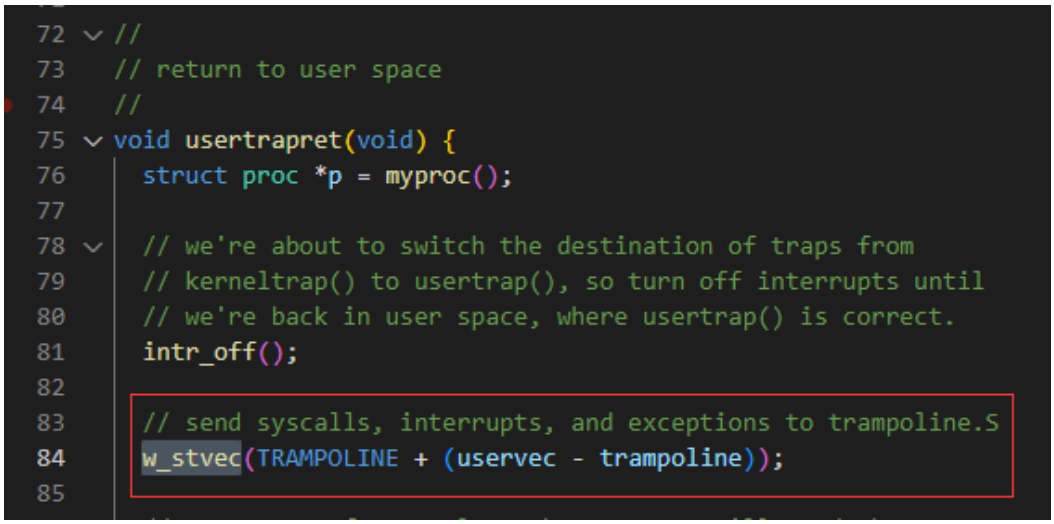
从上面的过程可以先记住两件事。

第一，`ecall` 本身由硬件完成三件关键事情：

1. 把执行模式从 user mode 切到 supervisor mode。
2. 把当前 PC 保存到 `SEPC` 寄存器，也就是保存 `ecall` 在用户空间的位置。之后内核会再把它保存到 `trapframe->epc` 中。



3. 跳转到 `STVEC` 指向的入口。对 xv6 来说，这个入口就是 `trampoline` 中的代码。



第二，在真正进入内核 C 代码之前，`uservec` 这一段汇编还会继续完成三件事：

1. 保存用户寄存器。
2. 从 user page table 切换到 kernel page table。
3. 把 `sp` 切到 `kernel stack`，为后续 C 代码准备好栈。

其中，`kernel satp` 和 `kernel stack` 都是在 `usertrapret` 中提前准备好的。

```
72  //
73  // return to user space
74  //
75  void usertrapret(void) {
76    struct proc *p = myproc();
77
78    // we're about to switch the destination of traps from
79    // kerneltrap() to usertrap(), so turn off interrupts until
80    // we're back in user space, where usertrap() is correct.
81    intr_off();
82
83    // send syscalls, interrupts, and exceptions to trampoline.S
84    w_stvec(TRAMPOLINE + (uservec - trampoline));
85
86    // set up trapframe values that uservec will need when
87    // the process next re-enters the kernel.
88    p->trapframe->kernel_satp = r_satp();          // kernel page table
89    p->trapframe->kernel_sp = p->kstack + PGSIZE;  // process's kernel stack
90    p->trapframe->kernel_trap = (uint64)usertrap;
91    p->trapframe->kernel_hartid = r_tp();  // hartid for cpuid()
92
```

3. 汇编之后的 C 代码

看到 `usertrap` 之后，执行流就已经进入内核的 C 代码了。对大多数同学来说，这一段会比前面的汇编更容易继续跟。

如果你现在主要关心系统调用本身，可以在 `syscall();` 这一行打断点，然后点击“运行”，让程序停在 `syscall()` 函数处。想继续深入的话，也可以顺着 `usertrap()` 往下分析。

接下来进入 `syscall.c` 里的 `syscall()` ，就可以继续按普通的 VSCode 调试方式往下看。

4. 一个简单的从用户到内核空间的调试方法

如果你现在不想先跟完整段汇编，而是想更快地停到内核里的 C 代码，可以按下面这组步骤做。下面仍以 `ls` 中的 `fstat` 系统调用为例。

Step1： 在终端执行 `make qemu-gdb` ，然后在 VSCode 中按 `F5` 连接。

Step2： 点击“运行”，让 xv6 正常启动，直到 shell 出现 `$` 提示符。

Step3： 在调试控制台输入 `interrupt` ，先把执行流停住。

Step4： 在 `kernel/trap.c:128` 打断点，再继续运行。

Step5： 在 xv6 的 shell 中输入 `ls` ，程序会停在 `kernel/trap.c:128` 处。

Step6： 删除原有内核态断点，并在调试控制台输入 `file user/_ls` ，切换到 `ls` 的调试符号。

Step7： 打开 `user/ls.c` ，在 `main()` 第 78 行打断点，点击“运行”。

Step8： 再在 `ls.c` 第 38 行打断点，继续运行，让程序停在执行 `fstat` 系统调用之前。

Step9： 在调试控制台输入 `file kernel/kernel` ，切回内核调试符号。

Step10： 在 `syscall.c` 中 `syscall();` 所在位置打断点，继续运行。程序就会停在系统调用对应的内核 C 代码处。

到这里，你就可以根据自己的需要，继续分析后续执行流程了。